

*To my family, and to everyone who discussed a factor graph with me
at improbable hours across improbable time zones.*

Contents

1	Introduction	10
1.1	Topic and motivation	10
1.2	Positioning and gap	11
1.3	Research questions	12
1.4	Contributions	13
1.5	Methodology	14
1.6	Structure of the thesis	15
2	From Hamiltonian Mechanics to Statistical Mechanics	16
2.1	Hamiltonian mechanics and phase space	16
2.2	Liouville’s theorem	18
2.3	From dynamics to ensembles	19
2.3.1	The microcanonical ensemble	19
2.3.2	The canonical ensemble and the Boltzmann distribution	20
2.4	The partition function and free energy	22
2.5	Interacting systems: the Ising paradigm	24
2.6	Complex systems, and what this thesis takes from physics	25
3	Stochastic Processes and Markov Chain Monte Carlo	26
3.1	Markov chains: kernels, stationarity, reversibility	27
3.2	The AR(1) chain: the running example of the thesis	28
3.3	Markov chain Monte Carlo	30

3.3.1	The Metropolis–Hastings algorithm	30
3.3.2	Gibbs sampling	31
3.3.3	Langevin dynamics: MCMC without rejections	32
3.4	From algorithmic chains to data chains	32
4	Stochastic Differential Equations, Itô Calculus, and the Fokker–Planck	
	Picture	33
4.1	From ODEs to SDEs	33
4.2	The Ornstein–Uhlenbeck process: the corruption channel	35
4.3	The Fokker–Planck equation: transporting densities	37
4.3.1	Stationary law of Langevin dynamics: closing the loop	38
4.4	Time reversal: the mother theorem of diffusion models	39
5	Energy-Based Models, Markov Random Fields, and Factor Graphs	41
5.1	Energy-based models	41
5.2	Markov random fields	42
5.3	Hidden Markov models: chains observed through noise	43
5.4	Factor graphs	44
5.5	What learning adds, and why we go exact	45
6	Diffusion Models: from Thermodynamics to Scores	46
6.1	The generative problem and the diffusion idea	46
6.2	Denoising diffusion probabilistic models	47
6.3	Score matching	48
6.4	Tweedie’s identity: the score is a denoiser	50
6.5	The statistical physics of generative diffusion	51
6.6	The gap, restated	52
7	The Gaussian Chain Solved Exactly	53
7.1	The model, and the object of study	53
7.2	The clean joint law: dense covariance, sparse precision	54

7.3	The noisy joint law and the score, two ways	56
7.3.1	Route one: linear algebra	56
7.3.2	Route two: Bayesian inference	57
7.3.3	$K = 2$: the smallest example, fully explicit	58
7.4	The lifecycle of the precision matrix	58
7.4.1	The band-fill law at small t , corrected	60
7.4.2	Return to isotropy at large t	61
7.5	The bulk of a long chain, in closed form	62
7.5.1	The locality law	64
7.6	Summary of the chapter	65
8	Beyond Gaussianity: Laplace Innovations	67
8.1	The $K = 1$ warm-up: one frame, no chain	68
8.1.1	The score becomes nonlinear	69
8.1.2	The Fourier representation: convolution becomes product	70
8.1.3	The curvature becomes a field	71
8.2	The Laplace chain, $K \geq 2$	72
8.2.1	The Markov-structured form of the joint score	72
8.2.2	The boundary message in closed form, and where closed forms die	73
8.2.3	Innovation coordinates: the coupling has to live somewhere	74
8.3	What survives, what is open	75
9	The Score as Posterior Expectation: Belief Propagation and the Gaussian Closure	76
9.1	The computational problem	77
9.1.1	The factor graph is a tree	77
9.1.2	Sum-product, and the continuous-message obstruction	78
9.2	Belief propagation on the chain	78
9.3	The Gaussian case: closure, Kalman, machine precision	79
9.3.1	The bulk fixed point: BP never breaks on the chain	82

9.4	AMP/TAP: same score, different variance, exact breakdown	83
9.5	The Gaussian closure beyond Gaussianity	87
9.5.1	Findings on the Laplace chain	88
9.5.2	Varying the innovation: what makes closure hard	88
9.5.3	Truncating the matrix versus truncating the inference	89
9.6	What this chapter established	90
10	Discussion and Conclusions	91
10.1	Summary of findings, by research question	91
10.2	Contributions to the literature	93
10.3	Limitations	93
10.4	Future work	94
10.5	Closing remark	94
A	Gaussian Identities Used Throughout	95
A.1	Quadratic forms and Gaussian normalisation	95
A.2	Products of Gaussians in the same variable	96
A.3	Affine propagation (marginalisation)	96
A.4	Gaussian conditioning	96
A.5	Woodbury and the two routes to the score	97
B	Code Listings	98
B.1	Gaussian chain: model and matrix-route score	98
B.2	Gaussian chain: belief propagation (Convention A)	103
B.3	Closed-form bulk formulas	107
B.4	AMP/TAP on the chain	112
B.5	Local (truncated-range) estimators	117
B.6	Non-Gaussian chains: exact grid BP	120
B.7	Non-Gaussian chains: Gaussian-projected BP	124
B.8	Exact scores for validation	128

C	Reproducibility	130
C.1	Repository layout	130
C.2	Audit protocol	131
C.3	Figure-to-code map	132
C.4	Environment	132

Chapter 1

Introduction

1.1 Topic and motivation

Generative diffusion models have become the dominant paradigm for synthesising high-dimensional data: they power state-of-the-art systems for images, audio, video, and molecular structures (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021; Karras et al., 2022). Their central object is the *score function* — the gradient of the log-density of progressively noised data,

$$S(x, t) = \nabla_x \log p_t(x), \quad (1.1)$$

which is all a model needs to know in order to reverse a noising process and turn pure noise back into data (Anderson, 1982). In practice the score is approximated by a neural network trained by denoising score matching (Hyvärinen, 2005; Vincent, 2011); the network is a black box, and the structure of what it has to learn remains poorly understood.

A complementary research programme, initiated in the statistical-physics community, replaces the black box with *exactly solvable models*: data distributions simple enough that the score can be computed in closed form, yet rich enough to exhibit the phenomena observed in real systems — speciation, memorisation, and dynamical phase transitions along the generation trajectory (Biroli and Mézard, 2023; Biroli et al., 2024; Raya and Ambrogioni, 2023; Ghio et al., 2024). Exact scores act as a microscope: every regime,

transition time, and failure mode can be traced back to explicit formulas.

This thesis extends the exactly solvable programme in a direction that is natural for data with *temporal or sequential structure*: the data are not a cloud of independent points but a *trajectory* $a_0, a_1, \dots, a_{K-1}$ of a Markov process — the prototype of a video, a time series, or any dynamic object. Each frame of the trajectory is corrupted independently by an Ornstein–Uhlenbeck (OU) channel, and the object of study is the *joint* score of the whole noisy sequence. Two classical bodies of theory then meet the modern one. First, the joint score of a Markov chain turns out to be governed by the algebra of structured matrices — Toeplitz covariances, tridiagonal precisions, and their spectral properties. Second, computing the score is an exercise in Bayesian inference on a tree-shaped factor graph, the home turf of *belief propagation* (BP) and its Gaussian projection, *approximate message passing* (AMP) (Pearl, 1988; Mézard and Montanari, 2009; Donoho et al., 2009). The thesis develops both routes in full detail, proves where they coincide, and measures precisely what is lost when they are forced apart.

1.2 Positioning and gap

The literature this work builds on can be organised in three strands.

Statistical physics of diffusion models. Closed-form analyses of diffusion generation exist for Gaussian mixtures, random energy models, and high-dimensional asymptotics (Biroli and Mézard, 2023; Biroli et al., 2024; Ghio et al., 2024). These works treat the data as unstructured collections of samples; the time axis they study is the *diffusion* time. The internal time of a *sequence* — the fact that a video frame is correlated with its neighbours through a dynamical rule — is absent from these analyses.

Inference on chains. Exact posterior inference on Gauss–Markov chains is classical: the Kalman filter and smoother (Kalman, 1960; Rauch et al., 1965), hidden Markov models (Rabiner, 1989), and their unifying formulation as sum–product message passing on factor graphs (Kschischang et al., 2001; Mézard and Montanari, 2009). What this literature does

not ask is the question that matters for generative modelling: *how does the exact posterior — and therefore the exact score — deform as a function of the diffusion time*, and what does that deformation imply for the architecture of a network that must learn it?

Message passing with continuous variables. BP is exact on trees for discrete variables, where messages are finite vectors of probabilities. For continuous variables the messages become probability *densities* — infinite-dimensional objects — and practical algorithms must close them within a parametric family, almost always the Gaussian one; this is the step that turns BP into AMP, known in physics as the Thouless–Anderson–Palmer (TAP) approach (Thouless et al., 1977; Donoho et al., 2009; Zdeborová and Krzakala, 2016). The classical justification of the Gaussian closure is a central-limit argument valid when every node receives many messages. On a chain each node receives exactly two: the central-limit argument fails by construction, and the quality of the Gaussian closure on chain-structured diffusion posteriors was, to the best of our knowledge, unquantified.

The gap, in one sentence: *there is no end-to-end, exactly solvable account of the joint diffusion score of a structured sequence — its matrix structure, its message-passing computation, and the price of the Gaussian closure when the sequence is not Gaussian.*

1.3 Research questions

The thesis addresses four questions.

- RQ1.** What is the exact joint score of a Markov chain corrupted by an OU channel, and how does its structure — concretely, the posterior precision matrix — evolve along the diffusion time, from the sparse Markov regime at $t \rightarrow 0$ to the isotropic regime at $t \rightarrow \infty$?
- RQ2.** Can the joint score be computed *locally*, by message passing on the chain’s factor graph, rather than globally by matrix inversion? In the Gaussian case, how does belief propagation relate to the classical Kalman smoother, and does it reproduce the closed-form score exactly?

- RQ3.** When the messages are forced into a Gaussian family — the AMP/TAP closure — what survives and what breaks? On the Gaussian chain, is the closure exact; and on a non-Gaussian (Laplace-innovation) chain, how large is the closure error as a function of the diffusion time?
- RQ4.** What do the exact results imply for *learned* scores: when is a local (banded, convolution-like) score architecture sufficient, and what is the exact law governing the error of local truncations?

1.4 Contributions

The main contributions, each developed in a dedicated chapter, are the following.

1. **A complete closed-form solution of the Gaussian benchmark** (Chapter 7). For the stationary AR(1) chain under OU corruption we derive the joint score, diagonalise it in closed form, and characterise the full lifecycle of the posterior precision matrix: a corrected *band-fill law* for the small- t growth of its off-diagonals, and exact bulk formulas for its inverse — variance, geometric decay rate $q(\alpha, t)$ of posterior correlations, and their limits. The band-fill law corrects a transcription error in an earlier working note; the correction is validated numerically.
2. **An equation-by-equation message-passing construction of the score** (Chapter 9). We build the factor graph of the noisy chain, specialise the sum-product equations to it, prove that Gaussian messages are closed under the updates (two elementary facts suffice), and show that the resulting algorithm — Gaussian BP, algebraically identical to a Kalman smoother — reproduces the matrix score to machine precision at cost $O(K)$ instead of $O(K^3)$.
3. **An exact phase boundary for the AMP closure** (Chapter 9). On the chain, BP and AMP produce the *same score* but *different posterior variances*. We show the difference is a single factor of two in a closure discriminant ($J_d^2 - 4\beta^2$ for BP

against $J_d^2 - 8\beta^2$ for AMP), prove that the BP fixed point exists for every coupling and diffusion time, and derive the exact AMP breakdown time $t_c(\alpha)$ together with the critical coupling $\alpha_c = \sqrt{2} - 1$ below which AMP never breaks down.

4. **A quantitative audit of the Gaussian closure beyond Gaussianity** (Chapters 8 and 9). For Laplace-innovation chains — where exact inference is still available through deterministic grid quadrature on the same factor graph — we measure the score error of Gaussian-projected BP: it peaks around 20% (relative L^2) at small diffusion time and decays below 1% as the noisy marginals Gaussianise, while the score *direction* remains accurate throughout. We also show that at small t *truncating the inference* (local BP sweeps) is markedly more accurate than *truncating the score matrix* (banding), quantifying the advantage of local message passing over local linear algebra.
5. **A reproducible computational companion.** All results are backed by open, dependency-light Python code with deterministic numerical audits (72/72 and 55/55 independent checks, re-executed at build time); the code and the figures it generates are part of the thesis repository and are listed in Appendix B.

1.5 Methodology

The methodology is deliberately old-fashioned: *no neural networks, no black boxes — closed forms first, numerics as audit*. Every claim is either proved by explicit algebra (the derivations are spelled out step by step, with the longer algebraic passages isolated in numbered *toolboxes*) or verified by deterministic numerical checks against brute-force ground truth, in the spirit of the exactly solvable programme of Biroli and Mézard (2023). Where an approximation is introduced (the Gaussian closure of BP messages, local truncations), its error is measured against an exact reference computed by an independent method (matrix algebra or dense grid quadrature). The empirical setting, the audit protocol, and the software packages are described alongside the results and in Appendix C.

1.6 Structure of the thesis

The thesis is organised in two movements: a foundations movement (Chapters 2–6) that builds, from first principles, every tool the research movement needs, and a research movement (Chapters 7–9) that contains the original contributions.

Chapter 2 develops statistical mechanics from Hamiltonian mechanics: phase space, Liouville’s theorem, the Boltzmann–Gibbs distribution, free energy, and the Ising paradigm of complex systems. Chapter 3 covers stochastic processes and Markov-chain Monte Carlo: stationarity, detailed balance, Metropolis–Hastings and Gibbs sampling, and introduces the AR(1) chain that serves as the running example of the whole thesis. Chapter 4 builds the continuous-time toolkit: Brownian motion, the Itô integral and Itô’s lemma, the Ornstein–Uhlenbeck process — the corruption channel of diffusion models — and the Fokker–Planck equation, closing the loop with the Boltzmann distribution of Chapter 2. Chapter 5 treats energy-based models, Markov random fields, hidden Markov models, and factor graphs — the representational language of the research chapters. Chapter 6 assembles these pieces into modern diffusion models: the DDPM construction, the score-SDE formulation, score matching, Tweedie’s identity, and the statistical-physics analyses of generative diffusion that motivate this work.

Chapter 7 solves the Gaussian chain exactly (RQ1). Chapter 8 leaves Gaussianity in the most controlled way possible — Laplace innovations — and shows what the joint score looks like when it is no longer linear (first half of RQ3). Chapter 9 recomputes everything by message passing: BP, its Gaussian specialisation and the Kalman connection (RQ2), the AMP closure and its exact phase boundary, the closure error on non-Gaussian chains (RQ3), and the locality laws that answer RQ4. Chapter 10 summarises the findings per research question, discusses limitations, and lays out the research programme that follows from them. Three appendices collect Gaussian identities, the code listings, and the reproducibility protocol.

Chapter 2

From Hamiltonian Mechanics to Statistical Mechanics

Statistical mechanics is the discipline that explains how probabilistic laws emerge from deterministic dynamics. It is also, less obviously, the intellectual ancestor of generative diffusion models: the Boltzmann–Gibbs distribution derived in this chapter is the stationary law of the Langevin dynamics of Chapter 4, the energy functions introduced here become the energy-based models of Chapter 5, and the very idea of *sampling by simulating a physical process* — central to Chapters 3 and 6 — is statistical mechanics turned into an algorithm. We therefore begin at the beginning: with Hamilton’s equations. The presentation follows classical references (Goldstein et al., 2002; Landau and Lifshitz, 1980), with the derivations that matter for the rest of the thesis carried out in full.

2.1 Hamiltonian mechanics and phase space

Consider a mechanical system with n degrees of freedom, described by generalised coordinates $q = (q_1, \dots, q_n)$ and conjugate momenta $p = (p_1, \dots, p_n)$. Its dynamics is generated by a single scalar function, the *Hamiltonian* $H(q, p)$, which for the systems considered

here has the mechanical form

$$H(q, p) = \underbrace{\sum_{i=1}^n \frac{p_i^2}{2m_i}}_{\text{kinetic}} + \underbrace{U(q_1, \dots, q_n)}_{\text{potential}}, \quad (2.1)$$

and equals the total energy of the system. The state of the system at any instant is the pair (q, p) , a point in the $2n$ -dimensional *phase space* $\Gamma \subseteq \mathbb{R}^{2n}$. The evolution is given by *Hamilton's equations*,

$$\dot{q}_i = \frac{\partial H}{\partial p_i}, \quad \dot{p}_i = -\frac{\partial H}{\partial q_i}, \quad i = 1, \dots, n. \quad (2.2)$$

Two structural properties of (2.2) are the pillars on which statistical mechanics rests.

Energy conservation. Along any trajectory,

$$\frac{dH}{dt} = \sum_{i=1}^n \left(\frac{\partial H}{\partial q_i} \dot{q}_i + \frac{\partial H}{\partial p_i} \dot{p}_i \right) = \sum_{i=1}^n \left(\frac{\partial H}{\partial q_i} \frac{\partial H}{\partial p_i} - \frac{\partial H}{\partial p_i} \frac{\partial H}{\partial q_i} \right) = 0. \quad (2.3)$$

The motion is therefore confined to the *energy shell* $\Gamma_E = \{(q, p) : H(q, p) = E\}$, a $(2n - 1)$ -dimensional hypersurface.

Incompressibility of the phase flow. The phase-space velocity field

$$v(q, p) = (\dot{q}, \dot{p}) = \left(\frac{\partial H}{\partial p}, -\frac{\partial H}{\partial q} \right) \quad (2.4)$$

has identically vanishing divergence:

$$\nabla \cdot v = \sum_{i=1}^n \left(\frac{\partial \dot{q}_i}{\partial q_i} + \frac{\partial \dot{p}_i}{\partial p_i} \right) = \sum_{i=1}^n \left(\frac{\partial^2 H}{\partial q_i \partial p_i} - \frac{\partial^2 H}{\partial p_i \partial q_i} \right) = 0. \quad (2.5)$$

Hamiltonian flow behaves like an incompressible fluid in phase space: it can stretch and fold a region, but never change its volume. This innocuous-looking identity is the seed of the entire probabilistic edifice, as we now show.

2.2 Liouville's theorem

A single trajectory is useless for macroscopic physics: for a gas, $n \sim 10^{23}$ and neither the initial condition nor the solution of (2.2) is accessible. The move that founds statistical mechanics is to describe our *ignorance* of the microstate by a probability density $\rho(q, p, t)$ on phase space: $\rho(q, p, t) dq dp$ is the probability that the system is in the infinitesimal cell around (q, p) at time t . Liouville's theorem states how this density evolves.

Theorem 2.1 (Liouville). *Under the Hamiltonian flow (2.2), the phase-space density obeys*

$$\frac{\partial \rho}{\partial t} = -\{\rho, H\} = \sum_{i=1}^n \left(\frac{\partial H}{\partial q_i} \frac{\partial \rho}{\partial p_i} - \frac{\partial H}{\partial p_i} \frac{\partial \rho}{\partial q_i} \right), \quad (2.6)$$

where $\{\cdot, \cdot\}$ is the Poisson bracket. Equivalently, the density is constant along trajectories: $d\rho/dt = 0$.

Toolbox 2.0 — Proof of Liouville's theorem, step by step

The density is locally conserved — probability mass is neither created nor destroyed, only transported by the flow — so it satisfies the continuity equation familiar from fluid dynamics:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho v) = 0, \quad v = (\dot{q}, \dot{p}).$$

Expand the divergence of the product using $\nabla \cdot (\rho v) = v \cdot \nabla \rho + \rho (\nabla \cdot v)$:

$$\frac{\partial \rho}{\partial t} + \underbrace{\sum_{i=1}^n \left(\dot{q}_i \frac{\partial \rho}{\partial q_i} + \dot{p}_i \frac{\partial \rho}{\partial p_i} \right)}_{v \cdot \nabla \rho} + \rho \underbrace{\sum_{i=1}^n \left(\frac{\partial \dot{q}_i}{\partial q_i} + \frac{\partial \dot{p}_i}{\partial p_i} \right)}_{\nabla \cdot v} = 0.$$

By the incompressibility identity (2.5) the last term vanishes. Substituting Hamilton's equations $\dot{q}_i = \partial H / \partial p_i$, $\dot{p}_i = -\partial H / \partial q_i$ into the middle term:

$$\frac{\partial \rho}{\partial t} + \sum_{i=1}^n \left(\frac{\partial H}{\partial p_i} \frac{\partial \rho}{\partial q_i} - \frac{\partial H}{\partial q_i} \frac{\partial \rho}{\partial p_i} \right) = 0 \quad \Longleftrightarrow \quad \frac{\partial \rho}{\partial t} = -\{\rho, H\}.$$

Finally, the total (convective) derivative along a trajectory is

$$\frac{d\rho}{dt} = \frac{\partial\rho}{\partial t} + v \cdot \nabla\rho = \frac{\partial\rho}{\partial t} + \{\rho, H\} = 0,$$

which is the second form of the statement: an observer riding a phase-space trajectory sees a constant local density. $\square$

Liouville's theorem has an immediate and profound consequence: *any density that is a function of conserved quantities only is stationary*. If $\rho(q, p) = f(H(q, p))$ for some function f , then $\{\rho, H\} = f'(H)\{H, H\} = 0$, so $\partial\rho/\partial t = 0$. Equilibrium statistical mechanics is the study of which f nature chooses.

2.3 From dynamics to ensembles

An *ensemble* is a probability distribution over microstates that is stationary under the dynamics and compatible with the macroscopic constraints we impose (fixed energy, fixed temperature, ...). Two ensembles matter for this thesis.

2.3.1 The microcanonical ensemble

For a perfectly isolated system, the energy is fixed at E and Liouville's theorem suggests the simplest choice compatible with it: a uniform density on the energy shell,

$$\rho_{\text{mc}}(q, p) = \frac{1}{\Omega(E)} \delta(H(q, p) - E), \quad \Omega(E) = \int_{\Gamma} \delta(H(q, p) - E) dq dp, \quad (2.7)$$

where $\Omega(E)$ counts the microstates at energy E . The postulate that all accessible microstates are equally likely — the *equal a priori probability postulate* — is motivated (not proved) by ergodicity: for ergodic dynamics, time averages along a single trajectory converge to averages over the shell, so the uniform measure is the one a long experiment

effectively samples. Boltzmann's entropy attaches a number to this count,

$$S(E) = k_B \log \Omega(E), \quad (2.8)$$

with k_B the Boltzmann constant. Entropy is the logarithm of the number of microscopic ways a macroscopic state can be realised.

2.3.2 The canonical ensemble and the Boltzmann distribution

Isolated systems are an idealisation; the physically ubiquitous situation is a system exchanging energy with a much larger environment (a *heat bath*) at temperature T . The stationary distribution of the system alone is then no longer uniform: it is the *Boltzmann–Gibbs distribution*,

$$\boxed{\rho_\beta(x) = \frac{1}{Z(\beta)} e^{-\beta H(x)}, \quad Z(\beta) = \int e^{-\beta H(x)} dx, \quad \beta = \frac{1}{k_B T},} \quad (2.9)$$

where $x = (q, p)$ collects the microstate and $Z(\beta)$, the *partition function*, normalises the density. Equation (2.9) can be derived in two independent ways, and both derivations are instructive enough to be carried out in full: the first (physical) traces the exponential to the entropy of the bath; the second (information-theoretic) characterises it as the least-committal distribution compatible with a fixed mean energy, a perspective due to Jaynes (1957) that returns, almost verbatim, in modern machine learning.

Toolbox 2.1 — The Boltzmann distribution from the heat bath

Let the total system = (system S) $\cup$ (bath B) be isolated with total energy E_{tot} , and let the coupling be weak, so $H_{\text{tot}} = H_S + H_B$. Apply the microcanonical postulate to the total system: the probability that S is in a *specific* microstate x with energy $H_S(x) = \epsilon$ is proportional to the number of bath microstates carrying the remaining energy,

$$\mathbb{P}(x) \propto \Omega_B(E_{\text{tot}} - \epsilon) = \exp\left[\frac{1}{k_B} S_B(E_{\text{tot}} - \epsilon)\right],$$

using the definition (2.8) of the bath entropy. Since the bath is huge, $\epsilon \ll E_{\text{tot}}$, expand S_B to first order:

$$S_B(E_{\text{tot}} - \epsilon) = S_B(E_{\text{tot}}) - \epsilon \left. \frac{\partial S_B}{\partial E} \right|_{E_{\text{tot}}} + O(\epsilon^2/E_{\text{tot}}).$$

The derivative of the bath entropy with respect to its energy *defines* the bath temperature, $\partial S_B/\partial E = 1/T$. Substituting,

$$\mathbb{P}(x) \propto \underbrace{e^{S_B(E_{\text{tot}})/k_B}}_{\text{constant in } x} \cdot e^{-\epsilon/(k_B T)} \propto e^{-\beta H_S(x)}.$$

The quadratic remainder is suppressed by the bath's heat capacity and vanishes in the thermodynamic limit. Normalising yields (2.9). Note what happened: the exponential form is nothing but the first-order Taylor expansion of the bath's entropy — the Boltzmann factor is the price, in bath microstates, of lending energy ϵ to the system.

□

Toolbox 2.2 — The Boltzmann distribution from maximum entropy

Forget physics; ask, with Jaynes (1957): among all densities ρ with a given mean energy $\mathbb{E}_\rho[H] = U$, which one assumes the least beyond the constraint? “Least” is measured by the Gibbs–Shannon entropy $S[\rho] = -k_B \int \rho \log \rho$. Maximise $S[\rho]$ subject to $\int \rho = 1$ and $\int \rho H = U$ with Lagrange multipliers λ_0, λ_1 :

$$\mathcal{L}[\rho] = -k_B \int \rho \log \rho \, dx - \lambda_0 \left(\int \rho \, dx - 1 \right) - \lambda_1 \left(\int \rho H \, dx - U \right).$$

Take the functional derivative and set it to zero pointwise:

$$\frac{\delta \mathcal{L}}{\delta \rho(x)} = -k_B (\log \rho(x) + 1) - \lambda_0 - \lambda_1 H(x) = 0 \implies \rho(x) = \exp \left[-1 - \frac{\lambda_0}{k_B} - \frac{\lambda_1}{k_B} H(x) \right].$$

The density is an exponential of the constrained quantity — this is completely general: constraining $\mathbb{E}[\phi_j(x)]$ for features ϕ_j produces $\rho \propto \exp[-\sum_j \beta_j \phi_j(x)]$, the exponen-

tial family. The multiplier λ_0 is fixed by normalisation and produces Z ; renaming $\lambda_1/k_B = \beta$ and identifying $\beta = 1/(k_B T)$ through thermodynamics recovers (2.9) exactly. Second-order check: $\delta^2 \mathcal{L}/\delta \rho^2 = -k_B/\rho < 0$, so the stationary point is the unique maximum (the entropy is strictly concave). $\square$

The second derivation deserves a remark that will echo through the thesis: the Boltzmann distribution is not a statement about molecules but about *inference under constraints*. Any time we posit a model of the form $p(x) \propto e^{-E(x)}$ — as we will for Markov random fields and energy-based models in Chapter 5 — we are doing statistical mechanics, whether the variables are spins, pixels, or the frames of a video.

2.4 The partition function and free energy

The partition function $Z(\beta)$ in (2.9) looks like a mere normaliser; it is in fact the central computational object of the theory — and the reason approximate inference exists as a field. Its logarithm defines the *Helmholtz free energy*,

$$F(\beta) = -\frac{1}{\beta} \log Z(\beta). \quad (2.10)$$

Toolbox 2.3 — Everything is a derivative of $\log Z$

Differentiate $\log Z$ with respect to β , carrying the derivative through the integral:

$$-\frac{\partial \log Z}{\partial \beta} = -\frac{1}{Z} \frac{\partial}{\partial \beta} \int e^{-\beta H} = \frac{\int H e^{-\beta H}}{\int e^{-\beta H}} = \mathbb{E}_{\rho_\beta}[H] = U,$$

the mean energy. Differentiate once more:

$$\begin{aligned} \frac{\partial^2 \log Z}{\partial \beta^2} &= \frac{\partial}{\partial \beta} \left(-\frac{\int H e^{-\beta H}}{Z} \right) = \frac{\int H^2 e^{-\beta H}}{Z} - \left(\frac{\int H e^{-\beta H}}{Z} \right)^2 \\ &= \mathbb{E}[H^2] - \mathbb{E}[H]^2 = \text{Var}(H) \geq 0. \end{aligned}$$

So $\log Z$ is convex in β , its first derivative gives the mean energy, its second the energy fluctuations; the latter equals $k_B T^2 C_V$ with C_V the heat capacity — a *fluctuation–dissipation* relation: how a system responds to heating is determined by its spontaneous equilibrium fluctuations. Finally, rewriting the Gibbs–Shannon entropy of ρ_β :

$$S[\rho_\beta] = -k_B \mathbb{E}[\log \rho_\beta] = -k_B \mathbb{E}[-\beta H - \log Z] = \frac{U}{T} + k_B \log Z,$$

and multiplying by $-T$: $-TS = -U + F$, i.e.

$$F = U - TS.$$

The free energy is the exact trade-off between energy and entropy: at low T minimising F means minimising energy (ordered states win), at high T it means maximising entropy (disordered states win). Phase transitions are the non-analyticities of F where the winner changes. □

There is one more characterisation of F worth recording, because it is the statistical-mechanics form of the variational principle that underlies both the ELBO of diffusion models (Chapter 6) and the Bethe/BP free energies (Chapter 9). For *any* trial density μ ,

$$\underbrace{\mathbb{E}_\mu[H] - \frac{1}{\beta} S[\mu]/k_B}_{\text{variational free energy } F[\mu]} = F + \frac{1}{\beta} D_{\text{KL}}(\mu \parallel \rho_\beta) \geq F, \quad (2.11)$$

with equality iff $\mu = \rho_\beta$: the Boltzmann distribution is the unique minimiser of the variational free energy, and the gap is exactly a Kullback–Leibler divergence. Every variational method in machine learning is a restriction of (2.11) to a tractable family of μ 's.

2.5 Interacting systems: the Ising paradigm

Everything so far holds for arbitrary H ; the physics (and the difficulty) enters when the energy couples many degrees of freedom. The canonical example is the *Ising model* (Ising, 1925): binary spins $\sigma_i \in \{-1, +1\}$ on the nodes of a graph $G = (V, \mathcal{E})$, with energy

$$H(\sigma) = - \sum_{(i,j) \in \mathcal{E}} J_{ij} \sigma_i \sigma_j - \sum_{i \in V} h_i \sigma_i, \quad (2.12)$$

where J_{ij} are pairwise couplings and h_i external fields. The Gibbs distribution $p(\sigma) \propto e^{-\beta H(\sigma)}$ makes neighbouring spins prefer alignment when $J_{ij} > 0$. Three lessons from this model shape the rest of the thesis.

(i) The partition function is the hard part. $Z = \sum_{\sigma} e^{-\beta H(\sigma)}$ runs over $2^{|V|}$ configurations. On two-dimensional lattices Onsager computed it exactly (Onsager, 1944); on general graphs the computation is #P-hard, and *all* of approximate inference — MCMC (Chapter 3), variational methods, message passing (Chapter 9) — exists because of this wall. On *trees*, however, Z and all marginals are computable exactly in linear time by the transfer-matrix/belief-propagation recursion; this tractable island is precisely where our research model lives.

(ii) Collective order emerges. For β beyond a critical value the two-dimensional model magnetises spontaneously: a macroscopic fraction of spins align even at $h = 0$, and the symmetric state splits into two ergodic components. This spontaneous symmetry breaking is the archetype of a *phase transition*, and its dynamical counterpart — a generation trajectory committing to one mode of a multimodal distribution — is exactly the *speciation* phenomenon of diffusion models (Biroli et al., 2024; Raya and Ambrogioni, 2023).

(iii) Disorder creates complexity. Making the couplings J_{ij} random (of both signs) yields spin glasses (Edwards and Anderson, 1975; Sherrington and Kirkpatrick, 1975): frus-

trated systems whose free-energy landscape shatters into exponentially many metastable valleys. The mean-field theory of these systems (Mézard et al., 1987) produced the cavity method and the TAP equations (Thouless et al., 1977) — the direct ancestors of the message-passing algorithms we deploy in Chapter 9 — and its modern export to inference problems (Zdeborová and Krzakala, 2016; Mézard and Montanari, 2009) is the methodological backbone of this thesis.

2.6 Complex systems, and what this thesis takes from physics

A *complex system* is, operationally, a system whose macroscopic behaviour is not readable off its microscopic rules: interactions generate collective phenomena — order, phases, transitions, slow dynamics — that must be derived, not postulated. Statistical mechanics is the toolbox for that derivation, and its objects map one-to-one onto the objects of modern generative modelling:

Statistical mechanics	Generative modelling
microstate x	data point / latent configuration
energy $H(x)$	negative log-density $-\log p(x)$ (up to Z)
Boltzmann distribution $e^{-\beta H}/Z$	model distribution
partition function Z	normalising constant / evidence
free energy F	negative ELBO (up to constants)
Langevin dynamics	sampling / reverse diffusion
phase transition	speciation, memorisation onset
cavity / TAP equations	belief propagation / AMP

The next four chapters make each row of this dictionary precise, starting with the probabilistic dynamics — Markov chains and their continuous-time limits — that turn the static ensembles of this chapter into processes that *move* probability around, which is what a diffusion model ultimately is.

Chapter 3

Stochastic Processes and Markov Chain Monte Carlo

Chapter 2 answered the question “*which distribution?*” — Boltzmann’s. This chapter answers “*how do we reach it, and how do we move within it?*”. The answer is the theory of Markov processes: chains whose future depends on the past only through the present. Markov chains play a triple role in this thesis: they are the *algorithms* that sample from intractable distributions (MCMC), they are the *data* whose diffusion score we compute exactly (the AR(1) sequence introduced in Section 3.2), and — run in continuous time — they are the *corruption and generation processes* of diffusion models (Chapters 4 and 6). Standard references for the material are Robert and Casella (2004) and Brockwell and Davis (1991).

3.1 Markov chains: kernels, stationarity, reversibility

Definition 3.1 (Markov chain). A sequence of random variables $(X_k)_{k \geq 0}$ with values in a state space $\mathcal{X}$ is a *Markov chain* if for every k and every measurable $A \subseteq \mathcal{X}$,

$$\mathbb{P}(X_{k+1} \in A \mid X_k, X_{k-1}, \dots, X_0) = \mathbb{P}(X_{k+1} \in A \mid X_k). \quad (3.1)$$

The conditional law is encoded by a *transition kernel* $M(x' \mid x)$: a density (or matrix, for discrete $\mathcal{X}$) with $\int M(x' \mid x) dx' = 1$ for every x .

The Markov property (3.1) is exactly a statement about *factorisation*: the joint law of the first K states is

$$p(x_0, x_1, \dots, x_{K-1}) = p_0(x_0) \prod_{k=0}^{K-2} M(x_{k+1} \mid x_k). \quad (3.2)$$

This one-line factorisation is the single most important equation of the thesis. It says that the joint distribution of a Markov trajectory is a product of *local* terms, each touching two consecutive variables — the structure that makes the chain a *tree* in the factor-graph sense of Chapter 5, that makes its precision matrix tridiagonal in the Gaussian case of Chapter 7, and that makes belief propagation exact in Chapter 9.

Marginal distributions evolve by integrating the kernel: $p_{k+1}(x') = \int M(x' \mid x) p_k(x) dx$, compactly $p_{k+1} = Mp_k$.

Definition 3.2 (Stationarity). A distribution π is *stationary* (or *invariant*) for M if $\pi = M\pi$, i.e.

$$\pi(x') = \int M(x' \mid x) \pi(x) dx \quad \text{for all } x'. \quad (3.3)$$

A chain started in π stays in π forever; the trajectory $(X_0, X_1, \dots)$ is then a *stationary process*, and all its finite-dimensional laws are invariant under time shifts.

Definition 3.3 (Detailed balance / reversibility). The kernel M satisfies *detailed balance*

with respect to π if

$$\pi(x) M(x' | x) = \pi(x') M(x | x') \quad \text{for all } x, x'. \quad (3.4)$$

Detailed balance says the equilibrium probability flux from x to x' equals the reverse flux — the chain, watched in equilibrium, is statistically indistinguishable run forwards or backwards. It is a *sufficient* condition for stationarity: integrating (3.4) over x gives $\int \pi(x) M(x' | x) dx = \pi(x') \int M(x | x') dx = \pi(x')$, which is (3.3). Its power is practical: detailed balance is a *local, pointwise* condition, checkable without ever computing an integral — which is why every MCMC algorithm in Section 3.3 is engineered to satisfy it.

Stationarity alone does not guarantee that the chain *finds* π ; for that one needs the chain to be able to reach everywhere (*irreducibility*) without getting trapped in cycles (*aperiodicity*). Under these conditions the fundamental convergence theorem holds (Robert and Casella, 2004): the marginal law of X_k converges to π from any initial condition, and time averages converge to expectations under π (the ergodic theorem),

$$\frac{1}{N} \sum_{k=1}^N f(X_k) \xrightarrow[N \rightarrow \infty]{\text{a.s.}} \mathbb{E}_\pi[f]. \quad (3.5)$$

Equation (3.5) is the rigorous form of the ergodic postulate invoked in Section 2.3, with the physical dynamics replaced by an algorithmic one.

3.2 The AR(1) chain: the running example of the thesis

We now introduce the specific Markov chain that the research chapters solve completely: the first-order autoregressive (AR(1)) process, the canonical Gaussian Markov chain (Brockwell and Davis, 1991). Let $\alpha \in (-1, 1)$ and let (η_k) be i.i.d. $\mathcal{N}(0, \sigma_\eta^2)$ innovations; define

$$a_{k+1} = \alpha a_k + \eta_k, \quad k = 0, 1, \dots, K-2. \quad (3.6)$$

The transition kernel is Gaussian, $M(a'|a) = \mathcal{N}(a'; \alpha a, \sigma_\eta^2)$, and the joint law of the trajectory follows from (3.2). Everything about this chain is computable in closed form; the next toolbox derives the three facts used throughout.

Toolbox 3.0 — Stationary distribution and autocovariance of AR(1)

(i) *Stationary variance.* Suppose $a_k \sim \mathcal{N}(0, \sigma^2)$ and impose that a_{k+1} has the same variance. Since η_k is independent of a_k ,

$$\text{Var}(a_{k+1}) = \alpha^2 \text{Var}(a_k) + \sigma_\eta^2 \stackrel{!}{=} \text{Var}(a_k) \implies \sigma^2 = \alpha^2 \sigma^2 + \sigma_\eta^2 \implies \sigma_\infty^2 = \frac{\sigma_\eta^2}{1 - \alpha^2}.$$

The fixed point exists precisely because $|\alpha| < 1$; the map $\sigma^2 \mapsto \alpha^2 \sigma^2 + \sigma_\eta^2$ is a contraction, so *any* initial variance converges to σ_∞^2 geometrically. Since Gaussianity is preserved by the affine recursion, the stationary law is $\pi = \mathcal{N}(0, \sigma_\infty^2)$.

(ii) *Autocovariance.* In the stationary regime, for $d \geq 1$, unroll the recursion d times: $a_{k+d} = \alpha^d a_k + \sum_{j=0}^{d-1} \alpha^j \eta_{k+d-1-j}$. All innovation terms are independent of a_k , hence

$$\text{Cov}(a_k, a_{k+d}) = \alpha^d \text{Var}(a_k) = \alpha^d \sigma_\infty^2,$$

and by symmetry $\text{Cov}(a_i, a_j) = \alpha^{|i-j|} \sigma_\infty^2$ for all i, j : correlations decay *geometrically* with the lag, with rate α . The covariance matrix of K consecutive frames,

$$(\Sigma_0)_{ij} = \sigma_\infty^2 \alpha^{|i-j|},$$

depends on (i, j) only through $i - j$: it is a symmetric *Toeplitz* matrix, a structure we exploit heavily in Chapter 7.

(iii) *Detailed balance.* With $\pi = \mathcal{N}(0, \sigma_\infty^2)$ and $M(a'|a) = \mathcal{N}(a'; \alpha a, \sigma_\eta^2)$, both sides of (3.4) are proportional to

$$\exp \left[-\frac{a^2}{2\sigma_\infty^2} - \frac{(a' - \alpha a)^2}{2\sigma_\eta^2} \right] = \exp \left[-\frac{(1 - \alpha^2)a^2 + (a')^2 - 2\alpha a a' + \alpha^2 a^2}{2\sigma_\eta^2} \right],$$

and the exponent $-[a^2 + (a')^2 - 2\alpha aa']/(2\sigma_\eta^2)$ is *symmetric* in (a, a') . Hence the stationary AR(1) chain is reversible: statistically, the movie of the chain looks the same played in either direction. $\square$

The AR(1) chain is thus the perfect miniature of everything this chapter discussed: a Markov kernel with an explicit stationary law, geometric convergence, reversibility, and — decisive for us — *exact Gaussian algebra* at every step. When in Chapter 7 we corrupt each frame of this chain with independent noise and ask for the joint score of the result, every quantity will remain in closed form.

3.3 Markov chain Monte Carlo

MCMC inverts the logic of the previous section. There, a kernel M was given and we asked for its stationary law π . In sampling problems we are given π — typically a Boltzmann distribution $\pi(x) \propto e^{-\beta H(x)}$ known only up to its partition function — and we must *design* a kernel M whose stationary law is π . The ergodic theorem (3.5) then converts a single long trajectory into expectation values. The idea originates, fittingly, in computational statistical mechanics (Metropolis et al., 1953).

3.3.1 The Metropolis–Hastings algorithm

Let $g(x'|x)$ be any proposal kernel we can sample from. The Metropolis–Hastings (MH) chain (Metropolis et al., 1953; Hastings, 1970) moves from x by (i) proposing $x' \sim g(\cdot|x)$, (ii) accepting the move with probability

$$A(x \rightarrow x') = \min \left\{ 1, \frac{\pi(x') g(x|x')}{\pi(x) g(x'|x)} \right\}, \quad (3.7)$$

and (iii) staying at x otherwise. Two features make (3.7) remarkable. First, π enters only through the *ratio* $\pi(x')/\pi(x)$: the intractable partition function cancels, so MH samples from $e^{-\beta H}/Z$ while only ever evaluating energy differences. Second, the rule is engineered

to satisfy detailed balance:

Toolbox 3.1 — Detailed balance for Metropolis–Hastings

The effective transition density for an accepted move $x \neq x'$ is $M(x'|x) = g(x'|x) A(x \rightarrow x')$. Compute the flux from x to x' :

$$\pi(x) M(x'|x) = \pi(x) g(x'|x) \min \left\{ 1, \frac{\pi(x') g(x|x')}{\pi(x) g(x'|x)} \right\} = \min \left\{ \pi(x) g(x'|x), \pi(x') g(x|x') \right\},$$

where the second equality uses $c \min\{1, b/c\} = \min\{c, b\}$ for $c > 0$. The final expression is *symmetric* under $x \leftrightarrow x'$; therefore

$$\pi(x) M(x'|x) = \pi(x') M(x|x'),$$

which is detailed balance (3.4). The rejection branch (staying at x) contributes only to the diagonal of the kernel and cannot violate the balance of fluxes between distinct states. Stationarity of π follows by the integration argument of Section 3.1; with an irreducible, aperiodic proposal, the convergence theorem applies. $\square$

3.3.2 Gibbs sampling

When $x = (x_1, \dots, x_n)$ is high-dimensional but each *conditional* $\pi(x_i | x_{\setminus i})$ is tractable, the Gibbs sampler (Geman and Geman, 1984) cycles through the coordinates, resampling each from its conditional given the current values of the others. Each coordinate update is an MH move with acceptance probability identically 1, so detailed balance holds coordinate-wise and π is stationary for the composite kernel. The Gibbs sampler thrives exactly when the model has sparse conditional structure — for a Markov chain, $\pi(x_k | x_{\setminus k})$ depends only on the neighbours (x_{k-1}, x_{k+1}) : locality of the model makes the algorithm local. The reader should hear, already, the theme of Chapter 9: *inference cost follows graph structure*.

3.3.3 Langevin dynamics: MCMC without rejections

For continuous x and differentiable energy, a different design principle uses gradients. Consider the discrete-time update

$$x_{k+1} = x_k + \epsilon \nabla \log \pi(x_k) + \sqrt{2\epsilon} \xi_k, \quad \xi_k \sim \mathcal{N}(0, I), \quad (3.8)$$

the (unadjusted) *Langevin algorithm*. The drift pushes towards high probability; the noise maintains exploration. Two observations preview the next chapters. First, (3.8) only requires the *score* $\nabla \log \pi$ — the normalising constant is never needed; this is precisely why the score is the natural object for generative modelling (Chapter 6), and why so much of this thesis is devoted to computing scores exactly. Second, as $\epsilon \rightarrow 0$ the recursion (3.8) becomes a stochastic differential equation — the Langevin diffusion — whose stationary law is exactly π ; making that limit rigorous requires the Itô calculus and Fokker–Planck theory of Chapter 4, where we will verify the claim by direct computation.

3.4 From algorithmic chains to data chains

A closing change of perspective, to set up the research problem. In this chapter Markov chains were *algorithms*: their stationary law was the target, their trajectory a means to an end. From Chapter 7 onwards the chain (3.6) plays the opposite role: it is the *data-generating process*, its trajectory $(a_0, \dots, a_{K-1})$ is one data point — a “video” of K frames — and the object of interest is the probability law of the *whole trajectory* after each frame has been independently corrupted by noise. The factorisation (3.2) then stops being a computational convenience and becomes the *structure to be discovered*: the entire research programme of this thesis can be phrased as asking how much of (3.2) survives noising, and how an inference algorithm — or a neural network — can exploit what survives.

Chapter 4

Stochastic Differential Equations, Itô Calculus, and the Fokker–Planck Picture

Diffusion models corrupt data by running a stochastic differential equation (SDE) and generate data by running another SDE backwards. This chapter builds that machinery from the ground up: Brownian motion, the Itô integral and Itô’s lemma, the Ornstein–Uhlenbeck (OU) process — the corruption channel used throughout the thesis — and the Fokker–Planck equation, which transports the *density* rather than the sample and closes the circle opened in Chapter 2: the stationary law of Langevin dynamics is exactly the Boltzmann distribution. References: Øksendal (2003); Gardiner (2009); Risken (1996).

4.1 From ODEs to SDEs

An ordinary differential equation $\dot{x} = f(x, t)$ moves a point along a deterministic velocity field. Physical and financial systems, however, feel rapid, unresolved forces — collisions,

order flow, thermal agitation — whose cumulative effect is well modelled by adding noise:

$$dX_t = \underbrace{f(X_t, t) dt}_{\text{drift}} + \underbrace{g(t) dW_t}_{\text{diffusion}}. \quad (4.1)$$

Here W_t is a *Wiener process* (standard Brownian motion): the unique process with $W_0 = 0$, independent increments, $W_t - W_s \sim \mathcal{N}(0, t - s)$, and continuous paths. Equation (4.1) is shorthand for the integral equation $X_t = X_0 + \int_0^t f(X_s, s) ds + \int_0^t g(s) dW_s$, and the whole subtlety lives in the last term: Brownian paths are nowhere differentiable, so $\int g dW$ cannot be a Riemann–Stieltjes integral. Itô’s construction (Itô, 1944) defines it as the limit of left-endpoint sums,

$$\int_0^t g(s) dW_s = \lim_{n \rightarrow \infty} \sum_{j=0}^{n-1} g(s_j) (W_{s_{j+1}} - W_{s_j}), \quad (4.2)$$

where the evaluation at the *left* endpoint s_j makes the integrand independent of the coming increment. Two consequences follow immediately: the Itô integral is a martingale (zero mean), and its variance is given by the *Itô isometry*, $\mathbb{E}[(\int_0^t g dW)^2] = \int_0^t g(s)^2 ds$, both because cross terms $\mathbb{E}[g(s_j)(W_{s_{j+1}} - W_{s_j})]$ vanish by independence.

The price of the construction is a new chain rule. Its origin is the *quadratic variation* of Brownian motion: increments scale as $dW_t \sim \sqrt{dt}$, so second-order terms in dW are first-order in dt and cannot be discarded.

Toolbox 4.0 — Itô’s lemma, and where the extra term comes from

Let $Y_t = \phi(X_t, t)$ with ϕ twice differentiable and X_t solving (4.1). Taylor-expand ϕ to second order:

$$dY = \frac{\partial \phi}{\partial t} dt + \frac{\partial \phi}{\partial x} dX + \frac{1}{2} \frac{\partial^2 \phi}{\partial x^2} (dX)^2 + \dots$$

Substitute $dX = f dt + g dW$ and expand the square:

$$(dX)^2 = f^2(dt)^2 + 2fg dt dW + g^2(dW)^2.$$

Now use the multiplication table of Itô calculus, which is nothing but the scaling

$dW \sim \sqrt{dt}$ made systematic:

$$(dt)^2 = 0, \quad dt dW = 0, \quad (dW)^2 = dt.$$

The last identity is the rigorous statement that the quadratic variation of W over $[0, t]$ equals t : for a partition of mesh $\rightarrow 0$, $\sum_j (W_{s_{j+1}} - W_{s_j})^2 \rightarrow t$ in mean square, since the sum has mean $\sum_j (s_{j+1} - s_j) = t$ and variance $2 \sum_j (s_{j+1} - s_j)^2 \rightarrow 0$. Keeping the surviving terms:

$$dY = \left(\frac{\partial \phi}{\partial t} + f \frac{\partial \phi}{\partial x} + \frac{g^2}{2} \frac{\partial^2 \phi}{\partial x^2} \right) dt + g \frac{\partial \phi}{\partial x} dW.$$

The half-Laplacian term — absent from the classical chain rule — is the engine of everything that follows: it is why densities *spread*, why the Fokker–Planck equation has a second-order term, and why exp-transformations of SDEs pick up variance corrections.

□

4.2 The Ornstein–Uhlenbeck process: the corruption channel

The noising process used by diffusion models — and by every result of this thesis — is the *Ornstein–Uhlenbeck process* (Uhlenbeck and Ornstein, 1930): Brownian motion pulled back to the origin by a linear spring,

$$dX_t = -X_t dt + \sqrt{2} dW_t, \quad X_0 = a. \quad (4.3)$$

(The drift and diffusion constants are a normalisation choice — this is the “variance-preserving” convention of the diffusion literature (Song et al., 2021); any other choice rescales time.) The OU process is the unique Gaussian, Markov, stationary process in continuous time, and it is exactly solvable.

Toolbox 4.1 — Exact solution of the OU SDE by integrating factor

Multiply (4.3) by e^t and consider $Y_t = e^t X_t$. By Itô's lemma with $\phi(x, t) = e^t x$ (here $\partial^2 \phi / \partial x^2 = 0$, so no correction term arises):

$$dY_t = e^t X_t dt + e^t dX_t = e^t X_t dt + e^t (-X_t dt + \sqrt{2} dW_t) = \sqrt{2} e^t dW_t.$$

The drift cancels exactly — that is what the integrating factor is for. Integrate from 0 to t and undo the substitution:

$$X_t = a e^{-t} + \sqrt{2} \int_0^t e^{-(t-s)} dW_s.$$

The integral is Gaussian (limit of Gaussian sums), with mean zero and variance given by the Itô isometry:

$$\text{Var}(X_t) = 2 \int_0^t e^{-2(t-s)} ds = 2 e^{-2t} \int_0^t e^{2s} ds = 2 e^{-2t} \frac{e^{2t} - 1}{2} = 1 - e^{-2t}.$$

Hence the *transition kernel* of the OU channel is

$$p(x_t | X_0 = a) = \mathcal{N}(x_t; a e^{-t}, \Delta_t), \quad \boxed{\Delta_t = 1 - e^{-2t}}.$$

□

The boxed kernel is used on every page of Chapters 7–9, so we fix its reading now. Corrupting a data point a for a time t produces

$$x = a e^{-t} + \sqrt{\Delta_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1) : \quad (4.4)$$

the signal is *contracted* by e^{-t} while isotropic noise of variance Δ_t fills in. At $t = 0$: $x = a$ (no corruption); as $t \rightarrow \infty$: $x \sim \mathcal{N}(0, 1)$ regardless of a (all information destroyed, the equilibrium of the spring). The signal-to-noise ratio e^{-2t}/Δ_t sweeps continuously from ∞ to 0: the OU channel interpolates between the data distribution and a standard Gaussian,

which is precisely what a diffusion model needs.

Two structural remarks, both load-bearing later. First, the OU kernel is Gaussian *in a as well as in x* : viewed as a function of the clean variable, $\mathcal{N}(x; ae^{-t}, \Delta_t) \propto \exp[-(e^{-2t}/2\Delta_t)(a - xe^t)^2]$ — a Gaussian “likelihood” factor. This is the fact that makes the posteriors of Chapter 7 jointly Gaussian and the messages of Chapter 9 closable. Second, if a random vector $a \sim \mathcal{N}(\mu, \Sigma_0)$ is pushed through (4.4) coordinate-wise with *independent* noises, the output is Gaussian with

$$\mu_t = e^{-t}\mu, \quad \Sigma_t = e^{-2t}\Sigma_0 + \Delta_t I, \quad (4.5)$$

since means and covariances transform linearly and the added noise is white. Equation (4.5) — shrink the covariance, add a multiple of the identity — is the entire “forward process” of this thesis in one line.

4.3 The Fokker–Planck equation: transporting densities

The SDE (4.1) moves *samples*; the corresponding motion of the *density* $p_t(x)$ is governed by the Fokker–Planck (or forward Kolmogorov) equation. This change of viewpoint — from trajectories to densities — is the same one Liouville’s theorem performed for Hamiltonian flow, and the derivation is parallel, with the Itô correction supplying a new, diffusive term.

Toolbox 4.2 — Derivation of the Fokker–Planck equation

Let ϕ be an arbitrary smooth test function of compact support, and compute $\frac{d}{dt}\mathbb{E}[\phi(X_t)]$ in two ways.

Way 1: through the samples. By Itô’s lemma,

$$d\phi(X_t) = \left(f\phi' + \frac{g^2}{2}\phi''\right)dt + g\phi' dW_t,$$

and taking expectations kills the martingale term:

$$\frac{d}{dt} \mathbb{E}[\phi(X_t)] = \mathbb{E}\left[f(X_t, t) \phi'(X_t)\right] + \frac{g(t)^2}{2} \mathbb{E}[\phi''(X_t)].$$

Way 2: through the density. $\mathbb{E}[\phi(X_t)] = \int \phi(x) p_t(x) dx$, so

$$\frac{d}{dt} \mathbb{E}[\phi(X_t)] = \int \phi(x) \frac{\partial p_t(x)}{\partial t} dx.$$

Equate the two, and integrate Way 1 by parts to move all derivatives off ϕ (boundary terms vanish by compact support): once for the drift term, twice for the diffusion term,

$$\int \phi \frac{\partial p_t}{\partial t} = \int \phi \left[-\frac{\partial}{\partial x} (f p_t) + \frac{g^2}{2} \frac{\partial^2 p_t}{\partial x^2} \right] dx.$$

Since ϕ is arbitrary, the integrands must agree pointwise:

$$\boxed{\frac{\partial p_t}{\partial t} = -\frac{\partial}{\partial x} (f(x, t) p_t(x)) + \frac{g(t)^2}{2} \frac{\partial^2 p_t(x)}{\partial x^2}.$$

The first term transports probability along the drift (as in Liouville); the second spreads it diffusively — it is the macroscopic footprint of the Itô correction. $\square$

4.3.1 Stationary law of Langevin dynamics: closing the loop

Apply the Fokker–Planck equation to the *Langevin SDE* associated with a potential U (the continuous-time limit of the algorithm (3.8) of Chapter 3):

$$dX_t = -\nabla U(X_t) dt + \sqrt{2\beta^{-1}} dW_t. \quad (4.6)$$

A stationary density solves $0 = \nabla \cdot (\nabla U p + \beta^{-1} \nabla p)$, i.e. the probability current $J = -(\nabla U) p - \beta^{-1} \nabla p$ is divergence-free; imposing the natural (decaying) boundary conditions forces $J \equiv 0$, hence $\nabla p/p = -\beta \nabla U$, hence

$$p_\infty(x) \propto e^{-\beta U(x)} : \quad (4.7)$$

Langevin dynamics equilibrates to the Boltzmann distribution of Chapter 2. This is the two-line proof of the claim left pending in Section 3.3, and it explains the deep kinship of sampling and physics: a Langevin sampler *is* an overdamped physical system at temperature $1/\beta$. Note also that the drift of (4.6) is (up to β) the *score* of the target, $-\nabla U = \nabla \log p_\infty$: dynamics that sample a distribution are driven by its score.

For the OU process, $U(x) = x^2/2$ and $\beta = 1$: the stationary law is $\mathcal{N}(0, 1)$, confirming the $t \rightarrow \infty$ limit of Section 4.2.

4.4 Time reversal: the mother theorem of diffusion models

One last piece of SDE theory converts everything above into a generative method. Suppose data $X_0 \sim p_0$ are corrupted by a forward SDE (4.1) up to time T , producing marginals p_t . Can the corruption be undone — can we find an SDE which, run from $X_T \sim p_T$ backwards, reproduces the same marginals? The answer (Anderson, 1982) is yes, and the reverse drift needs exactly one unknown object:

$$dX_t = \left[f(X_t, t) - g(t)^2 \nabla_x \log p_t(X_t) \right] dt + g(t) d\bar{W}_t, \quad (4.8)$$

where time runs backwards from T to 0 and $\bar{W}$ is a reverse-time Wiener process. The correction to the drift is the *score* $\nabla_x \log p_t$ of the noised marginal — the object defined in (1.1) and computed exactly, for Markov-chain data, in Chapters 7–9. A heuristic that makes (4.8) plausible follows from the Fokker–Planck equation: rewriting its diffusion term as a transport term,

$$\frac{g^2}{2} \frac{\partial^2 p_t}{\partial x^2} = \frac{\partial}{\partial x} \left(\frac{g^2}{2} p_t \frac{\partial \log p_t}{\partial x} \right),$$

one sees that the density evolution of (4.1) is *identical* to that of the noise-free ODE with velocity $f - \frac{g^2}{2} \nabla \log p_t$ (the *probability-flow ODE* of Song et al., 2021); running

that transport backwards, and re-injecting noise consistently, yields (4.8). The rigorous statement and proof are in Anderson (1982).

The chapter’s summary is one sentence: *the OU channel (4.4) destroys data at a known rate, the Fokker–Planck equation describes the destruction at the level of densities, and Anderson’s theorem says the destruction is invertible by anyone who knows the score $\nabla \log p_t$ — which is why the rest of this thesis is about computing that score, exactly, when the data have Markov structure.*

Chapter 5

Energy-Based Models, Markov Random Fields, and Factor Graphs

Chapter 2 derived the Boltzmann distribution from physics; this chapter adopts it as a *modelling language* for data. The three formalisms introduced here — energy-based models, Markov random fields, and factor graphs — are three views of the same object, ordered by increasing structural explicitness. The chapter is deliberately compact: every definition it contains is used, verbatim, in the research chapters, where the noisy Markov chain will be an energy-based model whose factor graph is a tree.

5.1 Energy-based models

An *energy-based model* (EBM) over $x \in \mathcal{X}$ is a probability distribution specified by an energy function E_θ :

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z(\theta)}, \quad Z(\theta) = \int_{\mathcal{X}} e^{-E_\theta(x)} \mathrm{d}x, \quad (5.1)$$

the Boltzmann distribution (2.9) with β absorbed into the energy and the physics replaced by a parametric family (Ackley et al., 1985; LeCun et al., 2006). The appeal of (5.1) is that E_θ is unconstrained — any function defines a valid unnormalised density; the price

is the partition function. Every basic operation hits it:

- *Evaluation* of $p_\theta(x)$ requires $Z(\theta)$ — intractable in general (Section 2.5).
- *Sampling* requires MCMC (Chapter 3), whose mixing time can be exponential in the presence of the metastability discussed in Section 2.5.
- *Learning* by maximum likelihood requires the gradient

$$-\nabla_\theta \log p_\theta(x) = \nabla_\theta E_\theta(x) - \mathbb{E}_{p_\theta}[\nabla_\theta E_\theta], \quad (5.2)$$

whose second term — the model expectation — again demands sampling from p_θ .

There are two classical escape routes from the partition function, and this thesis walks both. The first is the *score*: since $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$, the score is free of Z — differentiating in x kills the constant. Score matching (Chapter 6) and diffusion models are built on this observation. The second is *structure*: when the energy decomposes into local terms, marginals and even Z itself can become tractable by dynamic programming — that route is the rest of this chapter.

5.2 Markov random fields

Definition 5.1 (Markov random field). Let $G = (V, \mathcal{E})$ be an undirected graph with one random variable x_i per node. The collection $x = (x_i)_{i \in V}$ is a *Markov random field* (MRF) if it satisfies the local Markov property: for every node i ,

$$p(x_i \mid x_{V \setminus \{i\}}) = p(x_i \mid x_{\partial i}), \quad (5.3)$$

where ∂i is the set of neighbours of i in G .

The fundamental structure theorem connects this conditional-independence definition to the energy picture: by the Hammersley–Clifford theorem (proved in Besag, 1974), a

strictly positive density is an MRF on G if and only if it factorises over the *cliques* of G ,

$$p(x) = \frac{1}{Z} \prod_{C \in \text{cliques}(G)} \psi_C(x_C) = \frac{1}{Z} \exp \left[- \sum_C E_C(x_C) \right]. \quad (5.4)$$

Conditional independence, graphical locality, and additive local energies are one and the same thing. The Ising model (2.12) is the textbook example: cliques are edges, and $E_{(i,j)} = -J_{ij}\sigma_i\sigma_j$.

For *our* purposes the decisive example is one-dimensional. The Markov chain factorisation (3.2), $p(x) = p_0(x_0) \prod_k M(x_{k+1}|x_k)$, is of the form (5.4) with G a path graph: a Markov chain is an MRF on a line, with energy

$$E(x) = -\log p_0(x_0) - \sum_{k=0}^{K-2} \log M(x_{k+1}|x_k), \quad (5.5)$$

a sum of *pairwise, nearest-neighbour* terms. For the Gaussian AR(1) chain each term is quadratic, so E is a quadratic form whose coupling matrix connects only neighbours — read: a *tridiagonal precision matrix*. Chapter 7 derives this precision explicitly; the reader should note that its tridiagonality is not an accident of Gaussian algebra but the Hammersley–Clifford fingerprint of one-dimensional Markov structure.

5.3 Hidden Markov models: chains observed through noise

The research problem of this thesis is, structurally, a *hidden Markov model* (HMM) (Rabiner, 1989): a latent Markov chain $(a_0, \dots, a_{K-1})$ evolving by (3.6), observed only through independent noisy measurements — in our case, one OU channel (4.4) per frame,

$$x_k = a_k e^{-t} + \sqrt{\Delta_t} \varepsilon_k, \quad \varepsilon_k \text{ i.i.d. } \mathcal{N}(0, 1). \quad (5.6)$$

The joint law of latents and observations inherits both factorisations:

$$p(a, x) = \underbrace{p_0(a_0) \prod_{k=0}^{K-2} M(a_{k+1}|a_k)}_{\text{chain prior}} \cdot \underbrace{\prod_{k=0}^{K-1} \mathcal{N}(x_k; a_k e^{-t}, \Delta_t)}_{\text{local likelihoods}}. \quad (5.7)$$

Conditioning on the observations gives the *posterior* $p(a|x) \propto p(a, x)$, which is again an MRF on the line: the likelihood factors touch one variable each and therefore do not enlarge any clique. This innocent remark is the structural core of the thesis, for two reasons. First, as shown in Chapter 6, the diffusion score of the noisy sequence is exactly a posterior expectation under (5.7) — so computing scores *is* HMM inference. Second, posterior inference on chain-shaped MRFs is solvable in $O(K)$ by dynamic programming — the forward–backward algorithm for discrete HMMs (Rabiner, 1989), the Kalman smoother for linear-Gaussian ones (Kalman, 1960; Rauch et al., 1965) — and both are special cases of the single algorithm this thesis studies: belief propagation.

5.4 Factor graphs

MRFs record *which* variables interact but not *how* the joint factorises (a triangle of pairwise terms and one triple term have the same graph). The representation that makes factorisation itself graphical — and on which message passing is most naturally defined — is the *factor graph* (Kschischang et al., 2001).

Definition 5.2 (Factor graph). Given a factorisation $p(x) = \frac{1}{Z} \prod_a \psi_a(x_{\partial a})$, its factor graph is the bipartite graph with a *variable node* for each x_i , a *factor node* for each ψ_a , and an edge (i, a) whenever $x_i \in x_{\partial a}$ (i.e. variable i appears in factor a).

A factor graph is a *tree* when it is connected and cycle-free; trees are the domain where the sum–product algorithm of Chapter 9 computes every marginal exactly (Pearl, 1988; Mézard and Montanari, 2009; Bishop, 2006).

The posterior (5.7) of our noisy chain has factor graph: variable nodes $a_0, \dots, a_{K-1}$; pairwise factor nodes $M_k = M(a_{k+1}|a_k)$ between consecutive variables; and a unary fac-

tor node $L_k = \mathcal{N}(x_k; a_k e^{-t}, \Delta_t)$ hanging off each variable (the observations x_k are fixed numbers, not nodes). Pictorially it is a *caterpillar*: a spine of K variables alternating with $K - 1$ transition factors, plus one likelihood leaf per vertebra. Removing any edge disconnects it; it contains no cycle; it is a tree. Hence — recording the conclusion the research chapters will exploit —

Proposition 5.3. *For any Markov-chain prior and any per-frame observation model, the posterior $p(a | x)$ factorises over a tree-shaped factor graph. Exact marginal inference on it costs $O(K)$ message-passing operations.*

Note what Proposition 5.3 does *not* say: it does not say the messages are finitely parametrisable. For discrete chains they are vectors; for the Gaussian chain they close within the Gaussian family (Chapter 9); for general continuous chains they are infinite-dimensional functions, and *that* — not the graph topology — is where approximation enters. Keeping “tree-exactness of the algorithm” separate from “representability of the messages” is the conceptual hinge of this thesis, and the distinction the Gaussian closure results of Chapter 9 are designed to quantify.

5.5 What learning adds, and why we go exact

A learned generative model of sequences — an EBM with a neural energy, or the diffusion models of the next chapter — must discover the structure that equations (5.5)–(5.7) hand us for free. The modelling bet of this thesis is that the exactly solvable case is informative about the learned one: if even the *exact* posterior develops long-range couplings as noise grows (Chapter 7 makes this precise), then a learned score faces the same fate; and if exact inference degrades gracefully under locality constraints (Chapter 9), then local architectures are a principled bet, not just a cheap one. The next chapter completes the toolkit by explaining how modern diffusion models train against these objects at scale.

Chapter 6

Diffusion Models: from Thermodynamics to Scores

This chapter assembles the material of Chapters 2–5 into the modern theory of generative diffusion. We present the discrete-time construction (DDPM), the score-matching principle that makes training tractable, the continuous-time SDE formulation that unifies both, and the identity — Tweedie’s — that converts the score into a Bayesian posterior expectation. The chapter closes with the statistical-physics analyses of diffusion that motivate this thesis, and states the gap our research chapters fill.

6.1 The generative problem and the diffusion idea

Given samples of an unknown distribution p_0 on $\mathbb{R}^K$, a generative model must produce new samples. Diffusion models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021) solve this with a two-process design inspired by non-equilibrium thermodynamics: a fixed *forward process* gradually corrupts data into pure noise, and a learned *reverse process* undoes the corruption step by step. The forward process used throughout this thesis is the OU channel of Section 4.2: at diffusion time t ,

$$x_t = a e^{-t} + \sqrt{\Delta_t} \varepsilon, \quad a \sim p_0, \quad \varepsilon \sim \mathcal{N}(0, I), \quad \Delta_t = 1 - e^{-2t}, \quad (6.1)$$

so that the noised marginal is the convolution

$$p_t(x) = \int p_0(a) \mathcal{N}(x; ae^{-t}, \Delta_t I) da. \quad (6.2)$$

As t grows, p_t interpolates from the data distribution to $\mathcal{N}(0, I)$. By Anderson's theorem (Section 4.4), the corruption is undone by the reverse SDE (4.8), whose only unknown ingredient is the score $\nabla_x \log p_t(x)$ of the noised marginals. Generative diffusion is therefore, at bottom, *score estimation*: whoever knows $\nabla \log p_t$ for all t can sample from p_0 .

6.2 Denoising diffusion probabilistic models

The discrete-time formulation (Sohl-Dickstein et al., 2015; Ho et al., 2020) discretises (6.1) into a Markov chain $q(x_s|x_{s-1}) = \mathcal{N}(x_s; \sqrt{1 - \beta_s} x_{s-1}, \beta_s I)$ with a variance schedule $\beta_1, \dots, \beta_S$, and learns a reverse chain $p_\theta(x_{s-1}|x_s)$. Training maximises a variational lower bound on the data log-likelihood — the physicist will recognise the Gibbs variational principle (2.11) in modern dress.

Toolbox 6.0 — The evidence lower bound of a diffusion model

For any latent-variable model with joint $p_\theta(x_{0:S})$ and any inference distribution $q(x_{1:S}|x_0)$, Jensen's inequality gives

$$\log p_\theta(x_0) = \log \int p_\theta(x_{0:S}) dx_{1:S} = \log \mathbb{E}_q \left[\frac{p_\theta(x_{0:S})}{q(x_{1:S}|x_0)} \right] \geq \mathbb{E}_q \left[\log \frac{p_\theta(x_{0:S})}{q(x_{1:S}|x_0)} \right] =: \text{ELBO}.$$

Insert the two Markov factorisations $p_\theta(x_{0:S}) = p(x_S) \prod_s p_\theta(x_{s-1}|x_s)$ and $q(x_{1:S}|x_0) = \prod_s q(x_s|x_{s-1})$, then regroup telescopically using Bayes' rule $q(x_s|x_{s-1}) = q(x_{s-1}|x_s, x_0) q(x_s|x_0) / q(x_{s-1}|x_0)$:

$$-\text{ELBO} = \underbrace{D_{\text{KL}}(q(x_S|x_0) \| p(x_S))}_{\text{prior term}} + \sum_{s=2}^S \underbrace{\mathbb{E}_q D_{\text{KL}}(q(x_{s-1}|x_s, x_0) \| p_\theta(x_{s-1}|x_s))}_{\text{one denoising step each}} - \mathbb{E}_q \log p_\theta(x_0|x_1).$$

The decisive point: because the forward chain is Gaussian and conditioned on *both* end-points, each target $q(x_{s-1}|x_s, x_0)$ is an explicit Gaussian. Matching it with a Gaussian p_θ reduces each KL term to a weighted regression of the posterior mean — concretely, to predicting the noise ε that was added (Ho et al., 2020). The thermodynamic reading of the ELBO gap as an entropy-production bound goes back to Sohl-Dickstein et al. (2015). $\square$

The practical upshot of Ho et al. (2020) is the loss

$$\mathcal{L}(\theta) = \mathbb{E}_{s, a \sim p_0, \varepsilon} \left\| \varepsilon - \varepsilon_\theta \left(\underbrace{a e^{-t_s} + \sqrt{\Delta_{t_s}} \varepsilon}_{x_{t_s}}, s \right) \right\|^2, \quad (6.3)$$

a denoising regression. Its connection with the score is not incidental, as the next section shows: predicting the noise *is* estimating $\nabla \log p_t$ up to scale.

6.3 Score matching

Suppose we model the score directly by a function $s_\theta(x)$ and wish it to match $\nabla \log p(x)$ for data from p . The naive objective $\mathbb{E}_p \|s_\theta - \nabla \log p\|^2$ seems to require knowing the very score we lack. Two classical results remove the obstruction.

Toolbox 6.1 — Implicit score matching (Hyvärinen)

Expand the square and keep only θ -dependent terms:

$$\mathbb{E}_p \|s_\theta - \nabla \log p\|^2 = \mathbb{E}_p \|s_\theta\|^2 - 2 \mathbb{E}_p \langle s_\theta, \nabla \log p \rangle + \text{const.}$$

The cross term integrates by parts. Componentwise, using $p \partial_i \log p = \partial_i p$ and assuming $p s_{\theta,i} \rightarrow 0$ at infinity:

$$\int p s_{\theta,i} \partial_i \log p \, dx = \int s_{\theta,i} \partial_i p \, dx = - \int p \partial_i s_{\theta,i} \, dx.$$

Therefore

$$\mathbb{E}_p \|s_\theta - \nabla \log p\|^2 = \mathbb{E}_p \left[\|s_\theta(x)\|^2 + 2 \nabla \cdot s_\theta(x) \right] + \text{const} :$$

an objective involving only the model and data samples (Hyvärinen, 2005). Elegant, but the divergence term is expensive for large networks — which is why diffusion models use the denoising variant instead. $\square$

Toolbox 6.2 — Denoising score matching (Vincent)

Fix t and consider the noised marginal (6.2), written as $p_t(x) = \int p_0(a) q_t(x|a) da$ with Gaussian kernel $q_t(x|a) = \mathcal{N}(x; ae^{-t}, \Delta_t I)$. Then

$$\nabla_x \log p_t(x) = \frac{\int p_0(a) \nabla_x q_t(x|a) da}{p_t(x)} = \int \underbrace{\frac{p_0(a) q_t(x|a)}{p_t(x)}}_{=p(a|x)} \nabla_x \log q_t(x|a) da = \mathbb{E}[\nabla_x \log q_t(x|a) | x].$$

The score of the *marginal* is the posterior average of the score of the *kernel* — and the latter is trivial: $\nabla_x \log q_t(x|a) = -(x - ae^{-t})/\Delta_t$. A standard L^2 -projection argument turns this into a regression: for any s_θ ,

$$\mathbb{E}_{x \sim p_t} \|s_\theta(x) - \nabla \log p_t(x)\|^2 = \mathbb{E}_{a,x} \|s_\theta(x) - \nabla_x \log q_t(x|a)\|^2 + \text{const},$$

because conditional expectation is exactly the L^2 projection onto functions of x (Vincent, 2011). Substituting the Gaussian kernel score and $x = ae^{-t} + \sqrt{\Delta_t} \varepsilon$ gives $\nabla_x \log q_t = -\varepsilon/\sqrt{\Delta_t}$: *learning the score of noisy data is learning to predict the noise*, and the DDPM loss (6.3) is denoising score matching in disguise. $\square$

The continuous-time synthesis (Song and Ermon, 2019; Song et al., 2021) trains one network $s_\theta(x, t)$ against denoising score matching at all noise levels and samples with the reverse SDE (4.8) or its deterministic twin, the probability-flow ODE; an accessible exposition is Song (2021), and Karras et al. (2022) systematise the design space.

6.4 Tweedie’s identity: the score is a denoiser

The middle line of Vincent’s derivation deserves its own section, because it is the bridge on which the whole thesis walks. Rearranging $\nabla \log p_t(x) = \mathbb{E}[-(x - ae^{-t})/\Delta_t \mid x]$:

$$\boxed{\nabla_x \log p_t(x) = \frac{e^{-t} \mathbb{E}[a \mid x] - x}{\Delta_t} \iff \mathbb{E}[a \mid x] = e^t \left(x + \Delta_t \nabla_x \log p_t(x) \right).} \quad (6.4)$$

This is the Tweedie–Miyasawa formula of empirical Bayes (Robbins, 1956; Miyasawa, 1961; Efron, 2011): the score of the noisy marginal and the Bayes-optimal denoiser $\mathbb{E}[a \mid x]$ determine each other exactly, with no assumption whatsoever on p_0 . Three readings will recur:

1. *Geometric*: the score points from x towards (the contracted image of) the posterior mean of the clean data — denoising direction and score direction coincide.
2. *Statistical*: estimating the score is solving a Bayesian inference problem — compute a posterior expectation under the joint (5.7). Everything Chapter 5 said about inference on graphs now applies to scores.
3. *Algorithmic*: for sequence data with Markov prior, the posterior lives on a tree (Proposition 5.3), so the score is computable by message passing — the programme of Chapter 9.

For *vector-valued* data $a \in \mathbb{R}^K$ noised coordinate-wise, (6.4) holds componentwise with the *full* posterior:

$$\left(\nabla_x \log p_t(x) \right)_k = \frac{e^{-t} \mathbb{E}[a_k \mid x_0, \dots, x_{K-1}] - x_k}{\Delta_t}. \quad (6.5)$$

The conditioning is on *all* coordinates: even though the noise is independent across frames, the posterior mean of frame k — and hence the k -th score component — depends on every observation, because the prior couples the frames. Equation (6.5) is the precise reason the *joint* score differs from the per-frame marginal score, a distinction that is easy to blur

and whose importance for structured data was impressed on this project in supervision; Chapter 7 quantifies exactly how much coupling the conditioning induces, as a function of t .

6.5 The statistical physics of generative diffusion

A recent line of work analyses diffusion models with the tools of Chapter 2, in the regime where the score is *exact* — either computed from a solvable p_0 or from the empirical measure of n training points. Three findings frame our work.

Regimes and transitions. For high-dimensional solvable data (Gaussian mixtures, spin models), the reverse dynamics crosses sharp dynamical regimes (Biroli and Mézard, 2023; Biroli et al., 2024): a *speciation* time at which the trajectory commits to one mode — a spontaneous symmetry breaking in the sense of Section 2.5 (see also Raya and Ambrogioni, 2023) — and, when the score is learned from finitely many samples, a *memorisation* (collapse) time below which the dynamics condenses onto training points. The phase boundaries are computed with free-energy methods, in direct analogy with the random energy model.

Sampling as physics. Ghio et al. (2024) map generative samplers — flows, diffusions, autoregressive networks — onto statistical-physics problems and delimit where each is efficient, importing the algorithmic-hardness picture of spin glasses (Zdeborová and Krzakala, 2016).

The exactly solvable programme. The common method of these works is to choose p_0 rich enough to be interesting and simple enough that $\nabla \log p_t$ is available in closed form, then study the *exact* generation dynamics. What none of them vary is the *internal structure of a single data point*: their coordinates are exchangeable or i.i.d. conditional on a mode; there is no ordering, no dynamics *inside* the sample.

6.6 The gap, restated

Real sequential data — video, audio, trajectories — have exactly the structure the solvable programme leaves out: an internal (frame) time with a Markovian dependence along it. For such data the score (6.5) is a smoothing posterior on a chain, an object with seventy years of inference theory behind it (Chapters 3–5). Yet the questions a diffusion modeller asks about it — how does the coupling structure of $\nabla \log p_t$ deform with t ? can it be computed *locally*? what survives a Gaussian parametrisation of the messages when the data are not Gaussian? what does that imply for score architectures? — are not answered by the classical literature, which fixes the noise level, nor by the diffusion literature, which ignores internal structure. Answering them, in the fully solvable setting of the AR(1) chain and its non-Gaussian perturbations, is the contribution of Chapters 7–9.

Chapter 7

The Gaussian Chain Solved Exactly

This chapter answers RQ1. We take the simplest data distribution with genuine internal structure — the stationary Gaussian AR(1) chain of Section 3.2 — corrupt every frame with the OU channel of Section 4.2, and compute the joint score *exactly*, twice: once by linear algebra, once by Bayesian inference. The two routes must agree, and verifying that they do (to machine precision) is the first audit of the computational companion. We then characterise the complete lifecycle of the score’s coupling structure — the posterior precision matrix — from the sparse Markov regime at $t \rightarrow 0$ to the isotropic regime at $t \rightarrow \infty$, including a corrected small- t *band-fill law* and exact closed forms for the bulk of a long chain. Throughout, longer algebraic passages are isolated in toolboxes, and every boxed formula is covered by a named check of the numerical audit (Appendix C).

7.1 The model, and the object of study

Data. A single data point is a trajectory $a = (a_0, \dots, a_{K-1}) \in \mathbb{R}^K$ of the stationary AR(1) chain (3.6),

$$a_{k+1} = \alpha a_k + \eta_k, \quad \eta_k \sim \mathcal{N}(0, \sigma_\eta^2) \text{ i.i.d.}, \quad a_0 \sim \mathcal{N}(0, 1), \quad (7.1)$$

with $|\alpha| < 1$ and the *stationary normalisation* $\sigma_\eta^2 = 1 - \alpha^2$, which makes the marginal variance of every frame equal to 1 (Toolbox 3.1). One should think of k as an *internal* (frame) time — the index along the video — to be kept sharply distinct from the diffusion time t .

Corruption. Each frame passes independently through the OU channel (4.4) for a time t :

$$x_k = e^{-t}a_k + \sqrt{\Delta_t} \xi_k, \quad \xi_k \sim \mathcal{N}(0, 1) \text{ i.i.d.}, \quad \Delta_t = 1 - e^{-2t}. \quad (7.2)$$

Object of study. The *joint score* of the noisy sequence,

$$S(x, t) = \nabla_x \log P_t(x) \in \mathbb{R}^K, \quad P_t(x) = \int_{\mathbb{R}^K} P_0(a) \prod_{k=0}^{K-1} \mathcal{N}(x_k; e^{-t}a_k, \Delta_t) da. \quad (7.3)$$

We stress *joint*: the gradient is taken with respect to all K noisy frames simultaneously, under the law of the whole noisy trajectory. The per-frame *marginal* score $\partial_{x_k} \log p_t(x_k)$ is a different, strictly less informative object — for this model it is actually degenerate, as Section 7.3.3 shows — and conflating the two was an instructive early error of this project, corrected in supervision: a sequence is *one* sample from a joint law, not K samples from marginal laws. Only the joint score drives the reverse dynamics of the sequence as a whole, by Anderson’s theorem (Section 4.4) applied in $\mathbb{R}^K$.

7.2 The clean joint law: dense covariance, sparse precision

The chain factorisation (3.2) gives the clean joint density

$$P_0(a) = \mathcal{N}(a_0; 0, 1) \prod_{k=0}^{K-2} \mathcal{N}(a_{k+1}; \alpha a_k, \sigma_\eta^2) = \mathcal{N}(a; 0, \Sigma_0), \quad (7.4)$$

a centred Gaussian whose two natural parametrisations — covariance and precision — encode the chain’s structure in opposite ways.

Proposition 7.1 (Stationary covariance). $(\Sigma_0)_{ij} = \alpha^{|i-j|}$ for all i, j : a dense, symmetric Toeplitz matrix with geometrically decaying off-diagonals.

This was derived in Toolbox 3.1 (unrolling the recursion); stationarity is what makes the entries depend on $i - j$ only, i.e. what makes Σ_0 Toeplitz.

Proposition 7.2 (Tridiagonal precision). $Q_0 := \Sigma_0^{-1}$ is exactly tridiagonal:

$$(Q_0)_{kk} = \begin{cases} \frac{1}{\sigma_\eta^2}, & k \in \{0, K-1\}, \\ \frac{1+\alpha^2}{\sigma_\eta^2}, & 0 < k < K-1, \end{cases} \quad (Q_0)_{k,k\pm 1} = -\frac{\alpha}{\sigma_\eta^2}, \quad (7.5)$$

and every entry at distance ≥ 2 vanishes identically.

Toolbox 7.0 — Deriving the tridiagonal precision from the log-density

For a centred Gaussian, the precision is the Hessian of the negative log-density. From (7.4),

$$-2 \log P_0(a) = \frac{a_0^2}{\sigma_0^2} + \sum_{k=0}^{K-2} \frac{(a_{k+1} - \alpha a_k)^2}{\sigma_\eta^2} + \text{const}, \quad \sigma_0^2 = 1.$$

Read off the quadratic form term by term. *Off-diagonals*: the only term coupling a_k and a_{k+1} is the transition $(a_{k+1} - \alpha a_k)^2 / \sigma_\eta^2$, whose cross coefficient is $-2\alpha / \sigma_\eta^2$, i.e. a precision entry $-\alpha / \sigma_\eta^2$; *no term couples variables at distance ≥ 2* , so those precision entries are exactly zero. *Interior diagonal*: a_k appears in its left transition with coefficient $1 / \sigma_\eta^2$ and in its right transition with coefficient α^2 / σ_η^2 ; sum: $(1 + \alpha^2) / \sigma_\eta^2$. *Left boundary*: prior term $1 / \sigma_0^2 = (1 - \alpha^2) / \sigma_\eta^2$ plus right transition α^2 / σ_η^2 , summing to $1 / \sigma_\eta^2$; the right boundary is symmetric (left transition only). $\square$

Remark 7.3 (Tridiagonality is Markovianity, not Gaussianity). A zero in the precision matrix is a conditional independence given all other variables (the Hammersley–Clifford fingerprint of Section 5.2). The derivation above used Gaussianity only to identify the precision with the Hessian of $-\log P_0$; the *support* of that Hessian — nearest neighbours only — comes from the chain factorisation alone. Any Markov chain with smooth transitions has this sparsity pattern in $\nabla^2(-\log P_0)$; the Gaussian case just makes the entries

constants. This tridiagonal Q_0 is the only sparse object in the whole story, and every structural result below traces back to it.

The pair (Propositions 7.1, 7.2) is worth contemplating: the *covariance* is dense — every frame correlates with every other, at geometrically decaying strength — while the *precision* is tridiagonal. Correlation is global; conditional dependence is local. The score, we now show, is governed by the second object.

7.3 The noisy joint law and the score, two ways

7.3.1 Route one: linear algebra

Proposition 7.4 (Noised law). $P_t(x) = \mathcal{N}(x; 0, \Sigma_t)$ with

$$\boxed{\Sigma_t = e^{-2t} \Sigma_0 + \Delta_t I_K} \quad (7.6)$$

This is (4.5): the linear channel contracts the signal covariance and adds white noise; Gaussianity is preserved because an affine map plus independent Gaussian noise of a Gaussian is Gaussian. Note the diagonal: $e^{-2t} + \Delta_t = 1$ at every t (variance preservation); only the *off-diagonal* entries, $\alpha^{|i-j|} e^{-2t}$, feel the diffusion time — the noise of different frames is independent, so only the signal correlates.

Since $\log P_t(x) = -\frac{1}{2}x^\top \Sigma_t^{-1}x + \text{const}$,

$$\boxed{S(x, t) = -\Sigma_t^{-1}x = -Q_t x, \quad Q_t := \Sigma_t^{-1}.} \quad (7.7)$$

The joint score is *linear* in x — the hallmark of Gaussianity — and its entire structure is the noisy precision matrix Q_t : every question about the score (which frames talk to which, how strongly, at which t) is a question about a matrix. Sections 7.4 and 7.5 answer those questions completely.

7.3.2 Route two: Bayesian inference

Tweedie's identity (6.5) expresses the same score through the posterior mean of the clean chain:

$$S_k(x, t) = \frac{e^{-t} \mathbb{E}[a_k | x] - x_k}{\Delta_t}. \quad (7.8)$$

The posterior is itself Gaussian, with parameters worth recording once and using forever:

Proposition 7.5 (Posterior of the clean chain). $P(a | x) = \mathcal{N}(a; m, J^{-1})$ with

$$\boxed{J = \frac{e^{-2t}}{\Delta_t} I + Q_0, \quad J m = h := \frac{e^{-t}}{\Delta_t} x.} \quad (7.9)$$

Toolbox 7.1 — Posterior precision and field, by completing the square

Bayes' rule with the likelihood (7.2):

$$-2 \log P(a|x) = a^\top Q_0 a + \sum_k \frac{(x_k - e^{-t} a_k)^2}{\Delta_t} + \text{const} = a^\top \left(Q_0 + \frac{e^{-2t}}{\Delta_t} I \right) a - 2 \frac{e^{-t}}{\Delta_t} x^\top a + \text{const}.$$

Matching against the generic Gaussian form $a^\top J a - 2h^\top a$ gives (7.9) directly. Two structural observations. (i) The likelihood is *diagonal* in a : observing each frame through its own channel adds e^{-2t}/Δ_t — the signal-to-noise ratio of the channel, divergent as $1/2t$ at small t , vanishing at large t — to every diagonal entry of the prior precision, and touches nothing else. (ii) Consequently J is *tridiagonal at every t* : the posterior of the clean chain remains a nearest-neighbour MRF; what fills up with t is the precision of the *noisy* law, Q_t , not of the posterior. Keeping J (tridiagonal, posterior) and Q_t (dense, noisy marginal) distinct prevents much confusion. $\square$

Substituting the linear posterior mean $m = J^{-1}h$ into (7.8) must return $-Q_t x$ identically. It does — one can verify the operator identity $\frac{1}{\Delta_t} \left(e^{-2t} J^{-1} \frac{1}{\Delta_t} - I \right) = -Q_t$ directly from (7.6) and (7.9) by the Woodbury identity — and the computational companion checks the two routes agree to 2.7×10^{-15} across sweeps of (K, α, t) . The agreement is a genuine end-to-end audit: a $K \times K$ inversion against a Bayesian smoothing computation, built from different formulas.

7.3.3 $K = 2$: the smallest example, fully explicit

Everything above, written where nothing can hide. For $K = 2$,

$$\Sigma_t = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}, \quad \boxed{r = \alpha e^{-2t}} \quad \Rightarrow \quad Q_t = \frac{1}{1 - r^2} \begin{pmatrix} 1 & -r \\ -r & 1 \end{pmatrix}, \quad (7.10)$$

$$S_0(x, t) = -\frac{x_0 - r x_1}{1 - r^2}, \quad S_1(x, t) = -\frac{x_1 - r x_0}{1 - r^2}. \quad (7.11)$$

Contrast with the marginal score: each x_k is $\mathcal{N}(0, 1)$ at every t (variance preservation), so $\partial_{x_k} \log p_t(x_k) = -x_k$, independent of t and of the other frame — the marginal score of this model *carries no information about the dynamics at all*. The joint score (7.11) instead contains the cross term $-rx_1$: the denoiser of frame 0 borrows evidence from frame 1 with weight $r = \alpha e^{-2t}$ — the whole coupling lifecycle in miniature, equal to α at $t = 0$, decaying exponentially, gone at $t \rightarrow \infty$.

7.4 The lifecycle of the precision matrix

How does Q_t interpolate between its two known endpoints — the tridiagonal Q_0 at $t = 0$ and the identity at $t = \infty$? Three equivalent representations organise the answer.

Theorem 7.6 (Three forms of Q_t). *For all $t \geq 0$, with $\gamma_t := e^{-2t}/\Delta_t$ (the channel SNR) and $c_t := e^{2t} - 1$:*

$$Q_t = \frac{1}{\Delta_t} (I + \gamma_t \Sigma_0)^{-1} \quad (\text{SNR form}), \quad (7.12)$$

$$Q_t = U \operatorname{diag} \left(\frac{1}{e^{-2t} \omega_i + \Delta_t} \right) U^\top, \quad \Sigma_0 = U \operatorname{diag}(\omega_i) U^\top \quad (\text{spectral form}), \quad (7.13)$$

$$Q_t = e^{2t} Q_0 (I + c_t Q_0)^{-1} = e^{2t} \sum_{m \geq 0} (-c_t)^m Q_0^{m+1} \quad (\text{resolvent form}). \quad (7.14)$$

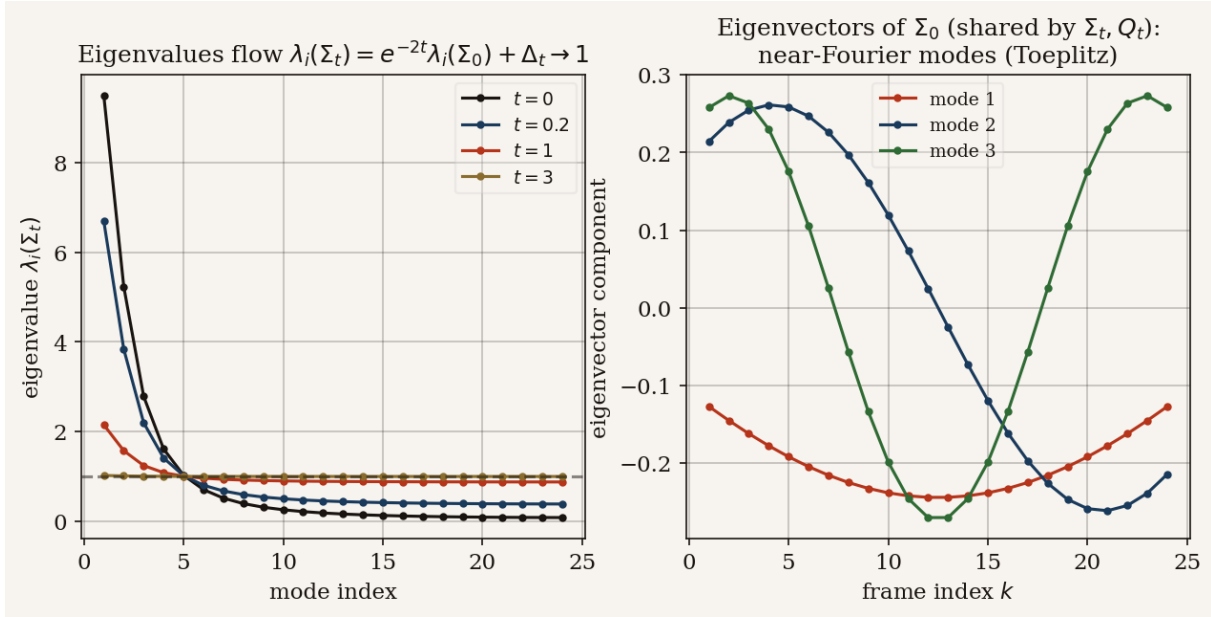


Figure 7.1: Spectral lifecycle. Left: the eigenvalues of Σ_t flow as $e^{-2t}\omega_i + \Delta_t$, all monotonically towards 1. Right: the eigenvectors of Σ_0 — shared by Σ_t and Q_t at every t — are near-Fourier modes of the chain. Figure generated by the companion code (Appendix C).

Toolbox 7.2 — Deriving the three forms

SNR form: factor Δ_t out of (7.6), $\Sigma_t = \Delta_t(I + \gamma_t \Sigma_0)$, and invert. *Spectral form:* Σ_0 is symmetric, so the spectral theorem gives an orthonormal eigenbasis $\Sigma_0 = U \text{diag}(\omega_i) U^\top$; since I commutes with everything, $\Sigma_t = U \text{diag}(e^{-2t}\omega_i + \Delta_t) U^\top$, and inverting a diagonal matrix is entrywise. *Resolvent form:* multiply (7.6) on the left by Q_0 and use $\Delta_t e^{2t} = c_t$: $Q_0 \Sigma_t = e^{-2t}(I + c_t Q_0)$; invert both sides. The Neumann series converges when $\|c_t Q_0\| < 1$, i.e. in the small- t /high-SNR regime, which is exactly where we deploy it below. $\square$

The spectral form contains a structural fact of independent interest: *the eigenvectors of Σ_t and Q_t are those of Σ_0 , for every t* — only the eigenvalues flow, $\omega_i \mapsto e^{-2t}\omega_i + \Delta_t \rightarrow 1$ (Figure 7.1). Because Σ_0 is symmetric Toeplitz, its eigenvectors are near-Fourier modes: in frequency space the denoising problem decouples into K scalar problems at all times, while in the *frame* basis — the operationally meaningful one for sequences — Q_t is sparse only at $t = 0$. Locality is a property of a matrix *in a basis*, and the two statements are complementary, not in tension.

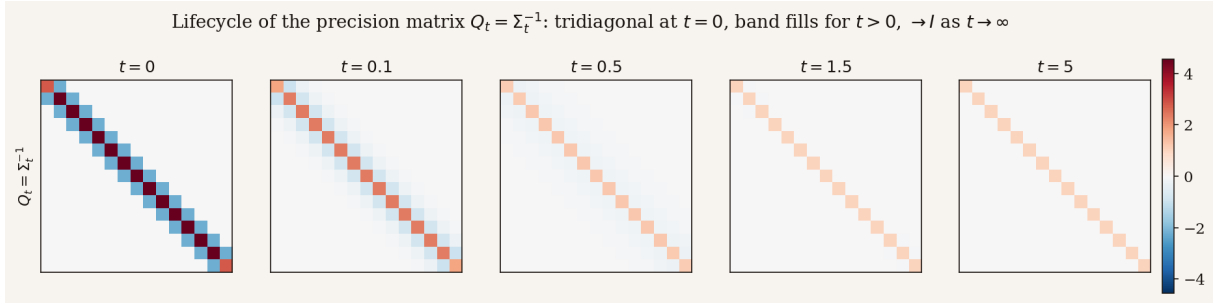


Figure 7.2: Lifecycle of Q_t in the frame basis ($K = 15$, $\alpha = 0.8$): exactly tridiagonal at $t = 0$ (Markov structure), the band fills at intermediate t (every frame couples to every other), and the matrix collapses to the identity as $t \rightarrow \infty$ (isotropic noise wins).

7.4.1 The band-fill law at small t , corrected

At $t = 0$ all entries of Q_t at distance $d \geq 2$ vanish. How fast do they switch on?

Theorem 7.7 (Band-fill leading order). *For small t and frame distance $d \geq 1$,*

$$(Q_t)_{i,i+d} = (-1)^{d-1} (2t)^{d-1} (Q_0^d)_{i,i+d} + O(t^d). \quad (7.15)$$

Toolbox 7.3 — Band fill from the Neumann series

In the resolvent form (7.14), substitute $c_t = 2t + 2t^2 + O(t^3)$ and $e^{2t} = 1 + O(t)$:

$$Q_t = (1 + O(t)) \left[Q_0 - c_t Q_0^2 + c_t^2 Q_0^3 - \dots \right].$$

Q_0 is tridiagonal, so Q_0^m has bandwidth m : the entry at distance d receives its *first* contribution from the single term $m = d$ of the series, which carries the prefactor $(-c_t)^{d-1} = (-1)^{d-1} (2t)^{d-1} (1 + O(t))$. Every later term is $O(t^d)$. Each additional frame of coupling range costs one power of t : the band fills *outward, one diagonal per order*.

□

Remark 7.8 (Correction of an earlier formula). An earlier working note of this project stated (7.15) with an extraneous factor $1/(d-1)!$. The factor does not follow from the resolvent expansion — the leading distance- d term is a *single* Neumann term, with no combinatorial sum behind it — and the numerical audit rejects it: the no-factorial

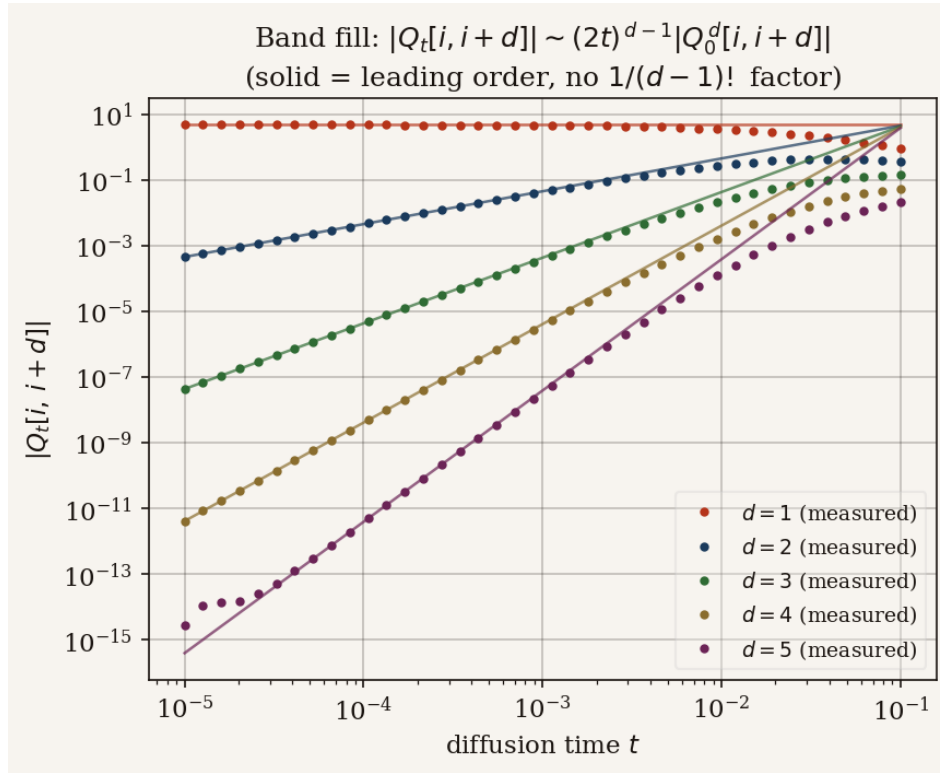


Figure 7.3: Band-fill law, log-log. Measured $|(Q_t)_{i,i+d}|$ against t (dots) and the leading order (7.15) (solid lines — no fitted parameters). The slopes are $d-1 = 0, \dots, 4$; agreement at small t is at the percent level for every d .

coefficient matches the measured $(Q_t)_{i,i+d}/t^{d-1}$ to better than 0.9% for every $d \in \{1, \dots, 5\}$ (at $\alpha = 0.9$, $K = 25$), while the factorial version fails by 86% at $d = 3$, 290% at $d = 4$ and over 800% at $d = 5$. Instructively, the error had survived because empirical *slope* fits (exponent $d-1$ in t) are blind to constant prefactors; only auditing the coefficient itself exposed it. Every prefactor in this thesis has since been audited, not only every exponent.

7.4.2 Return to isotropy at large t

Writing $\Sigma_t = I + e^{-2t}(\Sigma_0 - I)$ and expanding the inverse,

$$Q_t = I - e^{-2t}(\Sigma_0 - I) + e^{-4t}(\Sigma_0 - I)^2 + O(e^{-6t}), \quad (7.16)$$

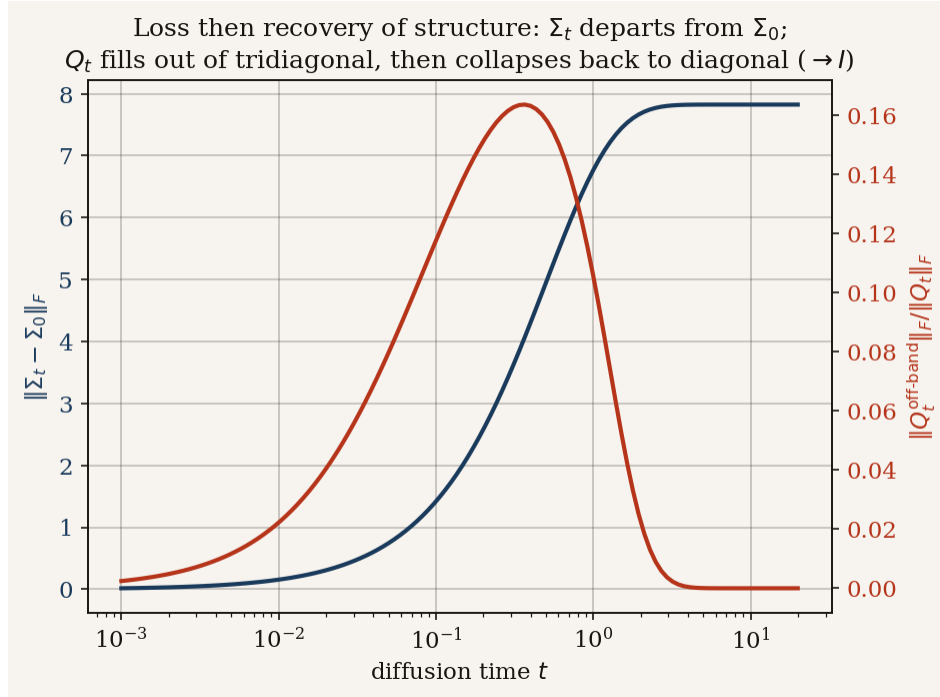


Figure 7.4: Structure lost and recovered, one curve each. $\|\Sigma_t - \Sigma_0\|_F$ (navy) grows monotonically as the covariance is driven to the identity; the off-tridiagonal Frobenius mass of Q_t (red) rises from zero, peaks at intermediate t , and decays back to zero: the band fills, then empties.

so $\|Q_t - I\|_F \simeq e^{-2t} \|\Sigma_0 - I\|_F$: all structure dies at the universal rate e^{-2t} , and the score collapses onto the isotropic $-x$. The full lifecycle is thus: *sparse* (tridiagonal, Markov) at $t = 0$; *dense* at intermediate t , with off-diagonals switching on at rate t^{d-1} per diagonal d ; *isotropic* at $t \rightarrow \infty$, with all structure decaying as e^{-2t} (Figure 7.4).

7.5 The bulk of a long chain, in closed form

For a long chain, the interior (“bulk”) is translation invariant and everything can be solved exactly. By Proposition 7.5 the posterior precision in the bulk is the tridiagonal Toeplitz matrix with

$$J_d = \frac{e^{-2t}}{\Delta_t} + \frac{1 + \alpha^2}{\sigma_\eta^2}, \quad \beta = -\frac{\alpha}{\sigma_\eta^2}, \quad \sigma_\eta^2 = 1 - \alpha^2. \quad (7.17)$$

The diagonal J_d has two parts with opposite behaviour in t : the *evidence* precision e^{-2t}/Δ_t (divergent as $1/2t$ at small t , vanishing at large t) and the constant *prior* precision. The coupling β is pure prior. The single dimensionless ratio $|\beta|/J_d \in (0, \frac{1}{2})$ controls everything below.

Theorem 7.9 (Exact bulk variance and geometric covariance). *In the bulk of a long chain, the posterior variance and covariance are*

$$V = \frac{1}{\sqrt{J_d^2 - 4\beta^2}}, \quad (J^{-1})_{i,i+d} = q^d V, \quad q = \frac{J_d - \sqrt{J_d^2 - 4\beta^2}}{2|\beta|} \in (0, 1). \quad (7.18)$$

Toolbox 7.4 — Transfer-matrix inversion of a tridiagonal Toeplitz matrix

Seek the inverse row in geometric form, $(J^{-1})_{i,i+d} = Cq^d$ for $d \geq 0$ (symmetric in $\pm d$). The defining relation $\sum_j J_{ij}(J^{-1})_{j,i+d} = \delta_{0d}$ reads, for $d \geq 1$,

$$\beta Cq^{d-1} + J_d Cq^d + \beta Cq^{d+1} = 0 \iff \beta q^2 + J_d q + \beta = 0,$$

a quadratic whose root inside the unit disc is $q = (J_d - \sqrt{J_d^2 - 4\beta^2})/(2|\beta|)$ (for $\alpha > 0$, i.e. $\beta < 0$, all inverse entries are positive; for $\alpha < 0$ they alternate in sign with the same modulus). The $d = 0$ relation fixes the prefactor: $J_d C + 2\beta Cq = 1$, and since $2\beta q = -(J_d - \sqrt{J_d^2 - 4\beta^2})$, $C = 1/\sqrt{J_d^2 - 4\beta^2} = V$. Existence of the real square root requires $J_d > 2|\beta|$, which *always* holds here:

$$J_d - 2|\beta| = \frac{e^{-2t}}{\Delta_t} + \frac{1 + \alpha^2 - 2|\alpha|}{\sigma_\eta^2} = \frac{e^{-2t}}{\Delta_t} + \frac{(1 - |\alpha|)^2}{\sigma_\eta^2} > 0$$

for every $|\alpha| < 1$ and $t > 0$ (the AM–GM inequality, with equality only at the excluded $|\alpha| = 1$). This positivity margin returns as a theorem about belief propagation in Chapter 9. $\square$

The number $q(\alpha, t)$ is the *memory of the posterior*: evidence travels $\xi = 1/\log(1/q)$

frames along the chain before fading. Its limits are exactly right:

$$t \rightarrow 0 : q \approx \frac{|\beta|}{J_d} \rightarrow 0 \quad (\text{each frame pinned by its own data}); \quad t \rightarrow \infty : q \rightarrow |\alpha| \quad (\text{posterior} \rightarrow \text{prior}) \quad (7.19)$$

the second limit because $q \rightarrow ((1 + \alpha^2) - (1 - \alpha^2))/(2|\alpha|) = |\alpha|$ when the evidence term dies. As diffusion time runs, the score's internal correlation length climbs monotonically from 0 to the prior's own $1/\log(1/|\alpha|)$.

7.5.1 The locality law

The bulk formulas answer RQ4's quantitative half exactly. Define the *local estimator of radius* r : the posterior mean of a_k given only the observations within distance r , $m_k^{(r)} = \mathbb{E}[a_k | x_{k-r}, \dots, x_{k+r}]$, all other frames marginalised out — the score a *banded* architecture of receptive field r could at best compute. Two facts make the comparison clean: any contiguous window of a stationary AR(1) chain is itself a stationary AR(1) chain, so $m_k^{(r)}$ is exact *on its window* — the only approximation is the truncation of range; and both estimators are linear in x , so the error $m_k - m_k^{(r)} = e^\top x$ has a closed-form RMS over the data law, $\text{RMS}_r = \sqrt{e^\top \Sigma_t e}$ — a deterministic number, no sampling involved.

Theorem 7.10 (Locality error decays exactly as q^r). *In the bulk, $\log \text{RMS}_r$ decreases linearly in r with slope $\log q(\alpha, t)$: truncating the evidence range at radius r costs a factor q per discarded frame.*

The mechanism is Theorem 7.9: the influence of evidence at distance d on the posterior mean at k is proportional to $(J^{-1})_{k,k \pm d} \propto q^d$, and the leading discarded evidence sits at distance $r + 1$. The audit verifies the *rate*: fitted slopes of $\log \text{RMS}_r$ over $r = 2, \dots, 13$ at the centre of a $K = 121$ chain match $\log q$ within 0.2% at $t \in \{0.1, 0.5, 2.0\}$ ($\alpha = 0.8$); see Figure 7.5. Note the earlier lesson repeated: single-realisation maximum-error fits are noisy and had misled a preliminary check at large t ; the deterministic RMS is the correct instrument.

For a learned score architecture the reading is direct: a receptive field $r \gtrsim \xi \log(1/\varepsilon)$,

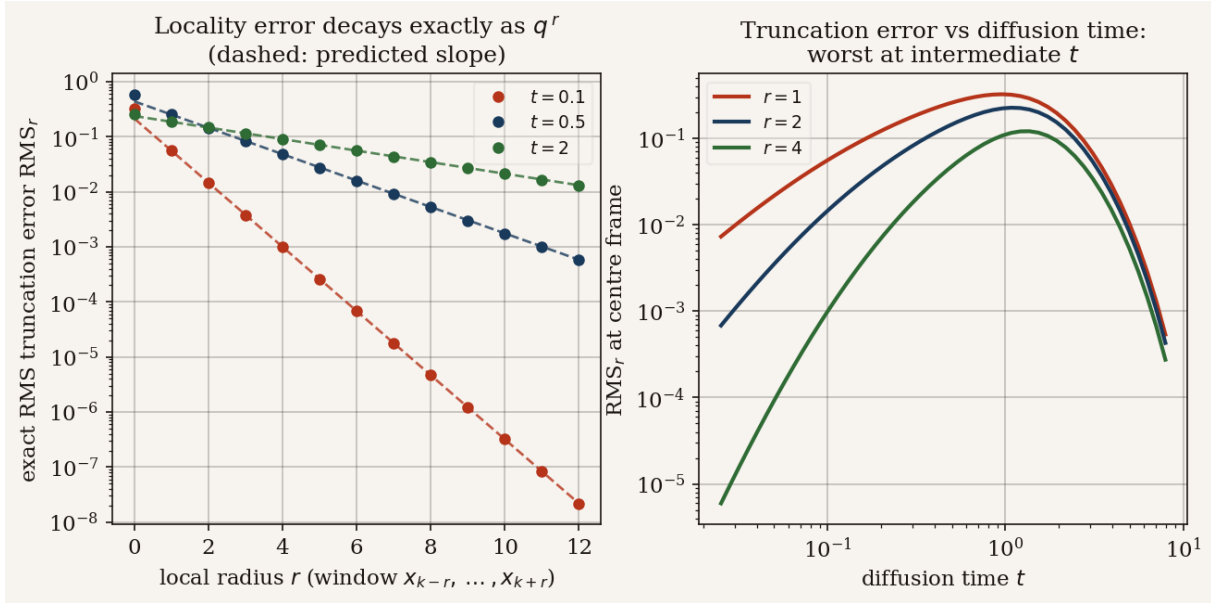


Figure 7.5: Locality of the joint score. Left: exact RMS truncation error of the radius- r local estimator (dots) at three diffusion times; the dashed lines have the *predicted* slopes $q(\alpha, t)^r$ from (7.18) — no fitting. Right: the same error at fixed radius as a function of t : it vanishes at both ends (at $t \rightarrow 0$ each frame is pinned by its own observation; at $t \rightarrow \infty$ the score itself decouples) and peaks at intermediate t .

with $\xi = 1/\log(1/q)$, guarantees relative error ε — and the requirement is most demanding at intermediate diffusion times, exactly where generative sampling spends its critical steps. In the Gaussian benchmark, the cost of locality is not a constant: it is a *rate*, and the rate is the single dial $q(\alpha, t)$.

7.6 Summary of the chapter

For the stationary AR(1) chain under the OU channel: the joint score is $-Q_t x$ with $Q_t = (e^{-2t}\Sigma_0 + \Delta_t I)^{-1}$, equal by Tweedie to the posterior-mean form (7.8) (both audited to machine precision); the clean precision is tridiagonal (Markov), the posterior precision stays tridiagonal at every t , the noisy precision fills at rate t^{d-1} per diagonal (no factorial) and returns to the identity at rate e^{-2t} ; the bulk posterior is solved exactly by (7.17)–(7.18), and the single parameter $q(\alpha, t) \in (0, |\alpha|]$ governs simultaneously the posterior correlation length and the exact decay rate of local-truncation error. What this

chapter did *not* yet provide is an algorithm: computing $Q_t x$ costs $O(K^3)$ (or $O(K^2)$ with structure), and it is global. Chapter 9 rebuilds the same score from local message passing at $O(K)$ — after Chapter 8 shows what changes when Gaussianity, and with it linearity, is abandoned.

Chapter 8

Beyond Gaussianity: Laplace Innovations

Every result of Chapter 7 rests on one algebraic miracle: Gaussians are closed under the two operations the model performs — multiplying densities and marginalising. This chapter removes the miracle in the most controlled way available: we keep the AR(1) chain, the OU channel, and the entire factor-graph structure, and change *only* the innovation law from Gaussian to Laplace. Heavy-tailed, kinked at the origin, yet still log-concave and analytically friendly, the Laplace distribution is the minimal step outside the Gaussian world — “Laplace and Gaussian, it’s not super different” was the supervision verdict, which is precisely why the comparison is informative: every deviation we find is attributable to non-Gaussianity alone, not to a change of structure. We proceed in the order of the research programme agreed in supervision: first the $K = 1$ building block, where the score first becomes nonlinear; then the chain $K \geq 2$, where we exhibit the Markov-structured form of the score, a clean closed form for the boundary message, and the precise point where closed forms die.

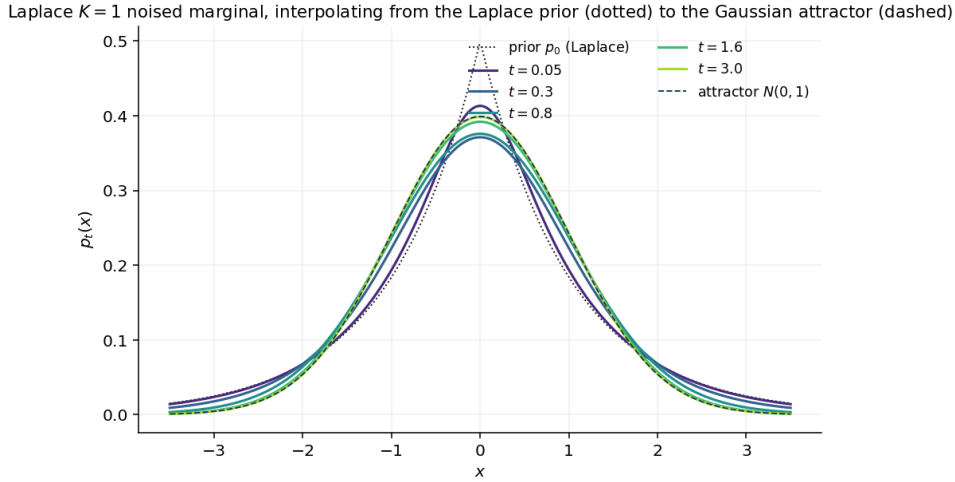


Figure 8.1: Laplace $K = 1$ noised marginal ($b = 1$) at increasing diffusion times, from the kinked Laplace prior (dotted) to the Gaussian attractor (dashed).

8.1 The $K = 1$ warm-up: one frame, no chain

For $K = 1$ there is no Markov structure; the model isolates the single ingredient “non-Gaussian prior meets Gaussian channel”. Let

$$a \sim \text{Laplace}(0, b), \quad p_0(a) = \frac{1}{2b} e^{-|a|/b}, \quad \text{Var}(a) = 2b^2, \quad (8.1)$$

and corrupt through the OU channel: $x = \mu a + \sqrt{\Delta_t} \xi$ with $\mu := e^{-t}$, $\xi \sim \mathcal{N}(0, 1)$. The noised marginal is the convolution

$$p_t^{(1)}(x) = \int p_0(a) \mathcal{N}(x; \mu a, \Delta_t) da. \quad (8.2)$$

Its limits are immediate: as $t \rightarrow 0$ the Gaussian kernel becomes a Dirac mass and $p_t^{(1)} \rightarrow p_0$ (the kinked Laplace shape); as $t \rightarrow \infty$ the kernel loses its a -dependence and $p_t^{(1)} \rightarrow \mathcal{N}(0, 1)$, the OU attractor. The family interpolates monotonically between the two (Figure 8.1).

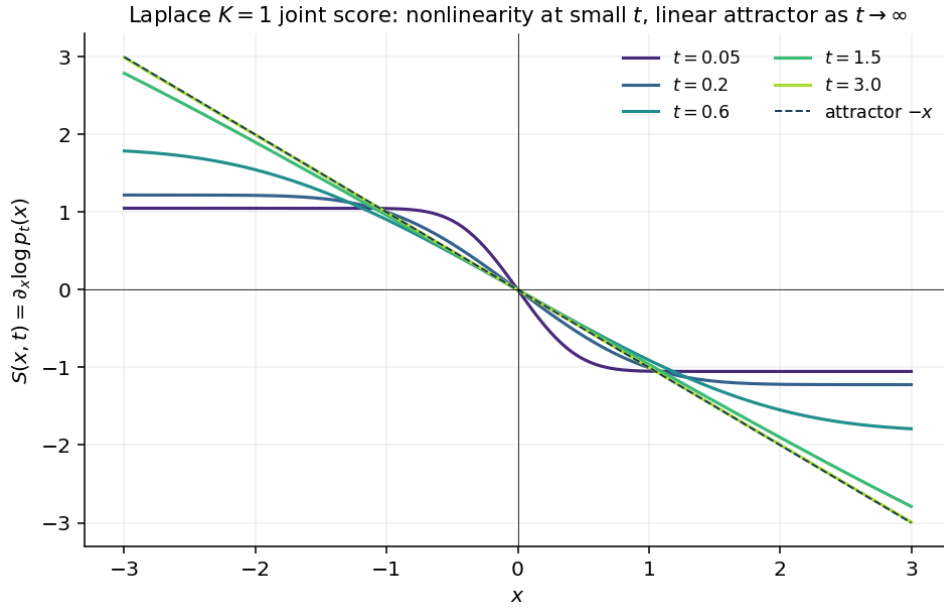


Figure 8.2: Laplace $K = 1$ score at the same diffusion times as Figure 8.1; the dashed line is the Gaussian attractor $-x$. The nonlinearity lives in the kink near $x = 0$ and flattens as t grows.

8.1.1 The score becomes nonlinear

Tweedie's identity (6.4) holds for *any* prior: $S(x, t) = (\mu \mathbb{E}[a|x] - x)/\Delta_t$. For a Gaussian prior the conditional mean is affine in x and the score is affine; for the Laplace prior $\mathbb{E}[a|x]$ interpolates nonlinearly between the regimes, and the score inherits the nonlinearity. Its two limits are again exact (Figure 8.3):

$$t \rightarrow 0 : \quad \Delta_t S(x, t) \rightarrow -\text{sign}(x) \cdot b^{-1} \cdot \Delta_t / \Delta_t \text{ i.e. } S \sim -\frac{\text{sign}(x)}{b} \cdot \frac{1}{1}, \quad t \rightarrow \infty : \quad S(x, t) \rightarrow -x, \quad (8.3)$$

that is: at small t the score converges (after the appropriate rescaling) to the *sign-shaped* score of the Laplace prior itself, $\partial_x \log p_0 = -\text{sign}(x)/b$, with all the nonlinearity concentrated in the kink near $x = 0$; at large t it flattens into the linear Gaussian-attractor score $-x$. The audit verifies the Tweedie route against central finite differences of $\log p_t^{(1)}$ computed by quadrature: maximum disagreement 1.19×10^{-7} over a grid of (b, t, x) , at the truncation floor of the finite difference; the Gaussian-attractor limit at $t = 6$ holds to 7.4×10^{-6} .

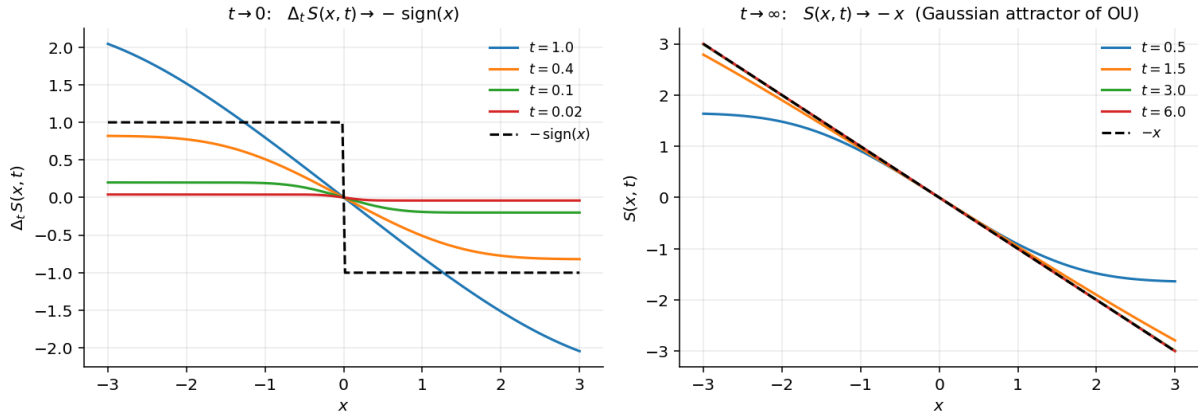


Figure 8.3: Limits of the Laplace $K = 1$ score ($b = 1$). Left: rescaled score for decreasing t , converging pointwise to $-\text{sign}(x)$. Right: unscaled score for increasing t , converging to $-x$.

8.1.2 The Fourier representation: convolution becomes product

The channel writes x as a sum of *independent* variables (μa and $\sqrt{\Delta_t} \xi$), so the density (8.2) is a convolution — awkward in real space, trivial in Fourier space.

Proposition 8.1 (Characteristic-function factorisation).

$$\hat{p}_t^{(1)}(k) = \underbrace{\frac{1}{1 + b^2 \mu^2 k^2}}_{\text{c.f. of } \mu a} \cdot \underbrace{e^{-\Delta_t k^2 / 2}}_{\text{c.f. of } \sqrt{\Delta_t} \xi}. \quad (8.4)$$

The proof is the convolution theorem plus two standard transforms: the Laplace characteristic function $(1 + b^2 k^2)^{-1}$ (a direct integral), rescaled by μ , and the Gaussian one. What the factorisation buys is a *differential* characterisation of the noised marginal:

Theorem 8.2 (Screened Poisson equation). *The Laplace noised marginal solves the linear, constant-coefficient ODE*

$$\boxed{(1 - b^2 \mu^2 \partial_x^2) p_t^{(1)}(x) = G_{\Delta_t}(x)} \quad G_{\Delta_t}(x) = \frac{e^{-x^2 / 2\Delta_t}}{\sqrt{2\pi\Delta_t}}. \quad (8.5)$$

Toolbox 8.0 — From the characteristic function to the screened Poisson equation

Multiply (8.4) by $(1 + b^2\mu^2k^2)$ to clear the rational factor:

$$(1 + b^2\mu^2k^2) \widehat{p}_t^{(1)}(k) = e^{-\Delta_t k^2/2},$$

whose right-hand side is exactly the characteristic function of G_{Δ_t} . Now translate back to real space with the differentiation rule: $\widehat{\partial_x f}(k) = ik \widehat{f}(k)$, applied twice, gives $\widehat{\partial_x^2 f}(k) = -k^2 \widehat{f}(k)$ — multiplication by k^2 in Fourier is $-\partial_x^2$ in real space, and this single minus sign converts $(1 + b^2\mu^2k^2)$ into the operator $(1 - b^2\mu^2\partial_x^2)$. Equation (8.5) follows by inverse-transforming term by term. The audit checks (8.4) against direct quadrature of $\int p_t^{(1)}(x)e^{ikx}dx$ to 2.9×10^{-6} , the floor of oscillatory quadrature. $\square$

Equation (8.5) is the structural heart of the Laplace case: the noised marginal is the solution of a screened Poisson equation with a Gaussian source, with screening length $b\mu = be^{-t}$ — the memory of the prior’s tails, shrinking with diffusion time. It also quantifies the supervision remark that Laplace “is not super different” from Gaussian: the deviation from Gaussianity is governed by a single local differential operator whose strength dies as e^{-t} .

8.1.3 The curvature becomes a field

Differentiating Tweedie’s identity once more,

$$H_t(x) := -\partial_x^2 \log p_t^{(1)}(x) = \frac{1}{\Delta_t} \left(1 - \frac{\mu^2}{\Delta_t} \text{Var}[a|x] \right). \quad (8.6)$$

For a Gaussian prior the posterior variance is x -independent, so H_t is a constant — the score is affine, its Hessian flat. For the Laplace prior $\text{Var}[a|x]$ depends on x , and $H_t(x)$ becomes a nontrivial *field*: the breakdown of Hessian constancy is the second-order shadow of the breakdown of score linearity, and both flow from the same source — a non-Gaussian prior makes $\mathbb{E}[a|x]$ nonlinear. This observation matters for Chapter 9: a Gaussian message is exactly the object that cannot represent an x -dependent curvature,

so (8.6) already tells us *what* the Gaussian closure will get wrong, before we measure *how much*.

8.2 The Laplace chain, $K \geq 2$

The Laplace AR(1) chain replaces the Gaussian innovations of (7.1) with Laplace ones:

$$P_0(a) = \frac{1}{(2b)^K} \exp\left(-\frac{1}{b} \sum_{k=0}^{K-1} |a_k - \alpha a_{k-1}|\right), \quad a_{-1} := 0. \quad (8.7)$$

The likelihood still factorises across frames, so the posterior inherits the *same* tree-shaped factor graph as the Gaussian model (Proposition 5.3): belief propagation remains exact in principle. What is lost is representability — the messages are no longer Gaussian, and computing them requires quadrature or an analytic representation. Three structural results, corresponding to the three threads of the supervision programme, delimit exactly what survives.

8.2.1 The Markov-structured form of the joint score

Theorem 8.3 (Joint score from chain messages). *For any chain prior — in particular (8.7) — and the OU likelihood, the joint score is, componentwise,*

$$S_k(x, t) = \frac{\mu \mathbb{E}[a_k|x] - x_k}{\Delta_t}, \quad \mathbb{E}[a_k|x] = \frac{\int a_k m_k(a_k) \mathcal{N}(x_k; \mu a_k, \Delta_t) b_k(a_k) da_k}{\int m_k(a_k) \mathcal{N}(x_k; \mu a_k, \Delta_t) b_k(a_k) da_k}, \quad (8.8)$$

where m_k and b_k are the forward and backward BP messages into frame k (Chapter 9).

The content of (8.8) — the direct answer to “express the score using the Markov structure” — is a complexity statement: the K -dimensional posterior integral has been replaced by two *one-dimensional* message recursions plus one one-dimensional integral per frame. Computational locality survives non-Gaussianity intact; what does not survive is the collapse of those 1D integrals into closed-form arithmetic, as we now make precise.

8.2.2 The boundary message in closed form, and where closed forms die

Proposition 8.4 (One-step Laplace integral). *For the Laplace transition $M(a'|a) = \frac{1}{2b}e^{-|a'-\alpha a|/b}$ and OU evidence $\mathcal{N}(x'; \mu a', \Delta_t)$,*

$$\int_{\mathbb{R}} M(a'|a) \mathcal{N}(x'; \mu a', \Delta_t) da' = p_t^{(1)}(x' - \mu\alpha a) : \quad (8.9)$$

the first backward message of the chain is the universal $K = 1$ noised marginal, evaluated at a shifted argument.

Toolbox 8.1 — Innovation change of variable, one step

Fix a and substitute the innovation $r := a' - \alpha a$; the transition density becomes the bare Laplace prior at r , and the Gaussian evidence at $a' = \alpha a + r$ becomes $\mathcal{N}(x' - \mu\alpha a; \mu r, \Delta_t)$. Hence

$$\int \frac{e^{-|r|/b}}{2b} \mathcal{N}(x' - \mu\alpha a; \mu r, \Delta_t) dr = p_t^{(1)}(x' - \mu\alpha a)$$

by the very definition (8.2). The shift $\mu\alpha a$ is what frame a predicts for the next frame's noisy observation under the AR(1) dynamics; the Act-one analysis of $p_t^{(1)}$ (screened Poisson equation included) recycles wholesale as the chain's boundary building block.

□

Remark 8.5 (Why iteration destroys the closed form). Equation (8.9) works because the integrand contains *exactly one* Laplace kink times *exactly one* Gaussian. Every subsequent BP step, in either direction, violates this: the incoming message already carries the propagated structure of several frames — a sum of products of kinks and Gaussians — so the next integrand contains two or more absolute-value kinks. On each sign-quadrant the integral reduces to half-line Gaussians, but globally it is a sum of error functions whose arguments depend on the conditioning variable non-affinely: the clean form does not survive one iteration. Contrast the Gaussian chain, where every step is Gaussian-

times-Gaussian and closure propagates forever. This is the precise, structural location of the difference between the two benchmarks — and the precise reason the Gaussian projection of Chapter 9 is an *approximation* here, with an error to be measured rather than an identity to be proved.

8.2.3 Innovation coordinates: the coupling has to live somewhere

The one-step trick suggests decoupling the whole prior at once. Define the triangular change of variables $r_0 = a_0$, $r_k = a_k - \alpha a_{k-1}$; its inverse is $a = Lr$ with $L_{ij} = \alpha^{i-j} \mathbf{1}_{i \geq j}$ unit lower-triangular, so the Jacobian is 1 and

$$P_0(r) = \frac{1}{(2b)^K} \prod_{k=0}^{K-1} e^{-|r_k|/b} : \quad (8.10)$$

in innovation coordinates the prior factorises into i.i.d. Laplace variables — as decoupled as it can be. But substituting $a = Lr$ in the likelihood produces

$$p(x|r) \propto \exp\left(-\frac{\mu^2}{2\Delta_t} r^\top L^\top L r + \frac{\mu}{\Delta_t} x^\top L r\right), \quad (8.11)$$

and $L^\top L$ is *dense* (its entries sum geometric series in α and vanish nowhere). The AR(1) coupling did not disappear; it migrated from the prior to the likelihood. In the original coordinates the prior carries the chain and the likelihood is diagonal; in innovation coordinates the prior is diagonal and the likelihood carries the chain. There is no basis in which both are diagonal at once — the non-Gaussian counterpart of the Gaussian statement that Q_t is tridiagonal in the frame basis and diagonal in the spectral basis, but never both (Section 7.4).

8.3 What survives, what is open

Chapter results, classified. *Exact*: the noised marginal limits, the score limits (8.3), the characteristic-function factorisation (8.4), the screened Poisson equation (8.5), the Markov-structured score (8.8), the one-step boundary form (8.9), the innovation factorisation (8.10)–(8.11), and the structural breakdown argument of Remark 8.5. *Numerical*: the quadrature audits quoted above (Tweedie vs finite differences; characteristic function vs direct integration). *Open*: a quadrature-free representation of general (non-boundary) messages at $K \geq 2$, e.g. by controlled small- α expansion; and a basis-independent statement of which locality of the joint score survives non-Gaussianity (candidates: sparsity of the x -dependent Hessian in a suitable basis, low-rank corrections to a tridiagonal core). An earlier conjecture of this project — that the Laplace $K = 2$ Hessian is approximately rank-two — proved regime-dependent under quick-resolution tests and is *not* asserted here.

The chapter leaves us exactly where the research programme wants us: the Laplace chain is structurally identical to the Gaussian one (same tree, same $O(K)$ message complexity, same Tweedie identity) but its messages are genuinely infinite-dimensional. The question “what does it cost to force them Gaussian?” is now sharply posed; Chapter 9 answers it.

Chapter 9

The Score as Posterior Expectation: Belief Propagation and the Gaussian Closure

Chapter 7 computed the joint score by inverting a $K \times K$ matrix; Tweedie’s identity (Section 6.4) says the same object is a posterior expectation. This chapter takes the Bayesian route seriously and rebuilds the score by *message passing*: local computations on the factor graph of the posterior. The questions answered are RQ2 and RQ3. First we show that belief propagation computes the score of the Gaussian chain *exactly*, in $O(K)$, and that the algorithm is — update by update — the classical Kalman filter and smoother. The closure of Gaussian messages is pure algebra, not a central-limit argument: this resolves, in the affirmative and with a precise mechanism, the first conjecture of the supervision discussion that shaped this chapter. Second, we analyse the per-node Gaussian closure (AMP, the TAP equations of statistical physics) and prove the second half of that discussion — “a CLT on two data points is usually not right” — as a theorem: on the chain, AMP still returns the exact score, but its variance closure develops an exact breakdown time $t_c(\alpha)$ with critical coupling $\alpha_c = \sqrt{2} - 1$. Third, we quantify the Gaussian closure on genuinely non-Gaussian chains, using deterministic grid quadrature as exact reference, and extract the locality lessons for learned score architectures.

9.1 The computational problem

Fix the setting of Chapters 7–8: a Markov chain prior $P_0(a) = p_0(a_0) \prod_k M(a_{k+1}|a_k)$, per-frame OU evidence, and the posterior

$$p(a | x) \propto \underbrace{\psi_{\text{pr}}(a_0) \prod_{k=0}^{K-2} \psi_{\text{tr}}^{(k)}(a_k, a_{k+1})}_{\text{chain prior}} \underbrace{\prod_{k=0}^{K-1} \psi_{\text{ob}}^{(k)}(a_k)}_{\text{evidence}}, \quad \psi_{\text{ob}}^{(k)}(a_k) = \mathcal{N}(x_k; e^{-t}a_k, \Delta_t). \quad (9.1)$$

By Tweedie, the k -th score component needs the posterior mean $\mathbb{E}[a_k|x]$ — a marginal of (9.1). Computing K marginals naively costs $O(K^2)$ one-dimensional eliminations ($O(K^3)$ by matrix inversion in the Gaussian case); message passing computes *all* of them in one forward and one backward sweep, $O(K)$ total. The gain is not merely computational: the message-passing form exposes *which* information reaches frame k from where — exactly the structural question RQ4 asks.

9.1.1 The factor graph is a tree

The factor graph of (9.1) has K variable nodes, $2K$ factor nodes (1 prior, $K-1$ transitions, K observations — the data x_k are constants clamped inside their factors, not nodes), and $3K-1$ edges: each transition factor has degree 2, prior and observations degree 1. A connected graph with $(\text{nodes} - 1)$ edges is a tree. Pictorially it is a *caterpillar*: a spine of variables and transition factors, one evidence leaf per vertebra — in the words of the supervision discussion, “very tree-like: a straight line with things on the side”. Every interior variable has degree three; every message update combines a *small, fixed* number of inputs. Two consequences, one per half of this chapter: BP on a tree is exact (Pearl, 1988; Mézard and Montanari, 2009); and this graph is the diametrical opposite of the dense, high-degree graphs for which the AMP closure was invented — which is where the BP/AMP contrast will come from.

9.1.2 Sum-product, and the continuous-message obstruction

On a generic factor graph, sum-product BP passes messages along every edge (Kschischang et al., 2001; Mézard and Montanari, 2009):

$$\nu_{i \rightarrow a}(a_i) \propto \prod_{b \in \partial i \setminus a} \hat{\nu}_{b \rightarrow i}(a_i), \quad \hat{\nu}_{a \rightarrow i}(a_i) \propto \int \psi_a(a_{\partial a}) \prod_{j \in \partial a \setminus i} \nu_{j \rightarrow a}(a_j) da_{\partial a \setminus i}, \quad (9.2)$$

with the belief at node i proportional to $\prod_{a \in \partial i} \hat{\nu}_{a \rightarrow i}(a_i)$. On a tree these equations have a unique solution, reached in one inward and one outward pass, and the beliefs are the exact marginals (Mézard and Montanari, 2009, Theorem 14.1).

For *discrete* variables each message is a finite probability vector and (9.2) is straightforward arithmetic. For *continuous* variables each message is a probability *density* — an infinite-dimensional object — and the update integrals must be carried out in function space. This is the obstruction, identified sharply in supervision, that makes “BP on continuous chains” a research question rather than a textbook exercise: tree-exactness of the *algorithm* is untouched, but the *representation* of messages becomes the whole problem. There are exactly three ways out: (i) discrete data (retreat to finite alphabets); (ii) a conjugate family closed under the updates — the Gaussian route, exact for our Gaussian chain, as we prove next; (iii) numerical function representation — the grid route, which we use in Section 9.5 as exact reference for non-Gaussian chains.

9.2 Belief propagation on the chain

On the chain, (9.2) collapses into one forward and one backward sweep. The bookkeeping deserves unusual care — an earlier version of this project’s notes fell into a trap here, caught only by the numerical audit.

Proposition 9.1 (Convention A). *Define messages*

$$\mu_{\rightarrow 0}(a_0) := \psi_{\text{pr}}(a_0), \quad \mu_{\rightarrow k+1}(a_{k+1}) := \int \psi_{\text{tr}}^{(k)}(a_k, a_{k+1}) \psi_{\text{ob}}^{(k)}(a_k) \mu_{\rightarrow k}(a_k) da_k, \quad (9.3)$$

$$\mu_{\leftarrow K-1}(a_{K-1}) := 1, \quad \mu_{\leftarrow k-1}(a_{k-1}) := \int \psi_{\text{tr}}^{(k-1)}(a_{k-1}, a_k) \psi_{\text{ob}}^{(k)}(a_k) \mu_{\leftarrow k}(a_k) da_k. \quad (9.4)$$

Then for every k ,

$$p(a_k | x) \propto \mu_{\rightarrow k}(a_k) \psi_{\text{ob}}^{(k)}(a_k) \mu_{\leftarrow k}(a_k) : \quad (9.5)$$

the forward message into a_k carries the evidence $x_0, \dots, x_{k-1}$ only, the backward message carries $x_{k+1}, \dots, x_{K-1}$ only, and the local evidence enters once, at combination time.

The proof is direct marginalisation: integrating (9.1) over all a_j , $j \neq k$, splits at a_k by the Markov property into independent left and right halves, and peeling the innermost variable of each half yields the recursions. What deserves the reader's full attention is the *bookkeeping*:

Remark 9.2 (The double-counting trap). A natural-looking alternative (call it Convention B) absorbs the local evidence into the forward message, $\mu_{\rightarrow k}^{\text{B}} = \mu_{\rightarrow k}^{\text{A}} \cdot \psi_{\text{ob}}^{(k)}$. Under Convention B the product (9.5) counts x_k *twice* and must be corrected by dividing one factor back out. Convention A is the bookkeeping under which the textbook combination rule is correct *as written*. The audit was decisive: under Convention A the BP score matches the matrix score to 10^{-14} ; under naive Convention B the disagreement is order one. This is the single most error-prone point of the construction — hence every message in this thesis is stated with its evidence content made explicit.

Each sweep computes $K - 1$ messages by one-dimensional integrals; the combination at each node multiplies three univariate functions. Total cost $O(K)$.

9.3 The Gaussian case: closure, Kalman, machine precision

For the Gaussian chain, two elementary facts make the message space finite-dimensional — and they are the *only* facts needed.

Toolbox 9.0 — The two closure lemmas of Gaussian message passing

Lemma 1 (products). For Gaussians in the same variable,

$$\mathcal{N}(a; m_1, v_1) \times \mathcal{N}(a; m_2, v_2) \propto \mathcal{N}(a; m, v), \quad \frac{1}{v} = \frac{1}{v_1} + \frac{1}{v_2}, \quad \frac{m}{v} = \frac{m_1}{v_1} + \frac{m_2}{v_2} :$$

add the exponents; the sum of two quadratics in a is a quadratic in a , with precisions and precision-weighted means adding. (In the natural parameters $\lambda = 1/v$, $\theta = m/v$ the rule is literally $(\lambda, \theta) \mapsto (\lambda_1 + \lambda_2, \theta_1 + \theta_2)$.)

Lemma 2 (affine marginalisation). Pushing a Gaussian through the AR(1) transition,

$$\int \mathcal{N}(a'; \alpha a, \sigma_\eta^2) \mathcal{N}(a; m, v) da = \mathcal{N}(a'; \alpha m, \alpha^2 v + \sigma_\eta^2),$$

because $a' = \alpha a + \eta$ with independent Gaussian summands: means and variances transform as stated, and Gaussianity is preserved. The same computation read in the reverse direction gives the backward analogue $\mathcal{N}(a; m'/\alpha, (v' + \sigma_\eta^2)/\alpha^2)$.

Together: the Gaussian family is closed under exactly the two operations the recursions (9.3)–(9.4) perform. Note also that the local evidence, read as a function of a_k , is itself Gaussian — $\psi_{\text{ob}}^{(k)}(a_k) \propto \mathcal{N}(a_k; e^t x_k, \Delta_t e^{2t})$, a pseudo-observation at the deconvolved location $e^t x_k$ with precision e^{-2t}/Δ_t , the pinning strength of Chapter 7. $\square$

Theorem 9.3 (Gaussian closure of BP on the chain). *For the Gaussian AR(1) chain under the OU channel, every BP message and every belief is exactly Gaussian, at every node, for every (K, α, t) ; each message is carried by two numbers, and BP computes the exact posterior marginals — hence, via Tweedie, the exact joint score — in $O(K)$.*

Derivation. Induction along each sweep. The initial messages are Gaussian (the prior; the flat message is the $\lambda \rightarrow 0$ limit of the family). Each step of (9.3)–(9.4) is one application of Lemma 1 (absorb the local evidence) followed by one of Lemma 2 (push through the transition); the combination (9.5) is Lemma 1 twice. Exactness follows from tree-exactness of sum–product. $\square$

Remark 9.4 (Closure is algebra, not a central limit theorem). The closure holds at node degree two just as it would at degree two thousand: it is a property of the Gaussian *family* under multiplication and affine marginalisation, not an aggregation-of-many- influences effect. This answers precisely the doubt raised in supervision — whether Gaussian messages on a chain could be trusted, given that the usual central-limit justification needs many incoming messages. For *exact* Gaussian BP, no CLT is invoked anywhere, so there is nothing to fail. The CLT enters only to justify the *per-node* closure of Section 9.4 — and there, on the chain, it fails exactly as predicted.

The classical identifications. Executing the sweeps in (m, v) -arithmetic, the forward recursion is a *predict–update* cycle: absorb the pseudo-observation (update), push through the dynamics (predict) — this is the Kalman filter (Kalman, 1960); the forward–backward combination (9.5) is the Rauch–Tung–Striebel smoother (Rauch et al., 1965); both are the Gaussian specialisation of sum–product on the chain (Kschischang et al., 2001; Bishop, 2006). Tweedie then gives the punchline of RQ2: *the k -th component of the joint diffusion score is an affine function of the Kalman-smoothed estimate of frame k* — score computation for Markov sequences is a smoothing problem, and seventy years of smoothing theory apply verbatim.

Machine-precision agreement. The computational companion runs BP against the matrix route of Chapter 7 over 80 configurations ($K \in \{2, 3, 5, 8, 16\}$, $\alpha \in \{0.2, 0.5, 0.9, -0.4\}$, $t \in \{0.05, 0.3, 1, 3\}$, random x): maximum disagreement 1.3×10^{-15} on posterior means, 3.9×10^{-15} on variances, 1.2×10^{-14} on scores — double-precision rounding accumulated along the recursion, nothing more. An independent implementation, written from scratch against the supervision-call specification alone, reproduces the same agreement; and a worked $K = 3$ instance with every message spelled out numerically is part of the companion notebooks. One qualitative detail of that instance is worth repeating: at $x = (1.2, -0.4, 0.8)$ the score component S_1 is *positive* although $x_1 < 0$ — the neighbours pull the posterior mean of the middle frame positive. No per-frame marginal score

can produce this; it is inter-frame evidence sharing, the joint effect of Section 7.3.3, in numbers.

9.3.1 The bulk fixed point: BP never breaks on the chain

Chapter 7 solved the bulk covariance by matrix algebra; the same formulas re-emerge from the message recursion, which is worth exhibiting because it localises *where* BP and AMP will differ. In the bulk, parametrise the forward message (evidence absorbed) by its precision λ_k ; Lemmas 1–2 give the scalar recursion

$$\lambda_{k+1} = J_d - \frac{\beta^2}{\lambda_k}, \quad (9.6)$$

with J_d, β the bulk parameters (7.17): absorbing one neighbour's summary costs the Gaussian-integration correction $-\beta^2/\lambda$. The fixed points are $\lambda^\pm = (J_d \pm \sqrt{J_d^2 - 4\beta^2})/2$, and since $J_d - 2|\beta| = e^{-2t}/\Delta_t + (1 - |\alpha|)^2/\sigma_\eta^2 > 0$ (the AM–GM margin of Chapter 7), the attractive upper fixed point

$$\lambda^* = \frac{J_d + \sqrt{J_d^2 - 4\beta^2}}{2} \quad (9.7)$$

exists for *every* $\alpha \in (-1, 1)$ and $t > 0$, with $|f'(\lambda^*)| = \beta^2/\lambda^{*2} < 1$: the sweep converges geometrically from any initialisation. Recombining the two directions and the local term, and using the fixed-point relation $\beta^2/\lambda^* = J_d - \lambda^*$,

$$\Lambda_{\text{bulk}} = J_d - \frac{2\beta^2}{\lambda^*} = 2\lambda^* - J_d = \sqrt{J_d^2 - 4\beta^2} \implies V_{\text{exact}} = \frac{1}{\sqrt{J_d^2 - 4\beta^2}}, \quad (9.8)$$

in exact agreement with Theorem 7.9. *Belief propagation on the Gaussian chain has no phase transition*: the cavity fixed point exists always, and the discriminant $J_d^2 - 4\beta^2$ is the quantity to watch as we now modify the closure.

9.4 AMP/TAP: same score, different variance, exact breakdown

AMP — the TAP equations of spin-glass theory (Thouless et al., 1977; Donoho et al., 2009; Zdeborová and Krzakala, 2016) — is the message passing one uses when the factor graph is *dense*: each message is then a sum of many weak contributions, approximately Gaussian by a central-limit argument, and one tracks per-*node* means and variances instead of per-*edge* messages, with the Onsager term correcting the self-influence. Rigorous derivations of AMP from BP exist in exactly that dense limit (Genovese and Piana, 2026). Our chain is the opposite regime in every respect — diagonal measurement, coupling in the prior, degree two — so it is the cleanest possible place to see what the per-node closure costs. Viewing the posterior as a Gaussian MRF with precision J and field h , the three schemes solve, at each node,

$$\text{mean field: } V_i = \frac{1}{J_{ii}}; \quad \text{AMP/TAP: } V_i = \frac{1}{J_{ii} - \sum_{k \neq i} J_{ik}^2 V_k}; \quad \text{BP: } V_i = (J^{-1})_{ii}. \quad (9.9)$$

The AMP closure is the BP cavity with the “exclude the receiving neighbour” correction dropped ($V_{k \rightarrow i} \approx V_k$): an $O(1/N)$ change on a dense graph, an $O(1)$ change on a chain with two neighbours.

Theorem 9.5 (The score is closure-independent). *For a Gaussian posterior, the fixed-point condition for the means is the same linear system $Jm = h$ under mean field, BP, and AMP. All three schemes hence return the exact posterior mean and — via Tweedie — the same, exact, joint score, for every (K, α, t) , regardless of whether their variance closures are accurate or even solvable.*

The proof is one line: in each scheme the mean update at node i reads $m_i \leftarrow (h_i - \sum_{k \neq i} J_{ik} m_k) / J_{ii}$ at the fixed point — the closures modify the uncertainty bookkeeping, not the linear relation among means — and the unique solution of $Jm = h$ is the exact posterior mean. Numerically: AMP means agree with exact means to 4.3×10^{-12} , and AMP scores with BP scores to 1.7×10^{-12} , across all tested chain configurations.

For the *variances*, the story splits. In the bulk the AMP closure (9.9) reads $V = 1/(J_d - 2\beta^2 V)$ — compare BP’s cavity $V_{\text{cav}} = 1/(J_d - \beta^2 V_{\text{cav}})$: *the entire difference between BP and AMP on the chain is a factor of two*, both neighbours included where BP excludes one.

Theorem 9.6 (AMP bulk variance, breakdown time, critical coupling). *The AMP bulk closure is the quadratic $2\beta^2 V^2 - J_d V + 1 = 0$, with physical root*

$$V_{\text{AMP}} = \frac{J_d - \sqrt{J_d^2 - 8\beta^2}}{4\beta^2}, \quad (9.10)$$

which exists iff $J_d \geq 2\sqrt{2}|\beta|$ — strictly stronger than BP’s always-true $J_d > 2|\beta|$. Since $J_d(t)$ decreases monotonically in t , the existence condition fails, if ever, past a single time; solving it as an equality (with $u := e^{-2t}$, $e^{-2t}/\Delta_t = u/(1-u)$) gives

$$t_c(\alpha) = -\frac{1}{2} \log \frac{g}{1+g}, \quad g(\alpha) = \frac{2\sqrt{2}|\alpha| - 1 - \alpha^2}{1 - \alpha^2}, \quad (9.11)$$

finite exactly when $g > 0$, i.e.

$$|\alpha| > \alpha_c = \sqrt{2} - 1 \approx 0.4142. \quad (9.12)$$

For $|\alpha| \leq \alpha_c$ the AMP variance has a fixed point at every diffusion time; for $|\alpha| > \alpha_c$ it ceases to exist for all $t > t_c(\alpha)$. In the weak-coupling regime the two closures agree to second order, with

$$V_{\text{AMP}} - V_{\text{exact}} = \frac{2\beta^4}{J_d^5} + O\left(\frac{\beta^6}{J_d^7}\right) > 0 : \quad (9.13)$$

AMP always overestimates the bulk variance, at fourth order in the coupling.

Toolbox 9.1 — Roots, branch, and the phase boundary

Branch selection. The quadratic has roots $V_{\pm} = (J_d \pm \sqrt{J_d^2 - 8\beta^2})/(4\beta^2)$; as $\beta \rightarrow 0$, expanding the square root gives $V_- \rightarrow 1/J_d$ (the decoupled answer) while V_+ diverges — V_- is the physical branch, and the attractive fixed point of the damped iteration

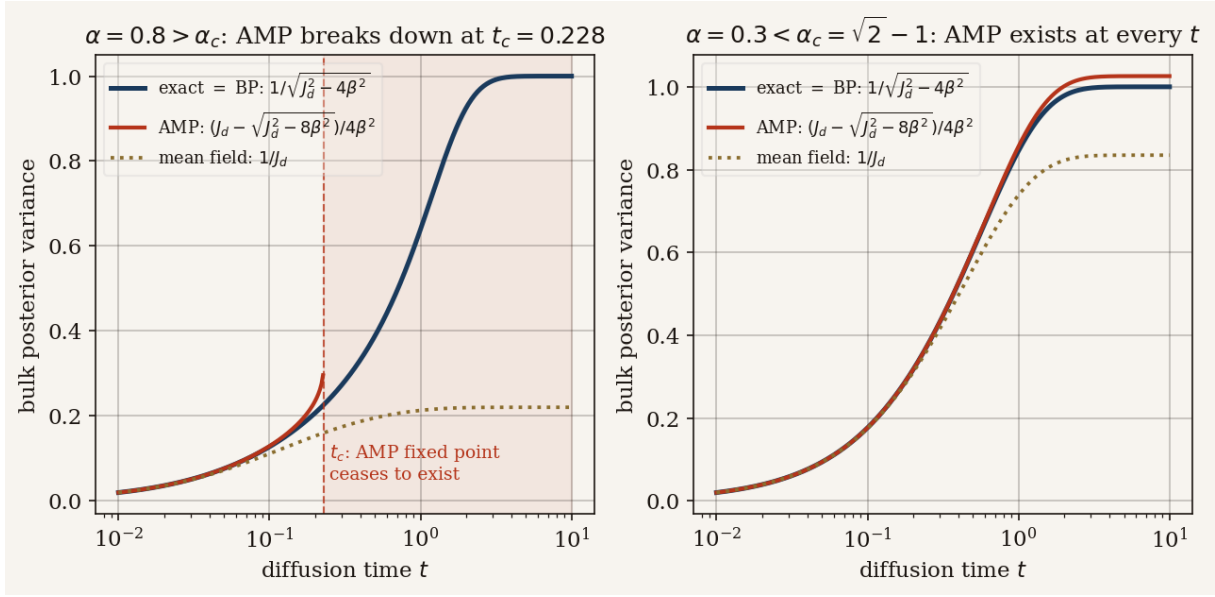


Figure 9.1: The three closures in closed form. Left ($\alpha = 0.8 > \alpha_c$): exact = BP, AMP (9.10), and mean field; the AMP branch terminates with a square-root singularity at the exact breakdown time $t_c(0.8) = 0.228$ from (9.11) (shaded: no fixed point). Right ($\alpha = 0.3 < \alpha_c$): AMP exists at every t and slightly overestimates the variance, as predicted by (9.13).

used in practice. *Critical coupling.* $g > 0 \iff \alpha^2 - 2\sqrt{2}|\alpha| + 1 < 0 \iff |\alpha| \in (\sqrt{2} - 1, \sqrt{2} + 1)$; intersecting with $|\alpha| < 1$ leaves $|\alpha| > \sqrt{2} - 1$. Sanity check at $t = \infty$: there the posterior is the bare prior, and the existence condition applied to Q_0 itself reads $(1 + \alpha^2) \geq 2\sqrt{2}|\alpha|$ — AMP applied to a bare AR(1) chain works iff $|\alpha| \leq \sqrt{2} - 1$, which is (9.12) read backwards. *Weak coupling.* With $\epsilon := \beta^2/J_d^2$: $(1 - 4\epsilon)^{-1/2} = 1 + 2\epsilon + 6\epsilon^2 + O(\epsilon^3)$ for the exact variance; $1 - \sqrt{1 - 8\epsilon} = 4\epsilon + 8\epsilon^2 + O(\epsilon^3)$, divided by $4\epsilon J_d$, gives $V_{\text{AMP}} = (1 + 2\epsilon + 8\epsilon^2 + O(\epsilon^3))/J_d$; subtract. $\square$

Numerically, on a $K = 12$ chain at $\alpha = 0.8$: the AMP score error stays at 10^{-13} – 10^{-12} for every t , while the variance error is 1.3×10^{-4} at $t = 0.05$, 3.5×10^{-2} at $t = 0.2$, and past $t \approx 0.23$ the variance update has no positive fixed point at all. The audit further verifies: the closed form (9.10) against the converged iteration to 2.5×10^{-12} ; the existence condition at all 180 points of an (α, t) scan; t_c against the bisected flip point of the iteration to 6.6×10^{-6} relative; convergence below α_c even at $t = 50$; and the

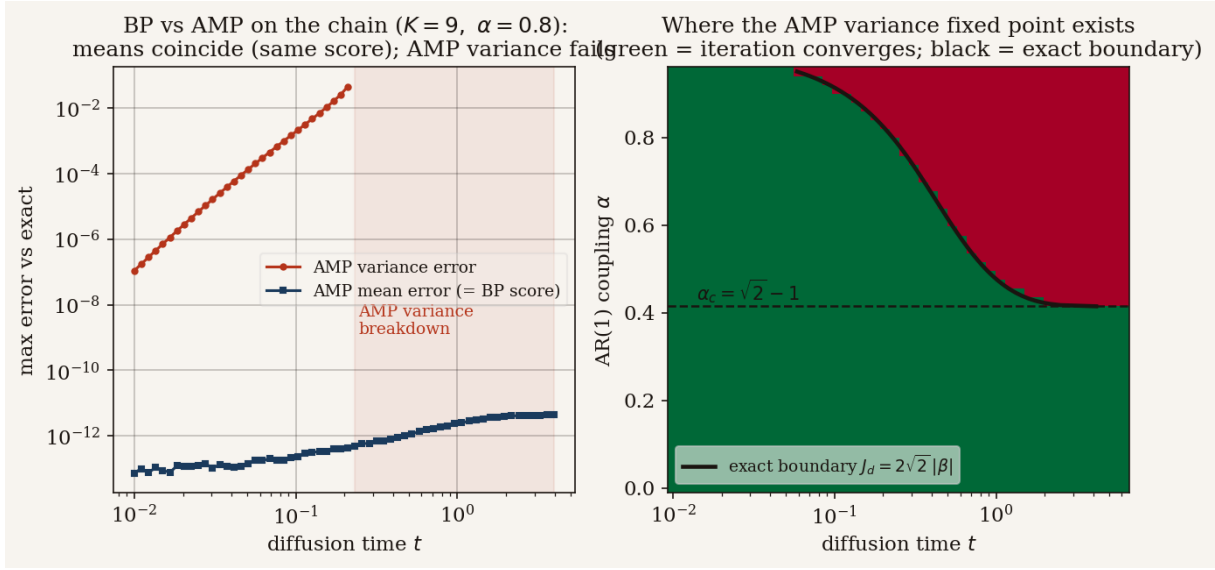


Figure 9.2: BP vs AMP on a finite chain ($K = 9$, $\alpha = 0.8$). Left: the AMP *mean* error ($\sim 10^{-12}$, flat in t : the score is closure-independent, Theorem 9.5) against the AMP *variance* error, which grows with t until the closure stops converging (shaded). Right: existence of the AMP variance fixed point in the (α, t) plane; the black curve is the exact boundary $t_c(\alpha)$ of (9.11), the dashed line $\alpha_c = \sqrt{2} - 1$; theory and iteration agree at all 180 scanned points.

weak-coupling ratio $(V_{\text{AMP}} - V_{\text{exact}})/(2\beta^4/J_d^5) \rightarrow 1$ within 0.4%.

The supervision remark that framed this section — the Gaussian message assumption is a CLT statement, and “a central limit theorem on two data points is usually not right” — is thus made exact, with both halves landing precisely where predicted. The *conjecture* (Gaussian message passing on the solved case recovers the known score) holds exactly: for BP by algebraic closure, for AMP because the mean fixed point is closure-independent. The *objection* targets the variance closure, and there it is not merely right but quantitative: a factor two in a discriminant, a phase boundary (9.11), and a critical coupling $\sqrt{2} - 1$. AMP is not wrong — on a dense benchmark (random $N = 600$ SPD precision) the same closure gets variances to 1.7×10^{-3} where mean field errs at 4×10^{-2} — it is built for a different graph.

9.5 The Gaussian closure beyond Gaussianity

On the Laplace chain of Chapter 8, BP is still exact in principle but its messages are genuinely infinite-dimensional (Remark 8.5). This is where the closure question becomes a measurement. Two solvers are compared on identical data.

Exact reference: grid BP. Messages are represented by their values on M equispaced points in $[-A, A]$ and updated by trapezoidal quadrature — option (iii) of Section 9.1. Calibration on the exactly solvable Gaussian chain shows the trapezoid rule on analytic, rapidly decaying integrands converges *spectrally* (aliasing error $\sim e^{-2\pi^2\sigma^2/dx^2}$), so failure is abrupt, not gradual: at $M = 101$, $A = 8$ the score error sits at the 10^{-15} floor down to $t = 0.02$ and fails only when the likelihood width $\sqrt{2t}$ approaches the grid step. The working configuration for all non-Gaussian experiments is $M = 401$, $A = 8$ (reference $M = 801$), with the resolution constraint $\sqrt{2t} \gtrsim 3 dx$ checked automatically; grid self-convergence error remains 10^4 – 10^7 times smaller than the effect being measured at every t . Within its validated regime, grid BP *is* the exact score.

The object under test: Gaussian-projected BP. The same sweeps, with every message forced into the Gaussian family by moment matching — the honest chain analogue of the AMP/TAP closure, and the algorithm one would actually deploy when quadrature is unaffordable.

The error instrument. All score errors are amplified posterior-mean errors: for any two solvers producing means $\widehat{m}, m_{\text{ref}}$ and scores by Tweedie,

$$\widehat{s} - s_{\text{ref}} = \frac{e^{-t}}{\Delta_t} (\widehat{m} - m_{\text{ref}}), \quad \frac{e^{-t}}{\Delta_t} \sim \frac{1}{2t} \quad (t \downarrow 0), \quad (9.14)$$

an exact identity (algebra, not approximation) whose numerical residual is logged in every experiment and sits at 10^{-15} – 10^{-12} throughout. Equation (9.14) explains *a priori* why small diffusion times are the danger zone: whatever the closure does to the posterior mean is amplified by $1/2t$ in the score.

9.5.1 Findings on the Laplace chain

On the Laplace chain ($\rho = 0.85$, $n = 40$, 60 trials per t , reference $M = 801$): the median relative score error of the Gaussian closure is

t	0.02	0.08	0.4	2.4
median relative score error	0.39	0.17	3.5×10^{-2}	7.6×10^{-5}

with failure probability $\mathbb{P}(\text{err} > 0.1)$ equal to 1 for $t \leq 0.08$ and 0 for $t \geq 0.9$ (Figure 9.3). The closure error is thus of order tens of percent in the small- t regime and decays below 10^{-3} once diffusion noise Gaussianises the tilted posteriors — the quantitative form of the supervision intuition that “at long diffusion times the world is Gaussian, so probably you have no problem”. Inspection of the worst trials shows the mechanism: the reference belief is sharply non-Gaussian (occasionally bimodal), the single Gaussian commits to a compromise mean, and (9.14) amplifies the shift. The score *direction*, however, remains far more accurate than its magnitude throughout: the median cosine similarity between projected and reference scores is 0.92 even at $t = 0.02$ (where the magnitude error is 39%), exceeds 0.99 for all $t \geq 0.12$, and the worst single trial of the whole sweep stays above 0.78.

9.5.2 Varying the innovation: what makes closure hard

Sweeping six innovation families with *identical* covariance $\rho^{|i-j|}$ — Laplace, Student- t ($\nu = 5, 8$), and symmetric Gaussian mixtures of increasing separation — isolates the shape of the innovation law as the only moving part. Three findings:

1. *Noise level dominates.* Every family’s error decays to $\sim 10^{-3}$ or below by $t = 2.4$; all meaningful closure error lives at $t \lesssim 0.4$.
2. *Both signs of non-Gaussianity hurt; bimodality hurts most.* At $t = 0.05$, $\rho = 0.5$: strongly bimodal mixture 0.94, Laplace 0.52, Student- $t_{\nu=5}$ 0.32, mildly bimodal mixture 0.07. A single Gaussian cannot represent two posterior modes, and commits to the gap between them.

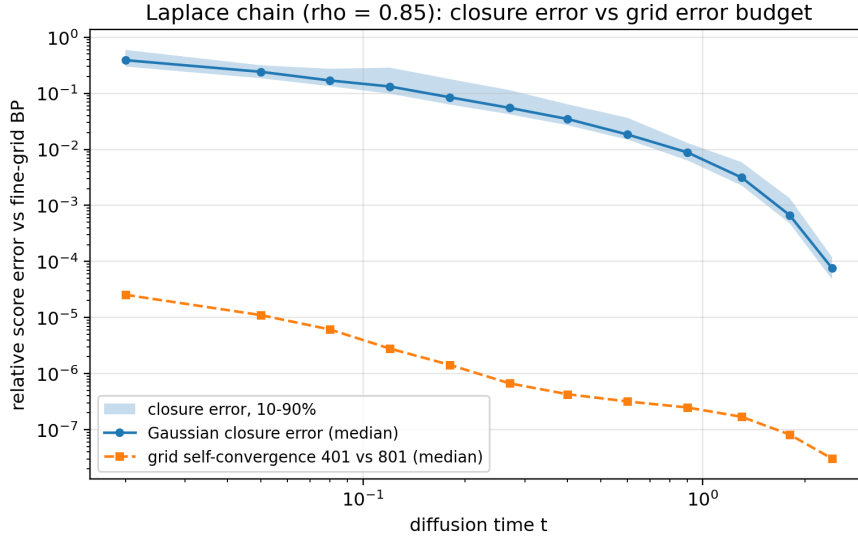


Figure 9.3: Gaussian closure error on the Laplace chain (median and 10–90% band) against the grid error budget measured on identical data ($\rho = 0.85$): the closure error dominates the reference error by 10^4 – 10^7 at every t , so the measurement is trustworthy where it is used.

3. *Stronger correlation helps, not hurts.* Laplace at $t = 0.05$: error 0.43, 0.27, 0.13 for $\rho = 0.5, 0.85, 0.95$. Informative neighbours act like extra Gaussian observations and Gaussianise the local tilted posterior; the naive intuition that message errors “propagate farther” at strong coupling is not what happens on chains.

9.5.3 Truncating the matrix versus truncating the inference

The last supervision question — take the exact score and truncate it to tridiagonal: how bad? — admits two inequivalent readings, and keeping them apart turns out to matter. *Truncate the answer* (variant a): band the matrix, $\hat{S} = -Q_t^{(b)}x$ with entries beyond $|i - j| \leq b$ zeroed. *Truncate the algorithm* (variant b): let messages travel at most r hops — exact inference on the window $x_{k-r}, \dots, x_{k+r}$, the local estimator of Section 7.5.1. Both estimators are linear in x , so their RMS errors over the data law are deterministic traces — no sampling. At equal locality budget ($b = r = 1$, $K = 40$, $\alpha = 0.8$):

relative RMS score error	$t = 0.05$	$t = 0.3$	$t = 1$	$t = 3$
banded matrix ($b = 1$)	0.210	0.302	0.135	0.0035
truncated inference ($r = 1$)	0.123	0.235	0.130	0.0035

Two lessons. First, the cost of locality is non-monotone in t and peaks at intermediate times — the same mid-diffusion window where the band of Q_t is fullest (Chapter 7) and where generative sampling spends its critical steps. Second, at small t *cutting the algorithm beats cutting the answer* (12.3% vs 21.0% at $t = 0.05$): the windowed inference solves a correctly normalised problem on its window, while zeroing entries of an inverse produces a matrix that is the inverse of nothing and misstates the diagonal. The two coincide at large t , where all off-diagonal content dies. The error of either truncation decays geometrically in the allowed range, at the rate $q(\alpha, t)$ of Theorem 7.10.

9.6 What this chapter established

Exact (proved, and verified to 10^{-12} – 10^{-15}): the factor graph of the noisy chain is a tree; Convention-A BP computes its marginals in $O(K)$; on the Gaussian chain all messages are Gaussian by algebraic closure — no CLT — and BP = Kalman smoother reproduces the matrix score; the bulk BP fixed point (9.7) exists for all (α, t) ; mean field, BP, and AMP share the exact score (Theorem 9.5); the AMP variance closure has the exact phase boundary (9.11)–(9.12) and the weak-coupling law (9.13). *Numerical* (measured against validated exact references): the Gaussian closure error on non-Gaussian chains — ~ 20 – 40% at small t , $< 10^{-3}$ at large t , worst for bimodal innovations, decreasing in ρ ; and the locality comparison, where truncated inference beats the truncated matrix at small t . *Heuristic*: the architecture reading — local, convolution-like score networks with receptive field sized by $q(\alpha, t)$ (or grown with t) are near-optimal outside the mid-diffusion window, and the exactly quantified penalty inside it is the price of locality, not an artefact. These findings, their limitations, and the follow-up programme are consolidated in the next chapter.

Chapter 10

Discussion and Conclusions

10.1 Summary of findings, by research question

RQ1 — the exact joint score and its lifecycle. For the stationary AR(1) chain under coordinatewise OU corruption, the joint score is the linear field $S(x, t) = -Q_t x$ with $Q_t = (e^{-2t}\Sigma_0 + \Delta_t I)^{-1}$, equal — by Tweedie’s identity, for any prior — to the amplified posterior-mean form $(e^{-t}\mathbb{E}[a|x] - x)/\Delta_t$. Its coupling structure is a complete, closed-form lifecycle: exactly tridiagonal at $t = 0$ (the Hammersley–Clifford fingerprint of Markovianity), band-filling at rate $(Q_t)_{i,i+d} = (-1)^{d-1}(2t)^{d-1}(Q_0^d)_{i,i+d} + O(t^d)$ — with a transcription error of an earlier note corrected and the correction audited — and returning to the identity at the universal rate e^{-2t} . In the bulk, one dimensionless dial $q(\alpha, t) \in (0, |\alpha|]$ governs simultaneously the posterior correlation length and the geometric decay of evidence influence, with the exactly right limits $q \rightarrow 0$ ($t \rightarrow 0$) and $q \rightarrow |\alpha|$ ($t \rightarrow \infty$).

RQ2 — local computation of a global object. The posterior of any Markov chain observed through per-frame noise lives on a tree-shaped factor graph, so sum–product message passing computes every marginal — hence every score component — exactly, in $O(K)$. On the Gaussian chain the messages remain Gaussian by *algebraic closure* (two lemmas: products and affine marginalisation), not by any central-limit argument; the resulting algorithm is, update for update, the Kalman filter plus the Rauch–Tung–

Striebel smoother, and it reproduces the matrix score to 10^{-14} . The joint diffusion score of a Markov sequence *is* a smoothing problem — a bridge that imports seventy years of estimation theory into generative modelling.

RQ3 — the price of the Gaussian closure. The answer splits into three cleanly separated statements. On the Gaussian chain, the *score* is closure-independent: mean field, BP, and AMP all solve the same linear system for the posterior mean, so all return the exact score. The *variance* closure is where AMP pays: the per-node (TAP) closure differs from the exact cavity by a single factor of two in the discriminant, which produces an exact breakdown time $t_c(\alpha)$ and a critical coupling $\alpha_c = \sqrt{2} - 1$ — a sharp, closed-form verdict on the “central limit theorem on two data points”. Beyond Gaussianity, on Laplace and other non-Gaussian chains measured against validated grid quadrature, the Gaussian message closure costs tens of percent of relative score error at small diffusion time (median 0.39 at $t = 0.02$; worst for bimodal innovations), decaying below 10^{-3} once the diffusion noise Gaussianises the posteriors — with the error amplified at small t by the exact factor $e^{-t}/\Delta_t \sim 1/2t$, and mitigated, counterintuitively, by *stronger* chain correlation.

RQ4 — implications for learned scores. Locality has a price and the price has a formula. The influence of evidence at distance d decays as $q(\alpha, t)^d$; the error of a radius- r local estimator decays as q^r ; and the penalty peaks in the mid-diffusion window, where the band of Q_t is fullest — precisely where generative sampling spends its critical steps. At equal locality budget, truncating the *inference* (windowed message passing) beats truncating the *matrix* (banding the score): 12.3% against 21.0% relative RMS at $t = 0.05$. For architecture design the readings are direct: local, convolution-like score heads are near-optimal at small and large t ; the receptive field should be sized as $r \gtrsim \xi \log(1/\varepsilon)$ with $\xi = 1/\log(1/q)$, or grown with diffusion time; and if a local computation must be used, it should be a local *algorithm*, not a banded linear readout.

10.2 Contributions to the literature

To the exactly solvable programme in the statistical physics of diffusion (Biroli and Mézard, 2023; Biroli et al., 2024; Ghio et al., 2024), this thesis adds the missing axis of *internal structure*: a solvable model in which a single data point has a dynamics of its own, and in which the interplay of internal correlation (α) and diffusion time (t) is charted exactly. To the message-passing literature (Mézard and Montanari, 2009; Zdeborová and Krzakala, 2016), it contributes a minimal, fully worked case where BP and AMP are *both* solvable and *provably different* — the factor-two discriminant, the phase boundary $t_c(\alpha)$, $\alpha_c = \sqrt{2} - 1$ — together with a quantitative audit of the Gaussian closure on chains, the regime where its classical CLT justification fails by construction. To the practice of diffusion modelling, it offers exact baselines: any learned score on Markov-like sequence data can now be compared, regime by regime, against a closed-form ground truth with known locality laws.

10.3 Limitations

Honesty about scope is part of the method. (i) The exact solutions concern *one-dimensional* state per frame and *linear* (AR(1)) dynamics; richer per-frame state keeps the tree structure but complicates the algebra. (ii) The non-Gaussian results are numerical, not closed-form: grid BP is an exact reference only within its validated resolution regime ($\sqrt{2t} \gtrsim 3dx$), and the Laplace closure error is a measured curve, not a formula. (iii) The locality laws are proved for the Gaussian bulk; their non-Gaussian counterparts are only structurally delimited (Chapter 8). (iv) No learning is performed: the architecture implications are readings of exact inference results, not training experiments. (v) An earlier conjecture on low-rank structure of the Laplace $K = 2$ Hessian proved regime-dependent and is deliberately not asserted. (vi) Reverse-generation dynamics — how score errors integrate along the sampling trajectory — was explored only preliminarily and is not claimed here.

10.4 Future work

Four directions follow naturally, in increasing ambition. *Mixture messages.* The single-Gaussian closure fails worst on bimodal beliefs; a two-component mixture closure on the same sweeps would separate the message-representation error from the model-mismatch error, and is the clearest next algorithmic step. *Discrete chains.* Replacing the continuous state with a finite alphabet (transition-matrix dynamics, OU noise only in the channel) makes BP messages finite vectors — exact inference with no closure at all — and connects directly to existing tree-reconstruction theory; this was the alternative route sketched in supervision. *Approximate Markovianity.* Real sequences are only approximately Markov; preliminary experiments with a chain-plus-global-latent model (rank-one residual) and with hybrid BP-plus-learned-residual scores suggest the BP baseline absorbs most of the structure at a fraction of the parameters, and deserve a systematic treatment. *Reverse dynamics.* Integrate the reverse SDE under exact, Gaussian-closed, and truncated scores, and measure where along the trajectory the closure and locality errors actually matter for sample quality — the pointwise-versus-dynamical error distinction.

10.5 Closing remark

The thesis began with a black box — a neural network asked to learn $\nabla \log p_t$ — and ended with a dial, $q(\alpha, t)$, and a phase boundary, $\alpha_c = \sqrt{2} - 1$. The distance between those two endpoints is the argument for exactly solvable models: when the data have structure, the score inherits it in a computable way, and both the algorithms that exploit it (message passing) and the shortcuts that betray it (closures, truncations) can be judged against ground truth rather than against each other. For sequential data with Markov structure, this programme is now complete at the Gaussian core and quantitatively mapped at its non-Gaussian rim; the frontier — mixtures, discreteness, approximate Markovianity, dynamics — is stated, and it is sharp.

Appendix A

Gaussian Identities Used Throughout

This appendix collects, with proofs, the handful of Gaussian identities on which Chapters 7–9 rely. Notation: a Gaussian density in *moment* parameters is $\mathcal{N}(a; m, v)$; in *natural* parameters it is $\exp(-\frac{\lambda}{2}a^2 + \theta a + \text{const})$ with $\lambda = 1/v$, $\theta = m/v$.

A.1 Quadratic forms and Gaussian normalisation

For $J \succ 0$ (symmetric positive definite) and any h ,

$$\int_{\mathbb{R}^K} \exp\left(-\frac{1}{2}a^\top J a + h^\top a\right) da = \frac{(2\pi)^{K/2}}{\sqrt{\det J}} \exp\left(\frac{1}{2}h^\top J^{-1}h\right), \quad (\text{A.1})$$

by completing the square, $-\frac{1}{2}a^\top J a + h^\top a = -\frac{1}{2}(a - J^{-1}h)^\top J(a - J^{-1}h) + \frac{1}{2}h^\top J^{-1}h$, and translating. Consequently a density $p(a) \propto \exp(-\frac{1}{2}a^\top J a + h^\top a)$ is the Gaussian with mean $m = J^{-1}h$ and covariance J^{-1} : *reading off (J, h) from a log-density is the fastest route to posterior parameters*, used in Proposition 7.5.

A.2 Products of Gaussians in the same variable

$$\mathcal{N}(a; m_1, v_1) \mathcal{N}(a; m_2, v_2) \propto \mathcal{N}(a; m, v), \quad \frac{1}{v} = \frac{1}{v_1} + \frac{1}{v_2}, \quad \frac{m}{v} = \frac{m_1}{v_1} + \frac{m_2}{v_2}. \quad (\text{A.2})$$

Proof. Add the exponents; the sum of two quadratics in a is a quadratic in a whose a^2 -coefficient is $-\frac{1}{2}(1/v_1 + 1/v_2)$ and whose linear coefficient is $m_1/v_1 + m_2/v_2$; apply (A.1) in one dimension. In natural parameters the rule is addition: $(\lambda, \theta) = (\lambda_1 + \lambda_2, \theta_1 + \theta_2)$ — which is why the BP updates of Chapter 9 are two additions per message. $\square$

A.3 Affine propagation (marginalisation)

If $a' = \alpha a + \eta$ with $\eta \sim \mathcal{N}(0, \sigma^2)$ independent of $a \sim \mathcal{N}(m, v)$, then

$$\int \mathcal{N}(a'; \alpha a, \sigma^2) \mathcal{N}(a; m, v) da = \mathcal{N}(a'; \alpha m, \alpha^2 v + \sigma^2). \quad (\text{A.3})$$

Proof. a' is a sum of independent Gaussians, hence Gaussian; its mean is αm and its variance $\alpha^2 v + \sigma^2$ by linearity of expectation and independence. Read in the reverse direction (solving for a given a Gaussian summary of a'), the same computation gives $\mathcal{N}(a; m'/\alpha, (v' + \sigma^2)/\alpha^2)$, the backward half-step of Chapter 9. $\square$

A.4 Gaussian conditioning

For a jointly Gaussian pair with precision blocks $\begin{pmatrix} J_{11} & J_{12} \\ J_{12}^T & J_{22} \end{pmatrix}$, the conditional of block 1 given block 2 is Gaussian with precision J_{11} (conditioning *reads off* the precision block) and mean shifted by $-J_{11}^{-1} J_{12} a_2$; marginalisation, dually, is trivial in covariance parameters. This precision/covariance duality — conditioning is local in J , marginalising is local in Σ — is the algebraic engine behind the cavity recursion (9.6): integrating a neighbour out of a tridiagonal precision produces the rank-one correction $-\beta^2/\lambda$ on the adjacent diagonal

entry, by the Schur complement

$$\begin{pmatrix} \lambda & \beta \\ \beta & J_d \end{pmatrix} \longrightarrow J_d - \frac{\beta^2}{\lambda}. \quad (\text{A.4})$$

A.5 Woodbury and the two routes to the score

The Woodbury/Sherman–Morrison–Woodbury identity, $(A+UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}$, specialised to $\Sigma_t = \Delta_t I + e^{-2t}\Sigma_0$, connects the noisy precision Q_t and the posterior precision $J = (e^{-2t}/\Delta_t)I + Q_0$:

$$Q_t = \frac{1}{\Delta_t} \left(I - \frac{e^{-2t}}{\Delta_t} J^{-1} \right), \quad (\text{A.5})$$

which is exactly the operator form of Tweedie’s identity: substituting the posterior mean $m = J^{-1}h = J^{-1}(e^{-t}/\Delta_t)x$ into $S = (e^{-t}m - x)/\Delta_t$ gives $S = -Q_t x$ identically. The machine-precision agreement of the two score routes (Section 7.3) is the numerical shadow of (A.5).

Appendix B

Code Listings

This appendix lists the core modules of the computational companion. They are dependency-light Python (NumPy/SciPy only), and every figure and numerical claim of Chapters 7–9 is generated by these modules together with the audit drivers described in Appendix C. The full repository layout is given there.

B.1 Gaussian chain: model and matrix-route score

Listing B.1: `ar1_utils.py` — AR(1) covariance, tridiagonal precision, OU channel, matrix-route score.

```
1  """
2  ar1_utils.py
3  =====
4
5  Gaussian AR(1) benchmark utilities.
6
7  Implements the closed-form objects that govern the joint score
8       $S(x, t) = \text{grad}_x \log P_t(x)$ 
9  of a stationary Gaussian AR(1) chain corrupted by a coordinate-wise
10 Ornstein-Uhlenbeck (OU) channel:
11
12       $a_{\{u+1\}} = \alpha * a_u + \text{eta}_u, \quad \text{eta}_u \sim N(0, \text{sigma\_eta}^2)$ 
13       $a_0 \sim N(\mu_0, \text{sigma}_0^2)$ 
14       $x_u = \exp(-t) a_u + \text{sqrt}(\text{Delta}_t) x_{i_u}, \quad x_{i_u} \sim N(0, 1)$ 
15       $\text{Delta}_t = 1 - \exp(-2 t)$ 
16
17  For stationarity we set  $\text{sigma}_0^2 = \text{sigma\_eta}^2 / (1 - \alpha^2)$ .
```

```

18
19 For Gaussian P_0 the joint score is exactly affine in x:
20  $S(x, t) = -\text{Sigma}_t^{-1} (x - \exp(-t) \mu_a)$ 
21 where  $\mu_a$  is the prior mean vector  $\mu_a[k] = \alpha^k \mu_0$ .
22
23 References:
24 main.pdf sections 2-5 (model, clean/noisy joint, score two ways)
25 and section 14 (three forms of  $Q_t$ , spectral preservation).
26 """
27
28 from __future__ import annotations
29
30 import numpy as np
31
32
33 # -----
34 # Forward-channel parameters
35 # -----
36
37 def ou_params(t: float) -> tuple[float, float]:
38     """Return ( $\mu_{\text{factor}}$ ,  $\Delta_t$ ) for the OU channel at time  $t$ .
39
40      $\mu_{\text{factor}} = \exp(-t)$  is the signal-attenuation coefficient, so that
41      $x = \mu_{\text{factor}} * a + \sqrt{\Delta_t} * \xi$ .
42     """
43     if t < 0:
44         raise ValueError("OU diffusion time must be non-negative.")
45     mu = np.exp(-t)
46     Delta = 1.0 - np.exp(-2.0 * t)
47     return float(mu), float(Delta)
48
49
50 # -----
51 # Clean Gaussian AR(1) statistics
52 # -----
53
54 def stationary_sigma_eta(alpha: float) -> float:
55     """Innovation std that yields unit stationary variance for AR(1).
56
57     Returns  $\sqrt{1 - \alpha^2}$  so that  $\text{Var}(a_u) = 1$  at stationarity.
58     """
59     if not -1.0 < alpha < 1.0:
60         raise ValueError("AR(1) coefficient |alpha| must be < 1.")
61     return float(np.sqrt(1.0 - alpha * alpha))
62
63
64 def ar1_covariance(K: int, alpha: float, sigma_eta: float | None = None,
65                   sigma_0: float | None = None) -> np.ndarray:
66     """Return the  $K \times K$  covariance  $\Sigma_0$  of the AR(1) chain.
67

```

```

68     If sigma_eta is None, it is set to sqrt(1 - alpha^2) (stationary).
69     If sigma_0 is None it is set so that the chain starts at stationarity:
70         sigma_0^2 = sigma_eta^2 / (1 - alpha^2).
71     The resulting Sigma_0 is the Toeplitz matrix
72         Sigma_0[i, j] = (sigma_eta^2 / (1 - alpha^2)) * alpha^{|i-j|}
73     in the stationary case.
74     """
75     if K < 1:
76         raise ValueError("K must be >= 1.")
77     if sigma_eta is None:
78         sigma_eta = stationary_sigma_eta(alpha)
79     if sigma_0 is None:
80         sigma_0 = sigma_eta / np.sqrt(1.0 - alpha * alpha)
81
82     # Build Var(a_u) by unrolling the recursion. Stationary case gives a
83     # constant column; non-stationary case uses the closed form
84     # Var(a_u) = alpha^{2u} sigma_0^2 + sigma_eta^2 (1 - alpha^{2u}) / (1 - alpha^2).
85     var_diag = np.empty(K)
86     var_diag[0] = sigma_0 * sigma_0
87     for u in range(1, K):
88         var_diag[u] = (alpha * alpha) * var_diag[u - 1] + sigma_eta * sigma_eta
89
90     Sigma_0 = np.empty((K, K))
91     for i in range(K):
92         for j in range(K):
93             Sigma_0[i, j] = (alpha ** abs(i - j)) * var_diag[min(i, j)]
94     return Sigma_0
95
96
97 def ar1_precision_clean(K: int, alpha: float,
98                        sigma_eta: float | None = None) -> np.ndarray:
99     """Tridiagonal precision Q_0 of the stationary AR(1) chain.
100
101     From the factorisation  $P_0(a) = p_0(a_0) \prod_u p(a_{u+1} | a_u)$ ,
102     the joint log-density is a sum of nearest-neighbour quadratics, so
103      $Q_0$  is tridiagonal with
104         diag interior =  $(1 + \alpha^2) / \sigma_{\eta}^2$ 
105         diag boundary =  $1 / \sigma_{\eta}^2$ 
106         off-diag      =  $-\alpha / \sigma_{\eta}^2$ 
107     """
108     if sigma_eta is None:
109         sigma_eta = stationary_sigma_eta(alpha)
110     s2 = sigma_eta * sigma_eta
111     Q = np.zeros((K, K))
112     for k in range(K):
113         if k == 0 or k == K - 1:
114             Q[k, k] = 1.0 / s2
115         else:
116             Q[k, k] = (1.0 + alpha * alpha) / s2
117         if k + 1 < K:

```

```

118         Q[k, k + 1] = -alpha / s2
119         Q[k + 1, k] = -alpha / s2
120     return Q
121
122
123 def prior_mean(K: int, alpha: float, mu_0: float = 0.0) -> np.ndarray:
124     """Prior mean  $\mu_a[k] = \alpha^k \mu_0$  for the AR(1) chain.
125
126     For the stationary, zero-mean case this is identically zero.
127     """
128     return mu_0 * np.power(alpha, np.arange(K))
129
130
131 # -----
132 # Noisy covariance and precision under OU corruption
133 # -----
134
135 def sigma_t(Sigma_0: np.ndarray, t: float) -> np.ndarray:
136     """Noisy covariance  $\Sigma_t = \exp(-2t) \Sigma_0 + (1 - \exp(-2t)) I$ ."""
137     _, Delta = ou_params(t)
138     decay = 1.0 - Delta
139     K = Sigma_0.shape[0]
140     return decay * Sigma_0 + Delta * np.eye(K)
141
142
143 def precision_t(Sigma_0: np.ndarray, t: float) -> np.ndarray:
144     """Noisy precision  $Q_t = \Sigma_t^{-1}$  computed by direct inversion."""
145     return np.linalg.inv(sigma_t(Sigma_0, t))
146
147
148 def precision_t_spectral(Sigma_0: np.ndarray, t: float
149                         ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
150     """Return  $Q_t$  via the eigenbasis of  $\Sigma_0$ .
151
152      $\Sigma_0 = U \text{diag}(\text{lam}_0) U^T$ , then
153          $\Sigma_t = U \text{diag}(\exp(-2t) \text{lam}_0 + \text{Delta}_t) U^T$ ,
154          $Q_t = U \text{diag}(1 / (\exp(-2t) \text{lam}_0 + \text{Delta}_t)) U^T$ .
155     Returns  $(Q_t, U, \text{lam}_t)$  where  $\text{lam}_t$  are the noisy eigenvalues.
156     """
157     lam_0, U = np.linalg.eigh(Sigma_0)
158     mu, Delta = ou_params(t)
159     lam_t = (mu * mu) * lam_0 + Delta
160     Q_t = (U * (1.0 / lam_t)) @ U.T
161     return Q_t, U, lam_t
162
163
164 # -----
165 # Joint score
166 # -----
167

```

```

168 def joint_score_matrix(x: np.ndarray, t: float, Sigma_0: np.ndarray,
169                        alpha: float, mu_0: float = 0.0) -> np.ndarray:
170     """Return  $S(x, t) = -\Sigma_t^{-1} (x - \exp(-t) \mu_a)$ .
171
172     This is the exact joint score for the Gaussian AR(1) chain at OU
173     diffusion time  $t$ . Linear in  $x$ .
174     """
175     K = Sigma_0.shape[0]
176     if x.shape[-1] != K:
177         raise ValueError("x has incompatible last dimension with Sigma_0.")
178     mu, _ = ou_params(t)
179     mu_a = prior_mean(K, alpha, mu_0)
180     Q_t = precision_t(Sigma_0, t)
181     return -(x - mu * mu_a) @ Q_t # works for x of shape (... ,K)
182
183
184 # -----
185 # Posterior mean  $E[a | x]$  via the matrix form (used by the audit)
186 # -----
187
188 def gaussian_posterior(x: np.ndarray, t: float, Sigma_0: np.ndarray,
189                       alpha: float, mu_0: float = 0.0
190                       ) -> tuple[np.ndarray, np.ndarray]:
191     """Return  $(\mu_{\text{post}}, \Sigma_{\text{post}})$  for  $p(a | x)$  under Gaussian AR(1) + OU.
192
193     The posterior is Gaussian with
194          $\Sigma_{\text{post}}^{-1} = (\exp(-2t) / \Delta_t) I + \Sigma_0^{-1}$ 
195          $\mu_{\text{post}} = \Sigma_{\text{post}} * ((\exp(-t) / \Delta_t) x + \Sigma_0^{-1} \mu_a)$ 
196     Used to cross-check the BP recursion in numerical_audit.py.
197     """
198     K = Sigma_0.shape[0]
199     mu, Delta = ou_params(t)
200     Sigma_0_inv = np.linalg.inv(Sigma_0)
201     prec_post = (mu * mu / Delta) * np.eye(K) + Sigma_0_inv
202     Sigma_post = np.linalg.inv(prec_post)
203     mu_a = prior_mean(K, alpha, mu_0)
204     mu_post = Sigma_post @ ((mu / Delta) * x + Sigma_0_inv @ mu_a)
205     return mu_post, Sigma_post
206
207
208 def joint_score_via_tweedie(x: np.ndarray, t: float, Sigma_0: np.ndarray,
209                            alpha: float, mu_0: float = 0.0) -> np.ndarray:
210     """Return  $S(x, t)$  computed through the Tweedie identity
211
212          $S_k(x, t) = (\exp(-t) E[a_k | x] - x_k) / \Delta_t$ 
213
214     using the exact Gaussian posterior mean. Equivalent to
215     joint_score_matrix() up to floating-point arithmetic; checking the
216     two forms agree is part of the numerical audit.
217     """

```

```

218     mu, Delta = ou_params(t)
219     mu_post, _ = gaussian_posterior(x, t, Sigma_0, alpha, mu_0)
220     return (mu * mu_post - x) / Delta
221
222
223 # -----
224 # Self-test when executed directly
225 # -----
226
227 if __name__ == "__main__":
228     rng = np.random.default_rng(0)
229     K = 8
230     alpha = 0.7
231     Sigma_0 = ar1_covariance(K, alpha)
232     Q_0 = ar1_precision_clean(K, alpha)
233
234     # Tridiagonality at t = 0
235     err = np.linalg.norm(Q_0 - np.linalg.inv(Sigma_0))
236     print(f"||Q_0 - Sigma_0^-1|| = {err:.2e}")
237
238     # Spectral preservation
239     t = 0.4
240     Q_a = precision_t(Sigma_0, t)
241     Q_b, _, _ = precision_t_spectral(Sigma_0, t)
242     print(f"||Q_t direct - Q_t spectral|| = {np.linalg.norm(Q_a - Q_b):.2e}")
243
244     # Score consistency
245     x = rng.standard_normal(K)
246     s1 = joint_score_matrix(x, t, Sigma_0, alpha)
247     s2 = joint_score_via_tweedie(x, t, Sigma_0, alpha)
248     print(f"||S_matrix - S_tweedie|| = {np.linalg.norm(s1 - s2):.2e}")

```

B.2 Gaussian chain: belief propagation (Convention A)

Listing B.2: `bp_score.py` — forward/backward Gaussian message passing, posterior combination, Tweedie score.

```

1  """
2  bp_score.py
3  =====
4
5  Belief propagation on the Gaussian AR(1) chain with independent OU noise

```

```

6 on each frame. Returns the posterior mean  $E[a_k | x]$  at every frame and
7 the joint score
8
9  $S_k(x, t) = (\exp(-t) E[a_k | x] - x_k) / \Delta_t$ 
10
11 via Tweedie's identity. The construction is derived in main.pdf,
12 section 8 (Convention A: local evidence enters at combination time,
13 never inside a recursion); this implementation is validated by
14 numerical_audit.py against the matrix posterior. Cost is  $O(K)$ .
15
16 The factor graph of the chain is a tree, the AR(1) prior is Gaussian
17 and the OU corruption is independent per frame, so all messages remain
18 Gaussian. Each message is therefore represented by two scalars (mean,
19 variance); a variance of +infty represents the uninformative initial
20 backward message and is handled exactly.
21
22 References:
23     main.pdf sections 8-10 (Convention A, K=3 worked example, Kalman/RTS).
24     """
25
26 from __future__ import annotations
27
28 import numpy as np
29
30 from ar1_utils import ou_params, stationary_sigma_eta
31
32
33 # -----
34 # Local evidence in 'a' coordinates
35 # -----
36
37 def _evidence_in_a(x_k: float, t: float) -> tuple[float, float]:
38     """Return (mean, variance) of the evidence factor  $p_t(x_k | a_k)$ 
39     viewed as a Gaussian in  $a_k$ .
40
41      $p_t(x_k | a_k)$  propto  $N(a_k ; \exp(t) x_k, \exp(2 t) \Delta_t)$ .
42     """
43     mu, Delta = ou_params(t)
44     inv_mu = 1.0 / mu #  $\exp(t)$ 
45     return inv_mu * x_k, inv_mu * inv_mu * Delta
46
47
48 # -----
49 # Gaussian product helper -- robust to  $v = +\infty$  (uninformative)
50 # -----
51
52 def _gaussian_product(m1: float, v1: float, m2: float, v2: float
53                       ) -> tuple[float, float]:
54     """Return (m, v) for the Gaussian product  $N(m1, v1) * N(m2, v2)$ ."""
55     if np.isinf(v1):

```



```

56     return m2, v2
57 if np.isinf(v2):
58     return m1, v1
59 inv_v = 1.0 / v1 + 1.0 / v2
60 v = 1.0 / inv_v
61 m = v * (m1 / v1 + m2 / v2)
62 return m, v
63
64
65 # -----
66 # Forward (Kalman filter) and backward (RTS smoother) recursions
67 # -----
68
69 def _forward_pass(x: np.ndarray, t: float, alpha: float, sigma_eta: float,
70                 mu_0: float, sigma_0: float
71                 ) -> tuple[np.ndarray, np.ndarray]:
72     """Convention A forward messages  $\mu_{\rightarrow k}(a_k) = N(a_k ; m_{to}[k], v_{to}[k])$ .
73
74     The forward message at node k carries the evidence  $x_0, \dots, x_{k-1}$ 
75     only; the local evidence at node k is inserted at combination time
76     (so this code never double-counts).
77     """
78     K = len(x)
79     m_to = np.empty(K)
80     v_to = np.empty(K)
81     m_to[0] = mu_0
82     v_to[0] = sigma_0 * sigma_0
83
84     for k in range(K - 1):
85         # Combine forward message with local evidence at node k
86         m_ev, v_ev = _evidence_in_a(x[k], t)
87         m_c, v_c = _gaussian_product(m_to[k], v_to[k], m_ev, v_ev)
88         # Predict to k+1 through the AR(1) transition
89         m_to[k + 1] = alpha * m_c
90         v_to[k + 1] = alpha * alpha * v_c + sigma_eta * sigma_eta
91     return m_to, v_to
92
93
94 def _backward_pass(x: np.ndarray, t: float, alpha: float, sigma_eta: float
95                  ) -> tuple[np.ndarray, np.ndarray]:
96     """Convention A backward messages  $\mu_{\leftarrow k}(a_k)$  prop to  $N(m_{lr}[k], v_{lr}[k])$ .
97
98     The backward message at node k carries  $x_{k+1}, \dots, x_{K-1}$ ; node K-1
99     receives a flat (uninformative,  $v = +\infty$ ) initial message.
100     """
101     if np.isclose(alpha, 0.0):
102         raise ValueError("Backward pass divides by alpha; choose |alpha| > 0.")
103     K = len(x)
104     m_lr = np.empty(K)
105     v_lr = np.empty(K)

```

```

106     m_lr[K - 1] = 0.0
107     v_lr[K - 1] = np.inf
108
109     for k in range(K - 1, 0, -1):
110         # Combine backward message with local evidence at node k
111         m_ev, v_ev = _evidence_in_a(x[k], t)
112         m_c, v_c = _gaussian_product(m_lr[k], v_lr[k], m_ev, v_ev)
113         # Back-propagate one AR(1) step:  $a_k = \alpha a_{k-1} + \eta_{k-1}$ 
114         m_lr[k - 1] = m_c / alpha
115         v_lr[k - 1] = (sigma_eta * sigma_eta + v_c) / (alpha * alpha)
116     return m_lr, v_lr
117
118
119 def bp_posterior(x: np.ndarray, t: float, alpha: float,
120                 sigma_eta: float | None = None,
121                 mu_0: float = 0.0, sigma_0: float | None = None
122                 ) -> tuple[np.ndarray, np.ndarray]:
123     """Return (mu_post, var_post) for  $p(a_k | x)$ , shape (K,).
124
125     Combines forward, local evidence, and backward messages at every
126     node. mu_post[k] equals  $E[a_k | x]$  (the Kalman-smoothed mean).
127     """
128     if sigma_eta is None:
129         sigma_eta = stationary_sigma_eta(alpha)
130     if sigma_0 is None:
131         sigma_0 = sigma_eta / np.sqrt(1.0 - alpha * alpha)
132
133     x = np.asarray(x, dtype=float)
134     m_to, v_to = _forward_pass(x, t, alpha, sigma_eta, mu_0, sigma_0)
135     m_lr, v_lr = _backward_pass(x, t, alpha, sigma_eta)
136
137     K = len(x)
138     mu_post = np.empty(K)
139     var_post = np.empty(K)
140     for k in range(K):
141         m_ev, v_ev = _evidence_in_a(x[k], t)
142         m_a, v_a = _gaussian_product(m_to[k], v_to[k], m_ev, v_ev)
143         m_b, v_b = _gaussian_product(m_a, v_a, m_lr[k], v_lr[k])
144         mu_post[k] = m_b
145         var_post[k] = v_b
146     return mu_post, var_post
147
148
149 def bp_score(x: np.ndarray, t: float, alpha: float,
150             sigma_eta: float | None = None,
151             mu_0: float = 0.0, sigma_0: float | None = None
152             ) -> np.ndarray:
153     """Return the joint score  $S(x, t)$  computed via belief propagation.
154
155     Tweedie's identity:

```

```

156     S_k(x, t) = (exp(-t) mu_post[k] - x_k) / Delta_t.
157     """
158     mu_post, _ = bp_posterior(x, t, alpha, sigma_eta, mu_0, sigma_0)
159     mu, Delta = ou_params(t)
160     return (mu * mu_post - np.asarray(x, dtype=float)) / Delta
161
162
163 # -----
164 # Self-test
165 # -----
166
167 if __name__ == "__main__":
168     from ar1_utils import ar1_covariance, gaussian_posterior, joint_score_matrix
169
170     rng = np.random.default_rng(1)
171     K, alpha, t = 8, 0.7, 0.4
172     Sigma_0 = ar1_covariance(K, alpha)
173     x = rng.standard_normal(K)
174
175     mu_bp, var_bp = bp_posterior(x, t, alpha)
176     mu_mat, Sigma_mat = gaussian_posterior(x, t, Sigma_0, alpha)
177     print(f"max |mu_bp - mu_matrix| = {np.max(np.abs(mu_bp - mu_mat)):.2e}")
178     print(f"max |var_bp - diag(Sigma_matrix)| = "
179           f"{np.max(np.abs(var_bp - np.diag(Sigma_mat))):.2e}")
180
181     s_bp = bp_score(x, t, alpha)
182     s_matrix = joint_score_matrix(x, t, Sigma_0, alpha)
183     print(f"max |S_bp - S_matrix| = {np.max(np.abs(s_bp - s_matrix)):.2e}")

```

B.3 Closed-form bulk formulas

Listing B.3: `chain_formulas.py` — bulk parameters (J_d, β) , exact variance V , decay rate q , BP fixed point, AMP variance, breakdown time $t_c(\alpha)$, critical coupling.

```

1  """
2  chain_formulas.py
3  =====
4
5  Closed-form results for the HOMOGENEOUS INTERIOR (bulk) of the posterior
6  precision of the AR(1)+OU model, and the fully explicit  $K = 2$  case.
7
8  Everything in this module is an explicit formula -- no iteration, no matrix
9  inversion. Each formula is verified against brute force in
10  'numerical_audit.py' (sections 10 and 11); the derivations are in the
11  companion note 'main.pdf':

```

```

12
13 Bulk of the chain (interior of a long chain, far from both boundaries).
14 The posterior precision  $J = (e^{-2t}/\Delta_t) I + Q_0$  is tridiagonal
15 Toeplitz in the interior, with
16
17  $J_d = e^{-2t}/\Delta_t + (1 + \alpha^2) / \sigma_\eta^2$  (diagonal)
18  $\beta = -\alpha / \sigma_\eta^2$  (off-diagonal),
19
20 and  $\sigma_\eta^2 = 1 - \alpha^2$  in the stationary unit-variance normalisation.
21
22 * Exact bulk posterior variance (= what BP returns in the bulk):
23
24  $V_{\text{exact}} = 1 / \sqrt{J_d^2 - 4 \beta^2}$ .
25
26 * Gaussian-BP cavity (message) precision fixed point:
27
28  $\lambda^* = (J_d + \sqrt{J_d^2 - 4 \beta^2}) / 2$ ,
29
30 which exists for EVERY  $(\alpha, t)$  because  $J_d \geq 2 |\beta|$  always
31 (AM-GM:  $1 + \alpha^2 \geq 2|\alpha|$ , plus the positive evidence term).
32 Recombining,  $J_d - 2 \beta^2 / \lambda^* = \sqrt{J_d^2 - 4 \beta^2}$ ,
33 i.e. BP reproduces  $V_{\text{exact}}$  -- BP never breaks down on the chain.
34
35 * AMP/TAP variance fixed point (cavity WITHOUT the exclusion):
36 the closure  $V = 1 / (J_d - 2 \beta^2 V)$  is the quadratic
37  $2 \beta^2 V^2 - J_d V + 1 = 0$ , whose physical root is
38
39  $V_{\text{amp}} = (J_d - \sqrt{J_d^2 - 8 \beta^2}) / (4 \beta^2)$ ,
40
41 and which has a real solution iff  $J_d \geq 2 \sqrt{2} |\beta|$ .
42 Below that line in the  $(\alpha, t)$  plane the AMP variance has no
43 physical fixed point at all (breakdown). Note the factor  $8 = 2*4$ :
44 AMP needs a strictly stronger condition than BP.
45
46  $K = 2$  explicitly. With unit stationary variance,  $\Sigma_0 = \begin{bmatrix} 1 & a \\ a & 1 \end{bmatrix}$ ,
47
48  $\Sigma_t = \begin{bmatrix} 1 & \alpha e^{-2t} \\ \alpha e^{-2t} & 1 \end{bmatrix}$ ,
49  $Q_t = 1/(1 - \alpha^2 e^{-4t}) \begin{bmatrix} 1 & -\alpha e^{-2t} \\ -\alpha e^{-2t} & 1 \end{bmatrix}$ ,
50
51  $S_0(x) = -(x_0 - \alpha e^{-2t} x_1) / (1 - \alpha^2 e^{-4t})$ ,
52
53 and symmetrically for  $S_1$ . The cross term  $-\alpha e^{-2t} x_1$  in  $S_0$  is the
54 inter-frame coupling that the per-frame marginal score misses entirely.
55 """
56
57 from __future__ import annotations
58
59 import numpy as np
60
61 from ar1_utils import ou_params

```

```

62
63
64 # -----
65 # Bulk (homogeneous interior) parameters of the posterior precision
66 # -----
67
68 def bulk_params(alpha: float, t: float) -> tuple[float, float]:
69     """(J_d, beta): diagonal and off-diagonal of the interior of J.
70
71      $J = (e^{-2t}/\Delta_t) I + Q_0$  with  $Q_0$  the clean tridiagonal precision;
72     in the interior  $Q_0$  has diagonal  $(1+\alpha^2)/\sigma_{\eta}^2$  and off-diagonal
73      $-\alpha/\sigma_{\eta}^2$ , with  $\sigma_{\eta}^2 = 1 - \alpha^2$ .
74     """
75     mu, Delta = ou_params(t)
76     s2 = 1.0 - alpha * alpha
77     J_d = mu * mu / Delta + (1.0 + alpha * alpha) / s2
78     beta = -alpha / s2
79     return J_d, beta
80
81
82 def bulk_variance_exact(alpha: float, t: float) -> float:
83     """Exact posterior variance in the bulk:  $1 / \sqrt{J_d^2 - 4 \beta^2}$ .
84
85     This is also what Gaussian BP returns in the bulk (BP is exact on the
86     chain); the identity  $J_d - 2 \beta^2/\lambda^* = \sqrt{J_d^2 - 4 \beta^2}$ 
87     is checked in the audit.
88     """
89     J_d, beta = bulk_params(alpha, t)
90     return 1.0 / np.sqrt(J_d * J_d - 4.0 * beta * beta)
91
92
93 def bp_cavity_precision(alpha: float, t: float) -> float:
94     """BP message-precision fixed point  $\lambda^* = (J_d + \sqrt{J_d^2 - 4 \beta^2})/2$ .
95
96     The discriminant  $J_d^2 - 4 \beta^2$  is positive for every (alpha, t):
97     BP always has a (unique attractive) fixed point on the chain.
98     """
99     J_d, beta = bulk_params(alpha, t)
100    return 0.5 * (J_d + np.sqrt(J_d * J_d - 4.0 * beta * beta))
101
102
103 def amp_bulk_variance(alpha: float, t: float) -> float:
104     """AMP/TAP bulk variance fixed point, or NaN when none exists.
105
106      $V_{\text{amp}} = (J_d - \sqrt{J_d^2 - 8 \beta^2}) / (4 \beta^2)$ ,  $J_d \geq 2\sqrt{2}|\beta|$ ;
107     NaN otherwise (no physical fixed point: AMP breakdown).
108
109     The physical root is the '-' branch: it is the one continuously
110     connected to the decoupled limit  $V = 1/J_d$  as  $\beta \rightarrow 0$ .
111     """

```

```

112     J_d, beta = bulk_params(alpha, t)
113     disc = J_d * J_d - 8.0 * beta * beta
114     if disc < 0.0:
115         return float("nan")
116     return (J_d - np.sqrt(disc)) / (4.0 * beta * beta)
117
118
119 def amp_fixed_point_exists(alpha: float, t: float) -> bool:
120     """Existence condition of the AMP variance closure:  $J_d \geq 2 \sqrt{2} |\beta|$ ."""
121     J_d, beta = bulk_params(alpha, t)
122     return J_d * J_d - 8.0 * beta * beta >= 0.0
123
124
125 def amp_critical_time(alpha: float) -> float:
126     """Exact diffusion time at which the AMP variance fixed point disappears.
127
128     Solving  $J_d(\alpha, t) = 2 \sqrt{2} |\beta(\alpha)|$  for  $t$ , with
129      $u := e^{-2t}$  and  $e^{-2t}/\Delta_t = u/(1-u)$ :
130
131      $g(\alpha) = (2 \sqrt{2} |\alpha| - 1 - \alpha^2) / (1 - \alpha^2)$ ,
132      $t_c = -(1/2) \log(g / (1 + g))$  if  $g > 0$ ,
133      $t_c = +\infty$  if  $g \leq 0$ .
134
135      $g > 0$  iff  $\alpha^2 - 2 \sqrt{2} |\alpha| + 1 < 0$  iff  $|\alpha| > \sqrt{2} - 1$ .
136     Hence the CRITICAL COUPLING
137
138      $\alpha_c = \sqrt{2} - 1 = 0.41421\dots$ :
139
140     for  $|\alpha| \leq \alpha_c$  the AMP variance has a fixed point at EVERY  $t$ ;
141     for  $|\alpha| > \alpha_c$  it breaks down for all  $t > t_c(\alpha)$ .
142     Verified against the flip point of the self-consistent iteration
143     (bisection,  $K=300$ ) to 4-5 digits in the audit.
144     """
145     a = abs(alpha)
146     g = (2.0 * np.sqrt(2.0) * a - 1.0 - a * a) / (1.0 - a * a)
147     if g <= 0.0:
148         return float("inf")
149     return -0.5 * np.log(g / (1.0 + g))
150
151
152 def bulk_correlation_decay(alpha: float, t: float) -> float:
153     """Posterior correlation decay rate  $q$  in  $(0, 1)$ :
154
155      $q = (J_d - \sqrt{J_d^2 - 4 \beta^2}) / (2 |\beta|)$ .
156
157     The bulk posterior covariance decays geometrically,
158      $(J^{-1})_{i, i+d} = q^d / \sqrt{J_d^2 - 4 \beta^2}$ ,
159     so  $1/\log(1/q)$  is the correlation length of the posterior in frames.
160     The same  $q$  is the decay rate of the locality (truncation) error of the
161     radius- $r$  local estimator -- see local_bp.rms_truncation_error and the

```

```

162     audit.  $q \rightarrow 0$  at small  $t$  (evidence pins each frame: posterior almost
163     local) and  $q \rightarrow$  alpha-dependent limit at large  $t$  (prior-dominated).
164     """
165     J_d, beta = bulk_params(alpha, t)
166     return (J_d - np.sqrt(J_d * J_d - 4.0 * beta * beta)) / (2.0 * abs(beta))
167
168
169 def bulk_covariance(alpha: float, t: float, d: int) -> float:
170     """Closed-form bulk posterior covariance  $\text{Cov}(a_i, a_{i+d} | x)$ :
171
172          $(J^{-1})_{i, i+d} = q^d / \sqrt{J_d^2 - 4 \beta^2}$ .
173
174     Verified against brute-force inversion at the centre of a long chain
175     in the audit (machine precision).
176     """
177     return bulk_variance_exact(alpha, t) * bulk_correlation_decay(alpha, t) ** d
178
179
180 def amp_weak_coupling_error(alpha: float, t: float) -> float:
181     """Leading weak-coupling AMP variance error:
182
183          $V_{\text{amp}} - V_{\text{exact}} = 2 \beta^4 / J_d^5 + O(\beta^6 / J_d^7)$ .
184
185     Both closures agree to  $O(\beta^2/J_d^2)$ ; AMP first deviates -- always
186     OVERestimating the variance -- at fourth order in the coupling.
187     """
188     J_d, beta = bulk_params(alpha, t)
189     return 2.0 * beta ** 4 / J_d ** 5
190
191
192 # -----
193 #  $K = 2$ : everything fully explicit
194 # -----
195
196 def k2_sigma_t(alpha: float, t: float) -> np.ndarray:
197     """Noised covariance for  $K=2$ :  $[[1, \alpha e^{-2t}], [\alpha e^{-2t}, 1]]$ .
198
199     Diagonal:  $e^{-2t} * 1 + \Delta_t = 1$  (variance-preserving channel on a
200     unit-variance frame). Off-diagonal:  $e^{-2t} * \alpha$  (the noise is
201     independent across frames, so only the signal part correlates).
202     """
203     r = alpha * np.exp(-2.0 * t)
204     return np.array([[1.0, r], [r, 1.0]])
205
206
207 def k2_precision_t(alpha: float, t: float) -> np.ndarray:
208     """Closed-form inverse of  $k2\_sigma\_t$  ( $2 \times 2$  inversion)."""
209     r = alpha * np.exp(-2.0 * t)
210     return np.array([[1.0, -r], [-r, 1.0]]) / (1.0 - r * r)
211

```

```

212
213 def k2_score(x: np.ndarray, alpha: float, t: float) -> np.ndarray:
214     """Joint score for K=2 in closed form:
215
216          $S_0 = -(x_0 - r x_1) / (1 - r^2), \quad r = \alpha e^{-2t},$ 
217
218         and symmetrically for  $S_1$ . The  $r x_1$  term in  $S_0$  is the inter-frame
219         coupling; the per-frame marginal score is just  $-x_0$  (unit variance)
220         and misses it.
221     """
222     x = np.asarray(x, dtype=float)
223     r = alpha * np.exp(-2.0 * t)
224     return np.array([
225         -(x[0] - r * x[1]) / (1.0 - r * r),
226         -(x[1] - r * x[0]) / (1.0 - r * r),
227     ])

```

B.4 AMP/TAP on the chain

Listing B.4: `amp.py` — per-node Gaussian closure: mean and variance iterations, damping, fixed-point existence check.

```

1  """
2  amp.py
3  =====
4
5  Gaussian message passing on the posterior precision of the AR(1)+OU model,
6  comparing three approximations of the marginals of a Gaussian graphical model
7
8       $p(a \mid x) \propto \exp(-1/2 a^T J a + h^T a),$ 
9
10 where (for our model, stationary zero-mean prior)
11
12      $J = (\exp(-2 t) / \Delta t) I + \Sigma_0^{-1}$       (posterior precision, tridiagonal)
13      $h = (\exp(-t) / \Delta t) x$                   (posterior field).
14
15 The exact marginals are mean =  $J^{-1} h$  and variances =  $\text{diag}(J^{-1})$ .
16
17 The point of this module is to make precise, and verify numerically, the
18 statement discussed with J. Garnier-Brun:
19
20 * The posterior MEAN solves the linear system  $J m = h$ . Mean-field, belief
21 propagation, and AMP are all iterative solvers of that *same* system, so
22 they share the *exact* mean -- hence the *same score*. (Confirms the
23 guess "BP in the Gaussian case recovers the right solution".)

```



```

24
25 * They differ only in the per-node VARIANCE estimate:
26     - mean field :  $V_i = 1 / J_{ii}$  (ignores neighbours)
27     - AMP / TAP :  $V_i = 1 / (J_{ii} - \sum_k J_{ik}^2 V_k)$  (cavity, no exclusion)
28     - BP / exact :  $V_i = (J^{-1})_{ii}$  (exact on a tree)
29     The AMP closure drops the "exclude one neighbour" correction. That is
30     exact when each node has many weak neighbours (a CLT over the incoming
31     messages -- the dense-graph regime AMP was built for), and only
32     approximate on the AR(1) chain, where every node has just two neighbours.
33
34 The dense-graph regime is used as a correctness gate in __main__: on a dense
35 random SPD  $J$  the AMP variance must match diag(J^{-1}); on the chain it must not.
36
37 References:
38     Mezard & Montanari, Information, Physics, and Computation (2009), TAP/cavity.
39     Zdeborova & Krzakala, Statistical Physics for Optimization and Learning,
40     EPFL Doctoral Lectures (2021), AMP / TAP equations.
41     Genovese & Piana, arXiv:2602.15191 (2026): BP means  $\rightarrow$  AMP asymptotically
42     on the dense (complete bipartite) factor graph.
43 """
44
45 from __future__ import annotations
46
47 import numpy as np
48
49 from ar1_utils import ar1_covariance, ar1_precision_clean, ou_params, prior_mean
50
51
52 # -----
53 # Posterior precision  $J$  and field  $h$  for the AR(1) + OU model
54 # -----
55
56 def posterior_precision_field(x: np.ndarray, t: float, alpha: float,
57                             sigma_eta: float | None = None,
58                             mu_0: float = 0.0
59                             ) -> tuple[np.ndarray, np.ndarray]:
60     """Return  $(J, h)$  of the Gaussian posterior  $p(a | x)$ .
61
62      $J = (\exp(-2t)/\Delta_t) I + \Sigma_0^{-1}$ ,  $h = (\exp(-t)/\Delta_t) x + \Sigma_0^{-1} \mu_a$ .
63
64     For the stationary zero-mean prior  $\mu_a = 0$  and  $h = (\exp(-t)/\Delta_t) x$ .
65      $J$  is tridiagonal because  $\Sigma_0^{-1}$  is tridiagonal and we add a multiple
66     of the identity.
67     """
68     x = np.asarray(x, dtype=float)
69     K = x.shape[0]
70     mu, Delta = ou_params(t)
71     Q0 = ar1_precision_clean(K, alpha, sigma_eta)
72     J = (mu * mu / Delta) * np.eye(K) + Q0

```

```

73     mu_a = prior_mean(K, alpha, mu_0)
74     h = (mu / Delta) * x + Q0 @ mu_a
75     return J, h
76
77
78 # -----
79 # Exact marginals
80 # -----
81
82 def exact_marginals(J: np.ndarray, h: np.ndarray
83                    ) -> tuple[np.ndarray, np.ndarray]:
84     """Exact (mean, variance) of  $p(a)$  propto  $\exp(-1/2 a^T J a + h^T a)$ ."""
85     Jinv = np.linalg.inv(J)
86     mean = Jinv @ h
87     var = np.diag(Jinv).copy()
88     return mean, var
89
90
91 # -----
92 # Shared mean iteration (the linear system  $J m = h$ )
93 # -----
94
95 def mean_iteration(J: np.ndarray, h: np.ndarray, damping: float = 0.5,
96                  max_iter: int = 10_000, tol: float = 1e-13
97                  ) -> tuple[np.ndarray, int, bool]:
98     """Damped Jacobi iteration whose fixed point is the exact mean  $J m = h$ .
99
100      $m_i \leftarrow (1 - g) m_i + g (h_i - \sum_{k \neq i} J_{ik} m_k) / J_{ii}$ .
101
102     Mean field, BP and AMP all have this same mean fixed point; only the
103     variance closure differs. Returned to demonstrate that the message-passing
104     mean converges to the exact posterior mean.
105     """
106     K = len(h)
107     d = np.diag(J).copy()
108     Off = J - np.diag(d)
109     m = np.zeros(K)
110     for it in range(1, max_iter + 1):
111         m_new = (h - Off @ m) / d
112         m_next = (1.0 - damping) * m + damping * m_new
113         if np.max(np.abs(m_next - m)) < tol:
114             return m_next, it, True
115         m = m_next
116     return m, max_iter, False
117
118
119 # -----
120 # Variance closures: mean field, AMP/TAP, exact
121 # -----
122

```

```

123 def mean_field_variance(J: np.ndarray) -> np.ndarray:
124     """Naive mean-field variance  $V_i = 1 / J_{ii}$  (neighbours ignored)."""
125     return 1.0 / np.diag(J)
126
127
128 def amp_variance(J: np.ndarray, max_iter: int = 10_000, tol: float = 1e-13,
129                 damping: float = 0.5) -> tuple[np.ndarray, int, bool]:
130     """AMP / TAP cavity variance, solved self-consistently:
131
132          $V_i = 1 / ( J_{ii} - \sum_{k \neq i} J_{ik}^2 V_k )$ .
133
134         This is the cavity variance with the "exclude one neighbour" correction
135         dropped ( $V_{\{k \rightarrow i\}}$  approximated by  $V_k$ ). Exact when each node has many
136         weak neighbours (dense regime); approximate on the chain, and -- when the
137         coupling is strong enough -- it has no physical (positive) fixed point at
138         all. In that case we report failure honestly (NaN, converged=False)
139         rather than a clipped meaningless number.
140     """
141     d = np.diag(J).copy()
142     W2 = J * J
143     np.fill_diagonal(W2, 0.0)
144     V = 1.0 / d
145     for it in range(1, max_iter + 1):
146         denom = d - W2 @ V
147         if np.any(denom <= 0):
148             # The self-consistent closure left the physical (SPD) region:
149             # no positive-variance fixed point exists for this coupling.
150             return np.full_like(V, np.nan), it, False
151         V_new = 1.0 / denom
152         V_next = (1.0 - damping) * V + damping * V_new
153         if np.max(np.abs(V_next - V)) < tol:
154             return V_next, it, True
155         V = V_next
156     return V, max_iter, False
157
158
159 # -----
160 # Convenience: marginals from each scheme
161 # -----
162
163 def amp_marginals(J: np.ndarray, h: np.ndarray
164                  ) -> tuple[np.ndarray, np.ndarray]:
165     """AMP marginals: exact mean ( $J m = h$ ) and AMP cavity variance."""
166     mean, _, _ = mean_iteration(J, h)
167     var, _, _ = amp_variance(J)
168     return mean, var
169
170
171 def amp_score(x: np.ndarray, t: float, alpha: float,
172              sigma_eta: float | None = None, mu_0: float = 0.0) -> np.ndarray:

```

```

173     """Joint score from the AMP posterior mean via Tweedie's identity."""
174     J, h = posterior_precision_field(x, t, alpha, sigma_eta, mu_0)
175     mean, _, _ = mean_iteration(J, h)
176     mu, Delta = ou_params(t)
177     return (mu * mean - np.asarray(x, dtype=float)) / Delta
178
179
180 # -----
181 # Correctness gate: dense vs chain
182 # -----
183
184 if __name__ == "__main__":
185     rng = np.random.default_rng(0)
186
187     # --- Gate 1: dense random SPD J -- AMP variance must match exact -----
188     N = 400
189     A = rng.standard_normal((N, N)) / np.sqrt(N)
190     J_dense = np.eye(N) + 0.5 * (A + A.T) / np.sqrt(2.0)
191     # shift to ensure SPD with a comfortable margin
192     J_dense += (abs(min(np.linalg.eigvalsh(J_dense))) + 0.5) * np.eye(N)
193     h_dense = rng.standard_normal(N)
194
195     m_ex, v_ex = exact_marginals(J_dense, h_dense)
196     m_it, n_it, ok_m = mean_iteration(J_dense, h_dense)
197     v_amp, n_v, ok_v = amp_variance(J_dense)
198     v_mf = mean_field_variance(J_dense)
199     print("=== Gate 1: dense random SPD J (N=%d) ===" % N)
200     print(f" mean iteration converged={ok_m} in {n_it} steps; "
201           f"max|m_iter - m_exact| = {np.max(np.abs(m_it - m_ex)):.2e}")
202     print(f" AMP variance converged={ok_v} in {n_v} steps; "
203           f"max|V_amp - V_exact| = {np.max(np.abs(v_amp - v_ex)):.2e} "
204           f"(should be small: AMP exact on dense)")
205     print(f" mean-field variance error = {np.max(np.abs(v_mf - v_ex)):.2e} "
206           f"(should be larger)")
207
208     # --- Gate 2: AR(1) chain -- AMP mean exact, AMP variance NOT exact -----
209     K, alpha = 9, 0.8
210     x = rng.standard_normal(K)
211     print(f"\n=== Gate 2: AR(1) chain (K={K}, alpha={alpha}) ===")
212     print(" mean is exact for every t (same score as BP); the AMP variance")
213     print(" closure is accurate at small t (evidence-dominated, weak coupling)")
214     print(" and degrades / breaks down at larger t (prior-dominated):")
215     for t in (0.05, 0.2, 0.5, 1.0, 2.0):
216         J_chain, h_chain = posterior_precision_field(x, t, alpha)
217         m_ex, v_ex = exact_marginals(J_chain, h_chain)
218         m_amp = mean_iteration(J_chain, h_chain)[0]
219         v_amp, _, ok = amp_variance(J_chain)
220         err_m = np.max(np.abs(m_amp - m_ex))
221         if ok:
222             err_v = f"{np.max(np.abs(v_amp - v_ex)):.2e}"

```

```

223     else:
224         err_v = "no physical fixed point (breakdown)"
225         print(f"    t={t:<4}    |m_amp-m_exact|={err_m:.1e}    AMP var: {err_v}")

```

B.5 Local (truncated-range) estimators

Listing B.5: `local_bp.py` — radius- r windowed inference and the deterministic RMS truncation error.

```

1  """
2  local_bp.py
3  =====
4
5  Local (truncated-range) message passing on the AR(1) + OU chain, to compare
6  with the full forward+backward belief propagation sweeps.
7
8  Question (from the meeting): what is the difference between running only
9  *local* messages -- a node listens to its immediate neighbourhood and ignores
10 the rest of the chain -- versus the full BP that propagates information all the
11 way along the chain through forward and backward sweeps?
12
13 Formalisation. The full BP posterior mean is
14
15      $E[a_k \mid x_0, \dots, x_{K-1}]$  (uses every frame).
16
17 The *local estimator of radius  $r$ * uses only the observations within distance  $r$ ,
18
19      $E[a_k \mid x_{k-r}, \dots, x_{k+r}]$  (the rest of the chain marginalised out).
20
21 Because any contiguous block of a stationary AR(1) chain is itself a stationary
22 AR(1) chain, this conditional expectation is computed exactly on the windowed
23 sub-chain -- no extra approximation beyond the truncation of range. Taking
24  $r \geq K-1$  makes the window the whole chain and recovers the exact full BP.
25
26 In the Gaussian case the full BP is exact, so the gap
27
28      $\| S_{\text{local}}^{\sim(r)}(x, t) - S_{\text{full}}(x, t) \|$ 
29
30 measures purely the error introduced by locality. It shrinks as  $r$  grows and as
31 the diffusion time  $t$  weakens the inter-frame coupling.
32 """
33
34 from __future__ import annotations
35
36 import numpy as np

```

```

37
38 from ar1_utils import ar1_covariance, ou_params
39
40
41 def local_posterior_mean(x: np.ndarray, t: float, alpha: float, radius: int,
42                        sigma_eta: float | None = None) -> np.ndarray:
43     """Return the radius-r local posterior mean  $E[a_k | x_{k-r..k+r}]$  for all  $k$ .
44
45     radius >= K-1 reproduces the exact full-chain posterior mean.
46     radius = 0 uses only the single local observation  $x_k$ .
47     """
48     x = np.asarray(x, dtype=float)
49     K = x.shape[0]
50     mu, Delta = ou_params(t)
51     ev_prec = mu * mu / Delta           # evidence precision in 'a' coordinates
52     ev_gain = mu / Delta                # maps  $x_j$  to the field  $h_j$ 
53
54     m = np.empty(K)
55     for k in range(K):
56         lo = max(0, k - radius)
57         hi = min(K - 1, k + radius)
58         idx = np.arange(lo, hi + 1)
59         k0 = k - lo                     # position of k inside the window
60         n = len(idx)
61
62         Sigma_sub = ar1_covariance(n, alpha, sigma_eta)
63         J = ev_prec * np.eye(n) + np.linalg.inv(Sigma_sub)
64         h = ev_gain * x[idx]
65         m_sub = np.linalg.solve(J, h)
66         m[k] = m_sub[k0]
67     return m
68
69
70 def local_score(x: np.ndarray, t: float, alpha: float, radius: int,
71               sigma_eta: float | None = None) -> np.ndarray:
72     """Joint score from the radius-r local posterior mean via Tweedie."""
73     mu, Delta = ou_params(t)
74     m = local_posterior_mean(x, t, alpha, radius, sigma_eta)
75     return (mu * m - np.asarray(x, dtype=float)) / Delta
76
77
78 # -----
79 # Exact (deterministic) RMS truncation error
80 # -----
81
82 def _smoother_matrix(K: int, alpha: float, t: float,
83                    sigma_eta: float | None = None) -> np.ndarray:
84     """The full-chain smoother  $C$  with  $E[a | x] = C x$  (linear, Gaussian)."""
85     from ar1_utils import ar1_precision_clean
86     mu, Delta = ou_params(t)

```

```

87     J = (mu * mu / Delta) * np.eye(K) + ar1_precision_clean(K, alpha, sigma_eta)
88     return np.linalg.solve(J, (mu / Delta) * np.eye(K))
89
90
91 def rms_truncation_error(K: int, alpha: float, t: float, k: int, radius: int,
92                          sigma_eta: float | None = None) -> float:
93     """Exact RMS error of the radius-r local estimator at frame k:
94
95         RMS_r(k) = sqrt( E_{x ~ P_t} [ ( E[a_k/x] - E[a_k/x_window] )^2 ] ).
96
97         Both estimators are LINEAR in x, so the error is e . x for a deterministic
98         row vector e, and the RMS over x ~ N(0, Sigma_t) is sqrt(e' Sigma_t e):
99         a closed-form deterministic quantity, no sampling involved.
100
101         The audit verifies that log RMS_r decays in r with slope log q, where
102         q = chain_formulas.bulk_correlation_decay(alpha, t): the locality error
103         inherits exactly the posterior correlation decay rate.
104     """
105     mu, Delta = ou_params(t)
106     Sigma_0 = ar1_covariance(K, alpha, sigma_eta)
107     Sigma_t = mu * mu * Sigma_0 + Delta * np.eye(K)
108
109     e = _smoother_matrix(K, alpha, t, sigma_eta)[k].copy()
110     lo, hi = max(0, k - radius), min(K - 1, k + radius)
111     Cw = _smoother_matrix(hi - lo + 1, alpha, t, sigma_eta)
112     e[lo:hi + 1] -= Cw[k - lo]
113     return float(np.sqrt(e @ Sigma_t @ e))
114
115
116 if __name__ == "__main__":
117     from ar1_utils import ar1_covariance as _cov, joint_score_matrix
118
119     rng = np.random.default_rng(3)
120     K, alpha = 21, 0.8
121     Sigma_0 = _cov(K, alpha)
122     x = rng.standard_normal(K)
123
124     print(f"AR(1) chain K={K}, alpha={alpha}: local radius-r score vs full BP")
125     print("  (full BP == exact -Sigma_t^-1 x; error is purely from locality)\n")
126     for t in (0.1, 0.5, 2.0):
127         s_full = joint_score_matrix(x, t, Sigma_0, alpha)
128         print(f"  t={t}")
129         for r in (0, 1, 2, 4, 8, K - 1):
130             s_loc = local_score(x, t, alpha, r)
131             err = np.max(np.abs(s_loc - s_full))
132             tag = "  (= full chain)" if r == K - 1 else ""
133             print(f"    radius r={r:<2} max|S_local - S_full| = {err:.2e}{tag}")
134     print()

```

B.6 Non-Gaussian chains: exact grid BP

Listing B.6: `bp_grid.py` — messages on an M -point grid over the interval from $-A$ to A , trapezoidal sum-product sweeps, exact reference scores for arbitrary innovation laws.

```

1  """Reference grid BP on a chain posterior.
2
3  Probability objects
4  -----
5  The posterior on a chain is
6
7       $p_t(a \mid x) \propto \mu(a_1) \prod_i K(a_i \mid a_{i-1}) \prod_i \ell_i(a_i; x_i)$ .
8
9  BP on this tree is *exact at the level of continuous message functions*; the only
10 approximation made here is the representation of those functions by their values
11 on an equally spaced grid  $u_1..u_M$  on  $[-A, A]$  with composite trapezoidal
12 quadrature. Two error sources follow:
13 (i) tail truncation: posterior mass outside  $[-A, A]$  is discarded;
14 (ii) quadrature:  $O(dx^2)$  trapezoid error, which becomes serious only when an
15 integrand (in practice the likelihood at small  $t$ ) is narrower than a few
16 grid steps.
17 Layer-2 experiments quantify both against the exactly solvable Gaussian case.
18
19 Messages are kept in the linear domain but renormalized to unit quadrature mass
20 after every step; the discarded constants are accumulated so that the *exact*
21 model evidence  $\log p_t(x)$  is also returned (testable against the closed-form
22 Gaussian marginal likelihood).
23 """
24
25 from __future__ import annotations
26
27 from dataclasses import dataclass
28
29 import numpy as np
30
31
32 def make_grid(half_width: float, size: int) -> tuple[np.ndarray, np.ndarray]:
33     """Equally spaced grid on  $[-A, A]$  with composite trapezoidal weights."""
34     grid = np.linspace(-half_width, half_width, size)
35     dx = float(grid[1] - grid[0])
36     weights = np.full(size, dx)
37     weights[0] *= 0.5
38     weights[-1] *= 0.5
39     return grid, weights
40
41
42 @dataclass(frozen=True)
43 class GridBPResult:

```



```

44     """Posterior summaries from grid BP.
45
46     means, variances : posterior marginal mean / variance per site (exact up to
47     grid error).
48     beliefs          : normalized posterior marginals on the grid, shape (n, M).
49     log_evidence      : log p_t(x), the chain marginal likelihood, including all
50     constants (likelihood normalizers and message rescalings).
51     """
52
53     means: np.ndarray
54     variances: np.ndarray
55     beliefs: np.ndarray
56     log_evidence: float
57
58
59 def grid_bp(
60     grid: np.ndarray,
61     weights: np.ndarray,
62     log_K: np.ndarray,
63     log_ell: np.ndarray,
64     x: np.ndarray,
65     alpha: float,
66     delta: float,
67     log_mu: np.ndarray | None = None,
68 ) -> GridBPResult:
69     """Sum-product BP with grid-represented messages on the chain posterior.
70
71     Parameters
72     -----
73     log_K    : log K[out, in] = log p(a_out | a_in) on the grid, shape (M, M).
74     log_ell  : row-shifted log-likelihood table from 'noising.log_likelihood_matrix',
75                 shape (n, M). Row shifts are undone in the evidence computation.
76     x        : observations (needed only to restore the exact evidence constants).
77     log_mu   : log initial density on the grid; defaults to standard normal.
78
79     Left message L_i excludes the local likelihood ell_i; right message R_i
80     likewise. Recursions (trapezoidal quadrature of the exact continuous ones):
81
82         L_{i+1} = K @ (L_i * ell_i * w),      R_{i-1} = K^T @ (R_i * ell_i * w).
83     """
84     n, m = log_ell.shape
85     if log_mu is None:
86         log_mu = -0.5 * grid**2 - 0.5 * np.log(2.0 * np.pi)
87
88     K = np.exp(log_K)
89     ell = np.exp(log_ell)
90
91     # Evidence bookkeeping: true log ell_i(a) = log_ell[i] + shift_i + gauss_norm,
92     # where shift_i is the row max removed in noising.log_likelihood_matrix.
93     z = x[:, None] - alpha * grid[None, :]

```

```

94     row_shift = (-0.5 * z**2 / delta).max(axis=1)
95     gauss_norm = -0.5 * np.log(2.0 * np.pi * delta)
96     log_const = float(np.sum(row_shift) + n * gauss_norm)
97
98     L = np.empty((n, m))
99     R = np.empty((n, m))
100    L[0] = np.exp(log_mu)
101    log_z_fwd = 0.0 # accumulated log of message rescaling constants
102
103    for i in range(n - 1):
104        incoming = L[i] * ell[i] * weights
105        out = K @ incoming
106        c = float(np.sum(out * weights))
107        if not np.isfinite(c) or c <= 0.0:
108            raise FloatingPointError(f"Forward message {i + 1} lost all mass.")
109        L[i + 1] = out / c
110        log_z_fwd += np.log(c)
111
112    R[-1] = np.ones(m)
113    for i in range(n - 1, 0, -1):
114        incoming = R[i] * ell[i] * weights
115        out = K.T @ incoming
116        c = float(np.sum(out * weights))
117        if not np.isfinite(c) or c <= 0.0:
118            raise FloatingPointError(f"Backward message {i - 1} lost all mass.")
119        R[i - 1] = out / c
120
121    beliefs = np.empty((n, m))
122    means = np.empty(n)
123    variances = np.empty(n)
124    for i in range(n):
125        raw = L[i] * ell[i] * R[i]
126        mass = float(np.sum(raw * weights))
127        if not np.isfinite(mass) or mass <= 0.0:
128            raise FloatingPointError(f"Belief {i} lost all mass.")
129        b = raw / mass
130        beliefs[i] = b
131        means[i] = float(np.sum(grid * b * weights))
132        variances[i] = float(np.sum((grid - means[i]) ** 2 * b * weights))
133
134    # Evidence: p(x) = (prod of forward rescale constants) * int L_n ell_n da,
135    # where L_n is the *normalized* forward message at the last site; restore the
136    # removed likelihood constants at the end.
137    tail = float(np.sum(L[-1] * ell[-1] * weights))
138    log_evidence = log_z_fwd + np.log(tail) + log_const
139
140    return GridBPResult(
141        means=means, variances=variances, beliefs=beliefs, log_evidence=log_evidence
142    )
143

```

```

144
145 def score_from_posterior_mean(
146     x: np.ndarray, means: np.ndarray, alpha: float, delta: float
147 ) -> np.ndarray:
148     """Exact OU identity  $s_i(x, t) = -(x_i - \alpha_t E[a_i | x]) / \Delta_t$ .
149
150     Feeding *approximate* posterior means through this identity produces an
151     approximate score whose error is exactly  $(\alpha/\delta)$  times the mean error.
152     """
153     return -(x - alpha * means) / delta
154
155
156 def grid_bp_batch(
157     grid: np.ndarray,
158     weights: np.ndarray,
159     log_K: np.ndarray,
160     X: np.ndarray,
161     alpha: float,
162     delta: float,
163     log_mu: np.ndarray | None = None,
164 ) -> tuple[np.ndarray, np.ndarray]:
165     """Vectorized grid BP over a batch of observations X of shape (B, n).
166
167     Same algorithm as 'grid_bp' (independent chains, one per batch row), with
168     the site loop shared and each message update a  $(M, M) @ (M, B)$  matmul.
169     Used by the reverse-dynamics experiments, where BP must run at every
170     integration step for every trajectory. Returns (means, variances), each of
171     shape (B, n). Evidence is not accumulated here.
172     """
173     b_size, n = X.shape
174     m = len(grid)
175     if log_mu is None:
176         log_mu = -0.5 * grid**2 - 0.5 * np.log(2.0 * np.pi)
177     K = np.exp(log_K)
178
179     # ell[i] has shape (B, M): row-shifted per (batch, site).
180     z = X[:, :, None] - alpha * grid[None, None, :]
181     log_ell = -0.5 * z**2 / delta
182     log_ell -= log_ell.max(axis=2, keepdims=True)
183     ell = np.exp(log_ell) # (B, n, M)
184
185     L = np.empty((n, b_size, m))
186     R = np.empty((n, b_size, m))
187     L[0] = np.exp(log_mu)[None, :]
188
189     for i in range(n - 1):
190         incoming = L[i] * ell[:, i, :] * weights[None, :] # (B, M)
191         out = (K @ incoming.T).T
192         mass = out @ weights
193         if not np.all(np.isfinite(mass)) or np.any(mass <= 0.0):

```

```

194         raise FloatingPointError(f"Batched forward message {i + 1} lost mass.")
195         L[i + 1] = out / mass[:, None]
196
197     R[-1] = 1.0
198     for i in range(n - 1, 0, -1):
199         incoming = R[i] * ell[:, i, :] * weights[None, :]
200         out = (K.T @ incoming.T).T
201         mass = out @ weights
202         if not np.all(np.isfinite(mass)) or np.any(mass <= 0.0):
203             raise FloatingPointError(f"Batched backward message {i - 1} lost mass.")
204         R[i - 1] = out / mass[:, None]
205
206     means = np.empty((b_size, n))
207     variances = np.empty((b_size, n))
208     for i in range(n):
209         raw = L[i] * ell[:, i, :] * R[i]
210         mass = raw @ weights
211         if not np.all(np.isfinite(mass)) or np.any(mass <= 0.0):
212             raise FloatingPointError(f"Batched belief {i} lost mass.")
213         bel = raw / mass[:, None]
214         means[:, i] = bel @ (grid * weights)
215         variances[:, i] = bel @ (grid**2 * weights) - means[:, i] ** 2
216     return means, variances

```

B.7 Non-Gaussian chains: Gaussian-projected BP

Listing B.7: `bp_gaussian_projected.py` — the same sweeps with every message moment-matched to a Gaussian: the closure under test in Section 9.5.

```

1  """Gaussian message passing on the chain posterior.
2
3  Two implementations with very different numerical status:
4
5  1. 'gaussian_chain_bp' (recommended): analytic information-form forward-backward.
6     Messages are parametrized by (precision  $\lambda \geq 0$ , information  $h$ ); a flat
7     message is exactly  $\lambda = 0$ . All updates are closed-form, no grid, no
8     truncation, no boundary pathologies.
9
10    * On a Gaussian  $AR(1)$  prior this is EXACT (Gaussian closure).
11    * On any *linear-transition* chain  $a_i = \rho a_{i-1} + \epsilon_i$  with
12       $\text{Var}(\epsilon) = q$  and Gaussian OU likelihoods, this coincides with
13      moment-matched assumed-density (single-Gaussian projected) BP:
14      the tilted product (Gaussian message  $\times$  Gaussian likelihood) is exactly
15      Gaussian, and the transition step maps  $N(m, v)$  to a density whose first two
16      moments are  $(\rho m, \rho^2 v + q)$  regardless of the innovation shape.

```

```

17     Consequently single-Gaussian ADF-BP == exact Gaussian BP on the
18     covariance-matched Gaussian model (equivalence proposition; verified in
19     tests). Its error on a non-Gaussian chain is therefore both the "Gaussian
20     message error" and the "Gaussian model error" -- they are the same object
21     at the single-Gaussian level.
22
23 2. 'grid_projected_gaussian_bp' (legacy, for the audit): the original package's
24     grid-mediated moment-matching projection. Kept verbatim in spirit to
25     reproduce and document its boundary-collapse instability at weakly
26     informative  $t$  (audit finding F1). Not used as a baseline in new experiments.
27 """
28
29 from __future__ import annotations
30
31 from dataclasses import dataclass
32
33 import numpy as np
34
35
36 @dataclass(frozen=True)
37 class GaussianBPResult:
38     means: np.ndarray
39     variances: np.ndarray
40     # Message parameters (prior-side, excluding the local likelihood):
41     L_prec: np.ndarray
42     L_info: np.ndarray
43     R_prec: np.ndarray
44     R_info: np.ndarray
45
46
47 def gaussian_chain_bp(
48     x: np.ndarray,
49     rho: float,
50     q: float,
51     alpha: float,
52     delta: float,
53     prior_var0: float = 1.0,
54 ) -> GaussianBPResult:
55     """Analytic information-form BP for the linear chain posterior.
56
57     Model:  $a_1 \sim N(0, \text{prior\_var0})$ ,  $a_i = \rho a_{i-1} + \text{innovation}(\text{var } q)$ ,
58      $x_i \mid a_i \sim N(\alpha a_i, \text{delta})$ .
59
60     The likelihood contributes precision  $\alpha^2/\text{delta}$  and information
61      $\alpha x_i / \text{delta}$  at each site. Forward step: absorb the local likelihood,
62     convert to moments, push through the transition  $(m, v) \rightarrow (\rho m,$ 
63      $\rho^2 v + q)$ . Backward step: absorb, then invert the transition
64      $(m, v) \rightarrow (m / \rho, (v + q) / \rho^2)$ .
65     """
66     n = len(x)

```

```

67     lam_lik = alpha**2 / delta
68     h_lik = alpha * x / delta
69
70     L_prec = np.empty(n)
71     L_info = np.empty(n)
72     R_prec = np.zeros(n)
73     R_info = np.zeros(n)
74
75     L_prec[0] = 1.0 / prior_var0
76     L_info[0] = 0.0
77
78     for i in range(n - 1):
79         lam_t = L_prec[i] + lam_lik
80         m_t = (L_info[i] + h_lik[i]) / lam_t
81         v_t = 1.0 / lam_t
82         v_next = rho**2 * v_t + q
83         L_prec[i + 1] = 1.0 / v_next
84         L_info[i + 1] = rho * m_t / v_next
85
86     # Terminal right message is flat: precision 0.
87     for i in range(n - 1, 0, -1):
88         lam_t = R_prec[i] + lam_lik
89         m_t = (R_info[i] + h_lik[i]) / lam_t
90         v_t = 1.0 / lam_t
91         v_prev = (v_t + q) / rho**2
92         R_prec[i - 1] = 1.0 / v_prev
93         R_info[i - 1] = (m_t / rho) / v_prev
94
95     lam_b = L_prec + R_prec + lam_lik
96     h_b = L_info + R_info + h_lik
97     means = h_b / lam_b
98     variances = 1.0 / lam_b
99     return GaussianBPResult(
100         means=means,
101         variances=variances,
102         L_prec=L_prec,
103         L_info=L_info,
104         R_prec=R_prec,
105         R_info=R_info,
106     )
107
108
109     # -----
110     # Legacy grid-projected variant (audit finding F1) -- kept for diagnosis only.
111     # -----
112
113     def _density_moments(
114         values: np.ndarray, grid: np.ndarray, weights: np.ndarray
115     ) -> tuple[float, float]:
116         values = np.maximum(values, 0.0)

```

```

117     mass = float(np.sum(values * weights))
118     if not np.isfinite(mass) or mass <= 0.0:
119         raise FloatingPointError("Cannot normalize projected message.")
120     p = values / mass
121     mean = float(np.sum(grid * p * weights))
122     var = float(np.sum((grid - mean) ** 2 * p * weights))
123     return mean, max(var, 1e-12)
124
125
126 def _gaussian_values(grid: np.ndarray, mean: float, var: float) -> np.ndarray:
127     if np.isinf(var):
128         return np.ones_like(grid)
129     return np.exp(-0.5 * (grid - mean) ** 2 / var) / np.sqrt(2.0 * np.pi * var)
130
131
132 def grid_projected_gaussian_bp(
133     grid: np.ndarray,
134     weights: np.ndarray,
135     log_K: np.ndarray,
136     log_ell: np.ndarray,
137 ) -> tuple[np.ndarray, np.ndarray]:
138     """Original grid-mediated Gaussian projection (means, variances).
139
140     Each message is evaluated on the grid, pushed through the exact grid update,
141     then moment-matched. This is mathematically the same projection as
142     'gaussian_chain_bp' for linear transitions, but its *numerics* differ: when
143     the outgoing message function is a near-flat ramp with maximizer outside
144     [-A, A] (weakly informative t), moment-matching the truncated function
145     collapses the message onto the grid boundary. Retained to demonstrate the
146     artifact; do not use as a scientific baseline.
147     """
148     n, m = log_ell.shape
149     K = np.exp(log_K)
150     ell = np.exp(log_ell)
151
152     L_mean = np.empty(n)
153     L_var = np.empty(n)
154     R_mean = np.empty(n)
155     R_var = np.empty(n)
156     L_mean[0], L_var[0] = 0.0, 1.0
157     R_mean[-1], R_var[-1] = 0.0, np.inf
158
159     for i in range(n - 1):
160         incoming = _gaussian_values(grid, L_mean[i], L_var[i]) * ell[i] * weights
161         L_mean[i + 1], L_var[i + 1] = _density_moments(K @ incoming, grid, weights)
162
163     for i in range(n - 1, 0, -1):
164         incoming = _gaussian_values(grid, R_mean[i], R_var[i]) * ell[i] * weights
165         R_mean[i - 1], R_var[i - 1] = _density_moments(K.T @ incoming, grid, weights)
166

```

```

167 means = np.empty(n)
168 variances = np.empty(n)
169 for i in range(n):
170     b = (
171         _gaussian_values(grid, L_mean[i], L_var[i])
172         * ell[i]
173         * _gaussian_values(grid, R_mean[i], R_var[i])
174     )
175     means[i], variances[i] = _density_moments(b, grid, weights)
176 return means, variances

```

B.8 Exact scores for validation

Listing B.8: `exact_scores.py` — closed-form Gaussian-chain score used to calibrate the grid solver.

```

1  """Closed-form scores and posterior means for Gaussian priors.
2
3  For any Gaussian clean prior  $a \sim N(0, \text{Sigma}_0)$  under the OU kernel,
4
5       $x_t \sim N(0, \text{Sigma}_t), \quad \text{Sigma}_t = \alpha_t^2 \text{Sigma}_0 + \Delta_t I,$ 
6       $s_{\text{true}}(x, t) = -\text{Sigma}_t^{-1} x$  (exact score),
7       $m_{\text{true}}(x, t) = \alpha_t \text{Sigma}_0 \text{Sigma}_t^{-1} x$  (exact posterior mean),
8       $\log p_t(x) = \log \text{pdf of } N(0, \text{Sigma}_t) \text{ at } x$  (exact evidence).
9
10 These are the calibration ground truths for Layers 2 and 4. Everything uses
11 'solve' (never explicit inverses) for numerical hygiene.
12 """
13
14 from __future__ import annotations
15
16 import numpy as np
17
18
19 def sigma_t(sigma0: np.ndarray, alpha: float, delta: float) -> np.ndarray:
20     n = sigma0.shape[0]
21     return alpha**2 * sigma0 + delta * np.eye(n)
22
23
24 def precision_matrix_score(
25     x: np.ndarray, sigma0: np.ndarray, alpha: float, delta: float
26 ) -> np.ndarray:
27     """Exact Gaussian score  $s = -\text{Sigma}_t^{-1} x$ ."""
28     return -np.linalg.solve(sigma_t(sigma0, alpha, delta), x)
29

```



```
30
31 def exact_gaussian_posterior_mean(
32     x: np.ndarray, sigma0: np.ndarray, alpha: float, delta: float
33 ) -> np.ndarray:
34     """Exact Gaussian posterior mean  $E[a | x] = \alpha \Sigma_0 \Sigma_t^{-1} x$ ."""
35     return alpha * sigma0 @ np.linalg.solve(sigma_t(sigma0, alpha, delta), x)
36
37
38 def gaussian_log_evidence(
39     x: np.ndarray, sigma0: np.ndarray, alpha: float, delta: float
40 ) -> float:
41     """log p_t(x) for the Gaussian prior (multivariate normal logpdf)."""
42     st = sigma_t(sigma0, alpha, delta)
43     n = len(x)
44     sign, logdet = np.linalg.slogdet(st)
45     if sign <= 0:
46         raise np.linalg.LinAlgError("Sigma_t is not positive definite.")
47     quad = float(x @ np.linalg.solve(st, x))
48     return -0.5 * (n * np.log(2.0 * np.pi) + logdet + quad)
49
50
51 def score_batch(
52     X: np.ndarray, sigma0: np.ndarray, alpha: float, delta: float
53 ) -> np.ndarray:
54     """Exact scores for a batch of observations, X of shape (B, n)."""
55     return -np.linalg.solve(sigma_t(sigma0, alpha, delta), X.T).T
```

Appendix C

Reproducibility

C.1 Repository layout

All source material of this thesis lives in a single repository, organised as follows (the thesis directory is self-contained: the listed code and every figure file are committed alongside the $\text{\LaTeX}$ source).

Path	Content
<code>thesis/</code>	this document: $\text{\LaTeX}$ source, figures, code listings
<code>research/gaussian-bp/</code>	Gaussian chain note, code, notebooks, 72-check audit
<code>research/bp-from-scratch/</code>	independent BP derivation and implementation
<code>research/unified-note/</code>	consolidated working note, code, 53-check audit
<code>research/nongaussian-bp/</code>	grid BP vs Gaussian-projected BP experiments
<code>research/gaussian-ar1-bp/</code>	Gaussian AR(1) package (BP = matrix score)
<code>research/bp-generalization/</code>	general non-Gaussian BP formulation notes
<code>research/session-summaries/</code>	source working notes of the project
<code>research/initial-experiments/</code>	early diffusion-setup experiments and figures

C.2 Audit protocol

Every closed-form claim is verified by a deterministic check against an independently computed reference (brute-force linear algebra, dense grid quadrature, or finite differences), collected in audit drivers that exit non-zero on any failure:

- `research/gaussian-bp/code/numerical_audit.py` — 72 checks covering: Σ_t assembly and inversion; Tweedie vs matrix score; the $K = 2$ closed forms; Convention-A BP vs matrix posterior (means, variances, scores); bulk variance, covariance decay q^d , and the recombination identity; AMP mean/score closure-independence; the AMP variance formula, existence boundary, breakdown time, and weak-coupling law; the locality slopes; the band-fill coefficients (factorial version rejected); spectral preservation and large- t return.
- `research/unified-note/code/numerical_audit.py` — 55 checks covering the consolidated note, including the Laplace $K = 1$ quadrature checks (Tweedie vs finite differences; characteristic function vs direct integration) and the one-step boundary-message identity.
- `research/nongaussian-bp/` — experiment drivers `exp_01–exp_05` with logged master-error-identity residuals (10^{-15} – 10^{-12} throughout), grid self-convergence gates, and per-trial CSV outputs; the grid calibration of Section 9.5 is experiment 01.

All checks were re-executed on the build machine at the time of writing: 72/72 and 55/55 pass on the audit drivers, and all experiment gates are green.

C.3 Figure-to-code map

Figure	File	Generated by
7.1	fig_spectral.png	gaussian-bp/code/experiments.py
7.2	fig_precision_lifecycle.png	gaussian-bp/code/experiments.py
7.3	fig_band_fill.png	gaussian-bp/code/experiments.py
7.4	fig_tridiag_loss.png	gaussian-bp/code/experiments.py
7.5	fig_local_vs_full.png	gaussian-bp/code/experiments.py
8.1	fig06_K1_density.png	unified-note/code
8.2	fig07_K1_score.png	unified-note/code
8.3	fig08_K1_score_limits.png	unified-note/code
9.1	fig_bulk_variance.png	gaussian-bp/code/experiments.py
9.2	fig_bp_vs_amp.png	gaussian-bp/code/experiments.py
9.3	laplace_closure_vs_grid_budget.png	nongaussian-bp/experiments/exp_02

C.4 Environment

Python ≥ 3.10 with NumPy, SciPy, and Matplotlib; no GPU, no neural network framework, no stochastic training. All experiments are deterministic given the seeds recorded in the experiment configurations; the error measures of Sections 7.5.1 and 9.5.3 are seed-free by construction (closed-form traces). This document compiles with a single run of `tectonic main.tex`.

Bibliography

- Ackley, D. H., Hinton, G. E., and Sejnowski, T. J. (1985). A learning algorithm for Boltzmann machines. *Cognitive Science*, 9(1):147–169.
- Anderson, B. D. O. (1982). Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326.
- Besag, J. (1974). Spatial interaction and the statistical analysis of lattice systems. *Journal of the Royal Statistical Society: Series B*, 36(2):192–225.
- Biroli, G., Bonnaire, T., De Bortoli, V., and Mézard, M. (2024). Dynamical regimes of diffusion models. *Nature Communications*, 15:9957.
- Biroli, G. and Mézard, M. (2023). Generative diffusion in very large dimensions. *Journal of Statistical Mechanics: Theory and Experiment*, 2023(9):093402.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer, New York.
- Brockwell, P. J. and Davis, R. A. (1991). *Time Series: Theory and Methods*. Springer, New York, 2 edition.
- Donoho, D. L., Maleki, A., and Montanari, A. (2009). Message-passing algorithms for compressed sensing. *Proceedings of the National Academy of Sciences*, 106(45):18914–18919.
- Edwards, S. F. and Anderson, P. W. (1975). Theory of spin glasses. *Journal of Physics F: Metal Physics*, 5(5):965–974.
- Efron, B. (2011). Tweedie’s formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614.

- Gardiner, C. W. (2009). *Stochastic Methods: A Handbook for the Natural and Social Sciences*. Springer, Berlin, 4 edition.
- Geman, S. and Geman, D. (1984). Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 6(6):721–741.
- Genovese, G. and Piana, A. (2026). Derivation of the AMP equations from belief propagation for the ℓ_2 minimisation problem. arXiv:2602.15191 [math.PR]. <https://arxiv.org/abs/2602.15191> [last access: 14/07/26].
- Ghio, D., Dandi, Y., Krzakala, F., and Zdeborová, L. (2024). Sampling with flows, diffusion, and autoregressive neural networks from a spin-glass perspective. *Proceedings of the National Academy of Sciences*, 121(27):e2311810121.
- Goldstein, H., Poole, C. P., and Safko, J. L. (2002). *Classical Mechanics*. Addison-Wesley, San Francisco, 3 edition.
- Hastings, W. K. (1970). Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, 57(1):97–109.
- Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851.
- Hyvärinen, A. (2005). Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709.
- Ising, E. (1925). Beitrag zur Theorie des Ferromagnetismus. *Zeitschrift für Physik*, 31(1):253–258.
- Itô, K. (1944). Stochastic integral. *Proceedings of the Imperial Academy, Tokyo*, 20(8):519–524.
- Jaynes, E. T. (1957). Information theory and statistical mechanics. *Physical Review*, 106(4):620–630.
- Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal*

- of Basic Engineering*, 82(1):35–45.
- Karras, T., Aittala, M., Aila, T., and Laine, S. (2022). Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 26565–26577.
- Koller, D. and Friedman, N. (2009). *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, MA.
- Kschischang, F. R., Frey, B. J., and Loeliger, H.-A. (2001). Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519.
- Landau, L. D. and Lifshitz, E. M. (1980). *Statistical Physics, Part 1*. Pergamon Press, Oxford, 3 edition.
- LeCun, Y., Chopra, S., Hadsell, R., Ranzato, M., and Huang, F. J. (2006). A tutorial on energy-based learning. In Bakır, G., Hofmann, T., Schölkopf, B., Smola, A. J., Taskar, B., and Vishwanathan, S., editors, *Predicting Structured Data*. MIT Press, Cambridge, MA.
- Malioutov, D. M., Johnson, J. K., and Willsky, A. S. (2006). Walk-sums and belief propagation in Gaussian graphical models. *Journal of Machine Learning Research*, 7:2031–2064.
- Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., and Teller, E. (1953). Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092.
- Mézard, M. and Montanari, A. (2009). *Information, Physics, and Computation*. Oxford University Press, Oxford.
- Mézard, M., Parisi, G., and Virasoro, M. A. (1987). *Spin Glass Theory and Beyond*. World Scientific, Singapore.
- Miyasawa, K. (1961). An empirical Bayes estimator of the mean of a normal population. *Bulletin of the International Statistical Institute*, 38(4):181–188.
- Øksendal, B. (2003). *Stochastic Differential Equations: An Introduction with Applications*.

- Springer, Berlin, 6 edition.
- Onsager, L. (1944). Crystal statistics. I. A two-dimensional model with an order-disorder transition. *Physical Review*, 65(3-4):117–149.
- Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, San Mateo, CA.
- Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286.
- Rauch, H. E., Tung, F., and Striebel, C. T. (1965). Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450.
- Raya, G. and Ambrogioni, L. (2023). Spontaneous symmetry breaking in generative diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.
- Risken, H. (1996). *The Fokker–Planck Equation: Methods of Solution and Applications*. Springer, Berlin, 2 edition.
- Robbins, H. (1956). An empirical Bayes approach to statistics. In *Proceedings of the Third Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 157–163.
- Robert, C. P. and Casella, G. (2004). *Monte Carlo Statistical Methods*. Springer, New York, 2 edition.
- Sherrington, D. and Kirkpatrick, S. (1975). Solvable model of a spin-glass. *Physical Review Letters*, 35(26):1792–1796.
- Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., and Ganguli, S. (2015). Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 2256–2265.
- Song, Y. (2021). Generative modeling by estimating gradients of the data distribution (blog post). <https://yang-song.net/blog/2021/score/> [last access: 14/07/26].

- Song, Y. and Ermon, S. (2019). Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. (2021). Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*.
- Thouless, D. J., Anderson, P. W., and Palmer, R. G. (1977). Solution of ‘Solvable model of a spin glass’. *Philosophical Magazine*, 35(3):593–601.
- Uhlenbeck, G. E. and Ornstein, L. S. (1930). On the theory of the Brownian motion. *Physical Review*, 36(5):823–841.
- Vincent, P. (2011). A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674.
- Weiss, Y. and Freeman, W. T. (2001). Correctness of belief propagation in Gaussian graphical models of arbitrary topology. *Neural Computation*, 13(10):2173–2200.
- Zdeborová, L. and Krzakala, F. (2016). Statistical physics of inference: thresholds and algorithms. *Advances in Physics*, 65(5):453–552.
- Zdeborová, L. and Krzakala, F. (2021). Statistical physics methods in optimization and machine learning. EPFL doctoral lecture notes. <https://sphinxteam.github.io/EPFLDoctoralLecture2021/Notes.pdf> [last access: 14/07/26].